Security in .NET

Article • 09/15/2021

The common language runtime and .NET provide many useful classes and services that enable developers to write secure code, use cryptography, and implement role-based security.

In this section

- Key Security Concepts
  Provides an overview of common language runtime security features.
- Role-Based Security
  Describes how to interact with role-based security in your code.
- Cryptography Model
  Provides an overview of cryptographic services provided by .NET.
- Secure Coding Guidelines
  Describes some of the best practices for creating reliable .NET applications.

Related sections

Development Guide
Provides a guide to all key technology areas and tasks for application development, including creating, configuring, debugging, securing, and deploying your application, and information about dynamic programming, interoperability, extensibility, memory management, and threading.

Key Security Concepts

Article • 05/26/2022

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see Overview of ASP.NET Core Security.

.NET offers role-based security to help address security concerns about mobile code and to provide support that enables components to determine what users are authorized to do.

Type safety and security

Type-safe code accesses only the memory locations it is authorized to access. (For this discussion, type safety specifically refers to memory type safety and should not be confused with type safety in a broader respect.) For example, type-safe code cannot read values from another object's private fields. It accesses types only in well-defined, allowable ways.

During just-in-time (JIT) compilation, an optional verification process examines the metadata and common intermediate language (CIL) of a method to be JIT-compiled into native machine code to verify that they are type safe. This process is skipped if the code has permission to bypass verification. For more information about verification, see Managed Execution Process.

Although verification of type safety is not mandatory to run managed code, type safety plays a crucial role in assembly isolation and security enforcement. When code is type safe, the common language runtime can completely isolate assemblies from each other. This isolation helps ensure that assemblies cannot adversely affect each other and it increases application reliability. Type-safe components can execute safely in the same process even if they are trusted at different levels. When code is not type safe, unwanted side effects can occur. For example, the runtime cannot prevent managed code from calling into native (unmanaged) code and performing malicious operations. When code is type safe, the runtime's security enforcement mechanism ensures that it does not access native code unless it has permission to do so. All code that is not type safe must have been granted SecurityPermission with the passed enum member SkipVerification to run.

> **Note**
>
> Code Access Security (CAS) has been deprecated across all versions of .NET Framework and .NET. Recent versions of .NET do not honor CAS annotations and produce errors if CAS-related APIs are used. Developers should seek alternative means of accomplishing security tasks.

Principal

A principal represents the identity and role of a user and acts on the user's behalf. Role-based security in .NET supports three kinds of principals:

- Generic principals represent users and roles that exist independent of Windows users and roles.
- Windows principals represent Windows users and their roles (or their Windows groups). A Windows principal can impersonate another user, which means that the principal can access a resource on a user's behalf while presenting the identity that belongs to that user.
- Custom principals can be defined by an application in any way that is needed for that particular application. They can extend the basic notion of the principal's identity and roles.

For more information, see Principal and Identity Objects.

Authentication

Authentication is the process of discovering and verifying the identity of a principal by examining the user's credentials and validating those credentials against some authority. The information obtained during authentication is directly usable by your code. You can also use .NET role-based security to authenticate the current user and to determine whether to allow that principal to access your code. See the overloads of the WindowsPrincipal.IsInRole method for examples of how to authenticate the principal for specific roles. For example, you can use the WindowsPrincipal.IsInRole(String) overload to determine if the current user is a member of the Administrators group.

A variety of authentication mechanisms are used today, many of which can be used with .NET role-based security. Some of the most commonly used mechanisms are basic, digest, Passport, operating system (such as NTLM or Kerberos), or application-defined mechanisms.

Example

The following example requires that the active principal be an administrator. The `name` parameter is `null`, which allows any user who is an administrator to pass the demand.

> **Note**
>
> In Windows Vista, User Account Control (UAC) determines the privileges of a user. If you are a member of the Built-in Administrators group, you are assigned two run-time access tokens: a standard user access token and an administrator access token. By default, you are in the standard user role. To execute the code that requires you to be an administrator, you must first elevate your privileges from standard user to administrator. You can do this when you start an application by right-clicking the application icon and indicating that you want to run as an administrator.

C#

```
using System;
using System.Threading;
using System.Security.Permissions;
using System.Security.Principal;

class SecurityPrincipalDemo
{

    public static void Main()
    {

AppDomain.CurrentDomain.SetPrincipalPolicy(PrincipalPolicy.WindowsPrincipal)
;
        PrincipalPermission principalPerm = new PrincipalPermission(null,
"Administrators");
        principalPerm.Demand();
        Console.WriteLine("Demand succeeded.");
    }
}
```

The following example demonstrates how to determine the identity of the principal and the roles available to the principal. An application of this example might be to confirm that the current user is in a role you allow for using your application.

C#

```
using System;
using System.Threading;
```

```
using System.Security.Permissions;
using System.Security.Principal;

class SecurityPrincipalDemo
{
    public static void DemonstrateWindowsBuiltInRoleEnum()
    {
        AppDomain myDomain = Thread.GetDomain();

        myDomain.SetPrincipalPolicy(PrincipalPolicy.WindowsPrincipal);
        WindowsPrincipal myPrincipal =
(WindowsPrincipal)Thread.CurrentPrincipal;
        Console.WriteLine($"{myPrincipal.Identity.Name.ToString()} belongs
to: ");
        Array wbirFields = Enum.GetValues(typeof(WindowsBuiltInRole));
        foreach (object roleName in wbirFields)
        {
            try
            {
                // Cast the role name to a RID represented by the
WindowsBuildInRole value.
                Console.WriteLine($"{roleName}?
{myPrincipal.IsInRole((WindowsBuiltInRole)roleName)}.");
                Console.WriteLine("The RID for this role is: " +
((int)roleName).ToString());
            }
            catch (Exception)
            {
                Console.WriteLine($"{roleName}: Could not obtain role for
this RID.");
            }
        }
        // Get the role using the string value of the role.
        Console.WriteLine($"'Administrators'?
{myPrincipal.IsInRole("BUILTIN\\" + "Administrators")}.");
        Console.WriteLine($"'Users'? {myPrincipal.IsInRole("BUILTIN\\" +
"Users")}.");
        // Get the role using the WindowsBuiltInRole enumeration value.
        Console.WriteLine($"{WindowsBuiltInRole.Administrator}?
{myPrincipal.IsInRole(WindowsBuiltInRole.Administrator)}.");
        // Get the role using the WellKnownSidType.
        SecurityIdentifier sid = new
SecurityIdentifier(WellKnownSidType.BuiltinAdministratorsSid, null);
        Console.WriteLine($"WellKnownSidType BuiltinAdministratorsSid
{sid.Value}? {myPrincipal.IsInRole(sid)}.");
    }

    public static void Main()
    {
        DemonstrateWindowsBuiltInRoleEnum();
    }
}
```

Authorization

Authorization is the process of determining whether a principal is allowed to perform a requested action. Authorization occurs after authentication and uses information about the principal's identity and roles to determine what resources the principal can access. You can use .NET role-based security to implement authorization.

See also

- ASP.NET Core Security

.NET Core Federal Information Processing Standard (FIPS) compliance

Article • 09/15/2021

The Federal Information Processing Standard (FIPS) Publication 140-2 is a U.S. government standard that defines minimum security requirements for cryptographic modules in information technology products, as defined in Section 5131 of the Information Technology Management Reform Act of 1996.

.NET Core:

- Passes cryptographic primitives calls through to the standard modules the underlying operating system provides.
- Does **not** enforce the use of FIPS Approved algorithms or key sizes in .NET Core apps.

The system administrator is responsible for configuring the FIPS compliance for an operating system.

If code is written for a FIPS-compliant environment, the developer is responsible for ensuring that non-compliant FIPS algorithms aren't used.

For more information on FIPS compliance, see the following articles:

- Windows FIPS Compliance
- Configuring Windows for FIPS Compliance
- Chapter 8. Federal Standards and Regulations Red Hat Enterprise Linux 7

Role-Based Security

Article • 09/15/2021

Roles are often used in financial or business applications to enforce policy. For example, an application might impose limits on the size of the transaction being processed depending on whether the user making the request is a member of a specified role. Clerks might have authorization to process transactions that are less than a specified threshold, supervisors might have a higher limit, and vice-presidents might have a still higher limit (or no limit at all). Role-based security can also be used when an application requires multiple approvals to complete an action. Such a case might be a purchasing system in which any employee can generate a purchase request, but only a purchasing agent can convert that request into a purchase order that can be sent to a supplier.

.NET role-based security supports authorization by making information about the principal, which is constructed from an associated identity, available to the current thread. The identity (and the principal it helps to define) can be either based on a Windows account or be a custom identity unrelated to a Windows account. .NET applications can make authorization decisions based on the principal's identity or role membership, or both. A role is a named set of principals that have the same privileges with respect to security (such as a teller or a manager). A principal can be a member of one or more roles. Therefore, applications can use role membership to determine whether a principal is authorized to perform a requested action.

To provide ease of use and consistency with code access security, .NET role-based security provides System.Security.Permissions.PrincipalPermission objects that enable the common language runtime to perform authorization in a way that is similar to code access security checks. The PrincipalPermission class represents the identity or role that the principal must match and is compatible with both declarative and imperative security checks. You can also access a principal's identity information directly and perform role and identity checks in your code when needed.

.NET provides role-based security support that is flexible and extensible enough to meet the needs of a wide spectrum of applications. You can choose to interoperate with existing authentication infrastructures, such as COM+ 1.0 Services, or to create a custom authentication system. Role-based security is particularly well-suited for use in ASP.NET Web applications, which are processed primarily on the server. However, .NET role-based security can be used on either the client or the server.

Before reading this section, make sure that you understand the material presented in Key Security Concepts.

See also

- Principal and Identity Objects
- Key Security Concepts
- System.Security.Permissions.PrincipalPermission
- ASP.NET Core Security

Principal and Identity Objects

Article • 09/15/2021

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see ASP.NET Core Security.

Managed code can discover the identity or the role of a principal through a IPrincipal object, which contains a reference to an IIdentity object. It might be helpful to compare identity and principal objects to familiar concepts like user and group accounts. In most network environments, user accounts represent people or programs, while group accounts represent certain categories of users and the rights they possess. Similarly, .NET identity objects represent users, while roles represent memberships and security contexts. In .NET, the principal object encapsulates both an identity object and a role. .NET applications grant rights to the principal based on its identity or, more commonly, its role membership.

Identity Objects

The identity object encapsulates information about the user or entity being validated. At their most basic level, identity objects contain a name and an authentication type. The name can either be a user's name or the name of a Windows account, while the authentication type can be either a supported logon protocol, such as Kerberos V5, or a custom value. .NET defines a GenericIdentity object that can be used for most custom logon scenarios and a more specialized WindowsIdentity object that can be used when you want your application to rely on Windows authentication. Additionally, you can define your own identity class that encapsulates custom user information.

The IIdentity interface defines properties for accessing a name and an authentication type, such as Kerberos V5 or NTLM. All **Identity** classes implement the **IIdentity** interface. There is no required relationship between an **Identity** object and the Windows process token under which a thread is currently executing. However, if the **Identity** object is a **WindowsIdentity** object, the identity is assumed to represent a Windows security token.

Principal Objects

The principal object represents the security context under which code is running. Applications that implement role-based security grant rights based on the role associated with a principal object. Similar to identity objects, .NET provides a GenericPrincipal object and a WindowsPrincipal object. You can also define your own custom principal classes.

The IPrincipal interface defines a property for accessing an associated **Identity** object as well as a method for determining whether the user identified by the **Principal** object is a member of a given role. All **Principal** classes implement the **IPrincipal** interface as well as any additional properties and methods that are necessary. For example, the common language runtime provides the **WindowsPrincipal** class, which implements additional functionality for mapping group membership to roles.

A **Principal** object is bound to a call context (CallContext) object within an application domain (AppDomain). A default call context is always created with each new **AppDomain**, so there is always a call context available to accept the **Principal** object. When a new thread is created, a **CallContext** object is also created for the thread. The **Principal** object reference is automatically copied from the creating thread to the new thread's **CallContext**. If the runtime cannot determine which **Principal** object belongs to the creator of the thread, it follows the default policy for **Principal** and **Identity** object creation.

A configurable application domain-specific policy defines the rules for deciding what type of **Principal** object to associate with a new application domain. Where security policy permits, the runtime can create **Principal** and **Identity** objects that reflect the operating system token associated with the current thread of execution. By default, the runtime uses **Principal** and **Identity** objects that represent unauthenticated users. The runtime does not create these default **Principal** and **Identity** objects until the code attempts to access them.

Trusted code that creates an application domain can set the application domain policy that controls construction of the default **Principal** and **Identity** objects. This application domain-specific policy applies to all execution threads in that application domain. An unmanaged, trusted host inherently has the ability to set this policy, but managed code that sets this policy must have the System.Security.Permissions.SecurityPermission for controlling domain policy.

When transmitting a **Principal** object across application domains but within the same process (and therefore on the same computer), the remoting infrastructure copies a reference to the **Principal** object associated with the caller's context to the callee's context.

See also

- How to: Create a WindowsPrincipal Object
- How to: Create GenericPrincipal and GenericIdentity Objects
- Replacing a Principal Object
- Impersonating and Reverting
- Role-Based Security
- Key Security Concepts
- ASP.NET Core Security

How to: Create a WindowsPrincipal Object

Article • 09/15/2021

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see ASP.NET Core Security.

There are two ways to create a WindowsPrincipal object, depending on whether code must repeatedly perform role-based validation or must perform it only once.

If code must repeatedly perform role-based validation, the first of the following procedures produces less overhead. When code needs to make role-based validations only once, you can create a WindowsPrincipal object by using the second of the following procedures.

To create a WindowsPrincipal object for repeated validation

1. Call the SetPrincipalPolicy method on the AppDomain object that is returned by the static AppDomain.CurrentDomain property, passing the method a PrincipalPolicy enumeration value that indicates what the new policy should be. Supported values are NoPrincipal, UnauthenticatedPrincipal, and WindowsPrincipal. The following code demonstrates this method call.

   C#

   ```
   AppDomain.CurrentDomain.SetPrincipalPolicy(
       PrincipalPolicy.WindowsPrincipal);
   ```

2. With the policy set, use the static Thread.CurrentPrincipal property to retrieve the principal that encapsulates the current Windows user. Because the property return type is IPrincipal, you must cast the result to a WindowsPrincipal type. The following code initializes a new WindowsPrincipal object to the value of the principal associated with the current thread.

   C#

```
WindowsPrincipal myPrincipal =
    (WindowsPrincipal) Thread.CurrentPrincipal;
```

3. When the principal object has been created, you can use one of several methods to validate it.

To create a WindowsPrincipal object for a single validation

1. Initialize a new WindowsIdentity object by calling the static WindowsIdentity.GetCurrent method, which queries the current Windows account and places information about that account into the newly created identity object. The following code creates a new WindowsIdentity object and initializes it to the current authenticated user.

```
C#

WindowsIdentity myIdentity = WindowsIdentity.GetCurrent();
```

2. Create a new WindowsPrincipal object and pass it the value of the WindowsIdentity object created in the preceding step.

```
C#

WindowsPrincipal myPrincipal = new WindowsPrincipal(myIdentity);
```

3. When the principal object has been created, you can use one of several methods to validate it.

See also

- Principal and Identity Objects
- ASP.NET Core Security

How to: Create GenericPrincipal and GenericIdentity Objects

Article • 09/15/2021

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see Overview of ASP.NET Core Security.

You can use the GenericIdentity class in conjunction with the GenericPrincipal class to create an authorization scheme that exists independent of a Windows domain.

To create a GenericPrincipal object

1. Create a new instance of the identity class and initialize it with the name you want it to hold. The following code creates a new **GenericIdentity** object and initializes it with the name `MyUser`.

   ```
   C#

   GenericIdentity myIdentity = new GenericIdentity("MyUser");
   ```

2. Create a new instance of the **GenericPrincipal** class and initialize it with the previously created **GenericIdentity** object and an array of strings that represent the roles that you want associated with this principal. The following code example specifies an array of strings that represent an administrator role and a user role. The **GenericPrincipal** is then initialized with the previous **GenericIdentity** and the string array.

   ```
   C#

   String[] myStringArray = {"Manager", "Teller"};
   GenericPrincipal myPrincipal = new GenericPrincipal(myIdentity,
   myStringArray);
   ```

3. Use the following code to attach the principal to the current thread. This is valuable in situations where the principal must be validated several times, it must be validated by other code running in your application, or it must be validated by a PrincipalPermission object. You can still perform role-based validation on the

principal object without attaching it to the thread. For more information, see Replacing a Principal Object.

```
C#

Thread.CurrentPrincipal = myPrincipal;
```

Example

The following code example demonstrates how to create an instance of a **GenericPrincipal** and a **GenericIdentity**. This code displays the values of these objects to the console.

```
C#

using System;
using System.Security.Principal;
using System.Threading;

public class Class1
{
    public static int Main(string[] args)
    {
    // Create generic identity.
    GenericIdentity myIdentity = new GenericIdentity("MyIdentity");

    // Create generic principal.
    String[] myStringArray = {"Manager", "Teller"};
    GenericPrincipal myPrincipal =
        new GenericPrincipal(myIdentity, myStringArray);

    // Attach the principal to the current thread.
    // This is not required unless repeated validation must occur,
    // other code in your application must validate, or the
    // PrincipalPermission object is used.
    Thread.CurrentPrincipal = myPrincipal;

    // Print values to the console.
    String name =  myPrincipal.Identity.Name;
    bool auth =  myPrincipal.Identity.IsAuthenticated;
    bool isInRole =  myPrincipal.IsInRole("Manager");

    Console.WriteLine("The name is: {0}", name);
    Console.WriteLine("The isAuthenticated is: {0}", auth);
    Console.WriteLine("Is this a Manager? {0}", isInRole);

    return 0;
    }
}
```

When executed, the application displays output similar to the following.

Console

```
The Name is: MyIdentity
The IsAuthenticated is: True
Is this a Manager? True
```

See also

- GenericIdentity
- GenericPrincipal
- PrincipalPermission
- Replacing a Principal Object
- Principal and Identity Objects

Replacing a Principal Object

Article • 09/15/2021

Applications that provide authentication services must be able to replace the **Principal** object (IPrincipal) for a given thread. Furthermore, the security system must help protect the ability to replace **Principal** objects because a maliciously attached, incorrect **Principal** compromises the security of your application by claiming an untrue identity or role. Therefore, applications that require the ability to replace **Principal** objects must be granted the System.Security.Permissions.SecurityPermission object for principal control. (Note that this permission is not required for performing role-based security checks or for creating **Principal** objects.)

The current **Principal** object can be replaced by performing the following tasks:

1. Create the replacement **Principal** object and associated **Identity** object.

2. Attach the new **Principal** object to the call context.

Example

The following example shows how to create a generic principal object and use it to set the principal of a thread.

C#

```
using System;
using System.Threading;
using System.Security.Permissions;
using System.Security.Principal;

class SecurityPrincipalDemo
{
    public static void Main()
    {
        // Retrieve a GenericPrincipal that is based on the current user's
        // WindowsIdentity.
        GenericPrincipal genericPrincipal = GetGenericPrincipal();

        // Retrieve the generic identity of the GenericPrincipal object.
        GenericIdentity principalIdentity =
            (GenericIdentity)genericPrincipal.Identity;

        // Display the identity name and authentication type.
        if (principalIdentity.IsAuthenticated)
        {
            Console.WriteLine(principalIdentity.Name);
```

```
            Console.WriteLine("Type:" +
principalIdentity.AuthenticationType);
        }

        // Verify that the generic principal has been assigned the
        // NetworkUser role.
        if (genericPrincipal.IsInRole("NetworkUser"))
        {
            Console.WriteLine("User belongs to the NetworkUser role.");
        }

        Thread.CurrentPrincipal = genericPrincipal;
    }

    // Create a generic principal based on values from the current
    // WindowsIdentity.
    private static GenericPrincipal GetGenericPrincipal()
    {
        // Use values from the current WindowsIdentity to construct
        // a set of GenericPrincipal roles.
        WindowsIdentity windowsIdentity = WindowsIdentity.GetCurrent();
        string[] roles = new string[10];
        if (windowsIdentity.IsAuthenticated)
        {
            // Add custom NetworkUser role.
            roles[0] = "NetworkUser";
        }

        if (windowsIdentity.IsGuest)
        {
            // Add custom GuestUser role.
            roles[1] = "GuestUser";
        }

        if (windowsIdentity.IsSystem)
        {
            // Add custom SystemUser role.
            roles[2] = "SystemUser";
        }

        // Construct a GenericIdentity object based on the current Windows
        // identity name and authentication type.
        string authenticationType = windowsIdentity.AuthenticationType!;
        string userName = windowsIdentity.Name;
        GenericIdentity genericIdentity =
            new GenericIdentity(userName, authenticationType);

        // Construct a GenericPrincipal object based on the generic identity
        // and custom roles for the user.
        GenericPrincipal genericPrincipal =
            new GenericPrincipal(genericIdentity, roles);

        return genericPrincipal;
    }
}
```

See also

- System.Security.Permissions.SecurityPermission
- Principal and Identity Objects
- ASP.NET Core Security

Impersonating and Reverting

Article • 09/15/2021

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see ASP.NET Core Security.

Sometimes you might need to obtain a Windows account token to impersonate a Windows account. For example, your ASP.NET-based application might have to act on behalf of several users at different times. Your application might accept a token that represents an administrator from Internet Information Services (IIS), impersonate that user, perform an operation, and revert to the previous identity. Next, it might accept a token from IIS that represents a user with fewer rights, perform some operation, and revert again.

In situations where your application must impersonate a Windows account that has not been attached to the current thread by IIS, you must retrieve that account's token and use it to activate the account. You can do this by performing the following tasks:

1. Retrieve an account token for a particular user by making a call to the unmanaged **LogonUser** method. This method is not in the .NET base class library, but is located in the unmanaged **advapi32.dll**. Accessing methods in unmanaged code is an advanced operation and is beyond the scope of this discussion. For more information, see Interoperating with Unmanaged Code. For more information about the **LogonUser** method and **advapi32.dll**, see the Platform SDK documentation.

2. Create a new instance of the **WindowsIdentity** class, passing the token. The following code demonstrates this call, where `hToken` represents a Windows token.

   ```
   C#

   WindowsIdentity impersonatedIdentity = new WindowsIdentity(hToken);
   ```

3. Begin impersonation by creating a new instance of the WindowsImpersonationContext class and initializing it with the WindowsIdentity.Impersonate method of the initialized class, as shown in the following code.

   ```
   ```

```
C#

WindowsImpersonationContext myImpersonation =
impersonatedIdentity.Impersonate();
```

4. When you no longer need to impersonate, call the WindowsImpersonationContext.Undo method to revert the impersonation, as shown in the following code.

```
C#

myImpersonation.Undo();
```

If trusted code has already attached a WindowsPrincipal object to the thread, you can call the instance method **Impersonate**, which does not take an account token. Note that this is only useful when the **WindowsPrincipal** object on the thread represents a user other than the one under which the process is currently executing. For example, you might encounter this situation using ASP.NET with Windows authentication turned on and impersonation turned off. In this case, the process is running under an account configured in Internet Information Services (IIS) while the current principal represents the Windows user that is accessing the page.

Note that neither **Impersonate** nor **Undo** changes the **Principal** object (IPrincipal) associated with the current call context. Rather, impersonation and reverting change the token associated with the current operating system process.

See also

- WindowsIdentity
- WindowsImpersonationContext
- Principal and Identity Objects
- Interoperating with Unmanaged Code
- ASP.NET Core Security

.NET cryptography model

Article • 02/13/2024

.NET provides implementations of many standard cryptographic algorithms.

Object inheritance

The .NET cryptography system implements an extensible pattern of derived class inheritance. The hierarchy is as follows:

- Algorithm type class, such as SymmetricAlgorithm, AsymmetricAlgorithm, or HashAlgorithm. This level is abstract.
- Algorithm class that inherits from an algorithm type class, for example, Aes, RSA, or ECDiffieHellman. This level is abstract.
- Implementation of an algorithm class that inherits from an algorithm class, for example, AesManaged, RC2CryptoServiceProvider, or ECDiffieHellmanCng. This level is fully implemented.

This pattern of derived classes lets you add a new algorithm or a new implementation of an existing algorithm. For example, to create a new public-key algorithm, you would inherit from the AsymmetricAlgorithm class. To create a new implementation of a specific algorithm, you would create a non-abstract derived class of that algorithm.

Going forward, this model of inheritance is not used for new kinds of primitives like AesGcm or Shake128. These algorithms are `sealed`. If you need an extensibility pattern or abstraction over these types, the implementation of the abstraction is the responsibility of the developer.

One-shot APIs

Starting in .NET 5, simpler APIs were introduced for hashing and HMAC. While slightly less flexible, these *one-shot* APIs:

- Are easier to use (and less prone to misuse)
- Reduce allocations or are allocation-free
- Are thread safe
- Use the best available implementation for the platform

The hashing and HMAC primitives expose a one-shot API through a static `HashData` method on the type, such as SHA256.HashData. The static APIs offer no built-in extensibility mechanism. If you're implementing your own algorithms, it's recommended to also offer similar static APIs of the algorithm.

The RandomNumberGenerator class also offers static methods for creating or filling buffers with cryptographic random data. These methods always use the system's cryptographically secure pseudorandom number generator (CSPRNG).

How algorithms are implemented in .NET

As an example of the different implementations available for an algorithm, consider symmetric algorithms. The base for all symmetric algorithms is SymmetricAlgorithm, which is inherited by Aes, TripleDES, and others that are no longer recommended.

Aes is inherited by AesCryptoServiceProvider, AesCng, and AesManaged.

In .NET Framework on Windows:

- `*CryptoServiceProvider` algorithm classes, such as AesCryptoServiceProvider, are wrappers around the Windows Cryptography API (CAPI) implementation of an algorithm.
- `*Cng` algorithm classes, such as ECDiffieHellmanCng, are wrappers around the Windows Cryptography Next Generation (CNG) implementation.
- `*Managed` classes, such as AesManaged, are written entirely in managed code. `*Managed` implementations are not certified by the Federal Information Processing Standards (FIPS), and may be slower than the `*CryptoServiceProvider` and `*Cng` wrapper classes.

In .NET Core and .NET 5 and later versions, all implementation classes (`*CryptoServiceProvider`, `*Managed`, and `*Cng`) are wrappers for the operating system (OS) algorithms. If the OS algorithms are FIPS-certified, then .NET uses FIPS-certified algorithms. For more information, see Cross-Platform Cryptography.

In most cases, you don't need to directly reference an algorithm implementation class, such as `AesCryptoServiceProvider`. The methods and properties you typically need are on the base algorithm class, such as `Aes`. Create an instance of a default implementation class by using a factory method on the base algorithm class, and refer to the base algorithm class. For example, see the highlighted line of code in the following example:

```
C#
```

```
using System.Security.Cryptography;

try
{
    using (FileStream fileStream = new("TestData.txt",
FileMode.OpenOrCreate))
    {
        using (Aes aes = Aes.Create())
        {
            byte[] key =
            {
                0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
                0x09, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16
            };
            aes.Key = key;

            byte[] iv = aes.IV;
            fileStream.Write(iv, 0, iv.Length);

            using (CryptoStream cryptoStream = new(
                fileStream,
                aes.CreateEncryptor(),
                CryptoStreamMode.Write))
            {
                // By default, the StreamWriter uses UTF-8 encoding.
                // To change the text encoding, pass the desired encoding as
the second parameter.
                // For example, new StreamWriter(cryptoStream,
Encoding.Unicode).
                using (StreamWriter encryptWriter = new(cryptoStream))
                {
                    encryptWriter.WriteLine("Hello World!");
                }
            }
        }
    }

    Console.WriteLine("The file was encrypted.");
}
catch (Exception ex)
{
    Console.WriteLine($"The encryption failed. {ex}");
}
```

Choose an algorithm

You can select an algorithm for different reasons: for example, for data integrity, for data privacy, or to generate a key. Symmetric and hash algorithms are intended for protecting data for either integrity reasons (protect from change) or privacy reasons (protect from viewing). Hash algorithms are used primarily for data integrity.

Here is a list of recommended algorithms by application:

- Data privacy:
  - Aes
- Data integrity:
  - HMACSHA256
  - HMACSHA512
- Digital signature:
  - ECDsa
  - RSA
- Key exchange:
  - ECDiffieHellman
  - RSA
- Random number generation:
  - RandomNumberGenerator.GetBytes
  - RandomNumberGenerator.Fill
- Generating a key from a password:
  - Rfc2898DeriveBytes.Pbkdf2

See also

- Cryptographic Services
- Cross-Platform Cryptography
- ASP.NET Core Data Protection

Cross-Platform Cryptography in .NET

Article • 09/07/2024

Cryptographic operations in .NET are done by operating system (OS) libraries. This dependency has advantages:

- .NET apps benefit from OS reliability. Keeping cryptography libraries safe from vulnerabilities is a high priority for OS vendors. To do that, they provide updates that system administrators should be applying.
- .NET apps have access to FIPS-validated algorithms if the OS libraries are FIPS-validated.

The dependency on OS libraries also means that .NET apps can only use cryptographic features that the OS supports. While all platforms support certain core features, some features that .NET supports can't be used on some platforms. This article identifies the features that are supported on each platform.

This article assumes you have a working familiarity with cryptography in .NET. For more information, see .NET Cryptography Model and .NET Cryptographic Services.

Hash and Message Authentication Algorithms

All hash algorithm and hash-based message authentication (HMAC) classes, including the `*Managed` classes, defer to the OS libraries with the exception of .NET on Browser WASM. In Browser WASM, SHA-1, SHA-2-256, SHA-2-384, SHA-2-512 and the HMAC equivalents are implemented using managed code.

Expand table

Algorithm	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android	Browser
MD5	✔	✔	✔	✔	✔	✘
SHA-1	✔	✔	✔	✔	✔	✔
SHA-2-256	✔	✔	✔	✔	✔	✔
SHA-2-384	✔	✔	✔	✔	✔	✔
SHA-2-512	✔	✔	✔	✔	✔	✔
SHA-3-256	Windows 11 Build 25324+	OpenSSL 1.1.1+	✘	✘	✘	✘

Algorithm	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android	Browser
SHA-3-384	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
SHA-3-512	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
SHAKE-128	Windows 11 Build 25324+	OpenSSL 1.1.1+[2]	✗	✗	✗	✗
SHAKE-256	Windows 11 Build 25324+	OpenSSL 1.1.1+[2]	✗	✗	✗	✗
HMAC-MD5	✓	✓	✓	✓	✓	✗
HMAC-SHA-1	✓	✓	✓	✓	✓	✓
HMAC-SHA-2-256	✓	✓	✓	✓	✓	✓
HMAC-SHA-2-384	✓	✓	✓	✓	✓	✓
HMAC-SHA-2-512	✓	✓	✓	✓	✓	✓
HMAC-SHA-3-256	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
HMAC-SHA-3-384	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
HMAC-SHA-3-512	Windows 11 Build 25324+	OpenSSL 1.1.1+	✗	✗	✗	✗
KMAC-128[1]	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗
KMAC-256[1]	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗
KMAC-XOF-128[1]	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗
KMAC-XOF-256[1]	Windows 11 Build 26016+	OpenSSL 3.0+	✗	✗	✗	✗

[1]Available starting in .NET 9.

[2]Streaming extensible output function (XOF) is available starting in .NET 9. On Linux, this requires OpenSSL 3.3.

Symmetric encryption

The underlying ciphers and chaining are done by the system libraries.

Expand table

Cipher + Mode	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
AES-CBC	✔	✔	✔	✔	✔
AES-ECB	✔	✔	✔	✔	✔
AES-CFB8	✔	✔	✔	✔	✔
AES-CFB128	✔	✔	✔	✔	✔
3DES-CBC	✔	✔	✔	✔	✔
3DES-ECB	✔	✔	✔	✔	✔
3DES-CFB8	✔	✔	✔	✔	✔
3DES-CFB64	✔	✔	✔	✔	✔
DES-CBC	✔	✔	✔	✔	✔
DES-ECB	✔	✔	✔	✔	✔
DES-CFB8	✔	✔	✔	✔	✔
RC2-CBC	✔	✔	✔	✔	✘
RC2-ECB	✔	✔	✔	✔	✘
RC2-CFB	✘	✘	✘	✘	✘

Authenticated encryption

Authenticated encryption (AE) support is provided for AES-CCM, AES-GCM, and ChaCha20Poly1305 via the System.Security.Cryptography.AesCcm, System.Security.Cryptography.AesGcm, and System.Security.Cryptography.ChaCha20Poly1305 classes, respectively.

Since authenticated encryption requires newer platform APIs to support the algorithm, support may not be present on all platforms. The `IsSupported` static property on the classes for the algorithm can be used to detect at runtime if the current platform supports the algorithm or not.

Expand table

Cipher + Mode	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android	Browser
AES-GCM	✔	✔	✔	⚠	✔	✘
AES-CCM	✔	✔	⚠	✘	✔	✘
ChaCha20Poly1305	Windows 10 Build 20142+	OpenSSL 1.1.0+	✔	⚠	API Level 28+	✘

AES-CCM on macOS

On macOS, the system libraries don't support AES-CCM for third-party code, so the AesCcm class uses OpenSSL for support. Users on macOS need to obtain an appropriate copy of OpenSSL (libcrypto) for this type to function, and it must be in a path that the system would load a library from by default. We recommend that you install OpenSSL from a package manager such as Homebrew.

The `libcrypto.0.9.7.dylib` and `libcrypto.0.9.8.dylib` libraries included in macOS are from earlier versions of OpenSSL and will not be used. The `libcrypto.35.dylib`, `libcrypto.41.dylib`, and `libcrypto.42.dylib` libraries are from LibreSSL and will not be used.

AES-GCM and ChaCha20Poly1305 on iOS, tvOS, and MacCatalyst

Support for AES-GCM and ChaCha20Poly1305 is available starting in .NET 9 on iOS and tvOS 13.0 and later, and all versions of MacCatalyst.

AES-CCM keys, nonces, and tags

- Key Sizes

  AES-CCM works with 128, 192, and 256-bit keys.

- Nonce Sizes

  The AesCcm class supports 56, 64, 72, 80, 88, 96, and 104-bit (7, 8, 9, 10, 11, 12, and 13-byte) nonces.

- Tag Sizes

  The AesCcm class supports creating or processing 32, 48, 64, 80, 96, 112, and 128-bit (4, 8, 10, 12, 14, and 16-byte) tags.

AES-GCM keys, nonces, and tags

- Key Sizes

  AES-GCM works with 128, 192, and 256-bit keys.

- Nonce Sizes

  The AesGcm class supports only 96-bit (12-byte) nonces.

- Tag Sizes On Windows and Linux, the AesGcm class supports creating or processing 96, 104, 112, 120, and 128-bit (12, 13, 14, 15, and 16-byte) tags. On Apple platforms, the tag size is limited to 128-bit (16-byte) due to limitations of the CryptoKit framework.

ChaCha20Poly1305 keys, nonces, and tags.

ChaCha20Poly1305 has a fixed size for the key, nonce, and authentication tag. ChaCha20Poly1305 always uses a 256-bit key, a 96-bit (12-byte) nonce, and 128-bit (16-byte) tag.

Asymmetric cryptography

This section includes the following subsections:

- RSA
- ECDSA
- ECDH
- DSA

RSA

RSA (Rivest–Shamir–Adleman) key generation is performed by the OS libraries and is subject to their size limitations and performance characteristics.

RSA key operations are performed by the OS libraries, and the types of key that can be loaded are subject to OS requirements.

.NET does not expose "raw" (unpadded) RSA operations.

Padding and digest support vary by platform:

Expand table

Padding Mode	Windows (CNG)	Linux (OpenSSL)	macOS	iOS, tvOS, MacCatalyst	Android	Windows (CAPI)
PKCS1 Encryption	✔	✔	✔	✔	✔	✔
OAEP - SHA-1	✔	✔	✔	✔	✔	✔
OAEP - SHA-2	✔	✔	✔	✔	✔	✘
OAEP - SHA-3	Windows 11 Build 25324+	OpenSSL 1.1.1+	✘	✘	✘	✘
PKCS1 Signature (MD5, SHA-1)	✔	✔	✔	✔	✔	✔
PKCS1 Signature (SHA-2)	✔	✔	✔	✔	✔	⚠[1]
PKCS1 Signature (SHA-3)	Windows 11 Build 25324+	OpenSSL 1.1.1+	✘	✘	✘	✘
PSS	✔	✔	✔	✔	✔	✘

[1] Windows CryptoAPI (CAPI) is capable of PKCS1 signature with a SHA-2 algorithm. But the individual RSA object may be loaded in a cryptographic service provider (CSP) that doesn't support it.

RSA on Windows

- Windows CryptoAPI (CAPI) is used whenever new RSACryptoServiceProvider() is used.

- Windows Cryptography API Next Generation (CNG) is used whenever new RSACng() is used.
- The object returned by RSA.Create is internally powered by Windows CNG. This use of Windows CNG is an implementation detail and is subject to change.
- The GetRSAPublicKey extension method for X509Certificate2 returns an RSACng instance. This use of RSACng is an implementation detail and is subject to change.
- The GetRSAPrivateKey extension method for X509Certificate2 currently prefers an RSACng instance, but if RSACng can't open the key, RSACryptoServiceProvider will be attempted. The preferred provider is an implementation detail and is subject to change.

RSA native interop

.NET exposes types to allow programs to interoperate with the OS libraries that the .NET cryptography code uses. The types involved do not translate between platforms, and should only be directly used when necessary.

Expand table

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
RSACryptoServiceProvider	✔	⚠[1]	⚠[1]	⚠[1]	⚠[1]
RSACng	✔	✘	✘	✘	✘
RSAOpenSsl	✘	✔	⚠[2]	✘	✘

[1] On non-Windows, RSACryptoServiceProvider can be used for compatibility with existing programs. In that case, any method that requires OS interop, such as opening a named key, throws a PlatformNotSupportedException.

[2] On macOS, RSAOpenSsl works if OpenSSL is installed and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

ECDSA

ECDSA (Elliptic Curve Digital Signature Algorithm) key generation is done by the OS libraries and is subject to their size limitations and performance characteristics.

ECDSA key curves are defined by the OS libraries and are subject to their limitations.

Expand table

Elliptic Curve	Windows 10	Windows 7 - 8.1	Linux	macOS	iOS, tvOS, MacCatalyst	Android
NIST P-256 (secp256r1)	✔	✔	✔	✔	✔	✔
NIST P-384 (secp384r1)	✔	✔	✔	✔	✔	✔
NIST P-521 (secp521r1)	✔	✔	✔	✔	✔	✔
Brainpool curves (as named curves)	✔	✘	⚠[1]	✘	✘	⚠[4]
Other named curves	⚠[2]	✘	⚠[1]	✘	✘	⚠[4]
Explicit curves	✔	✘	✔	✘	✘	✔
Export or import as explicit	✔	✘[3]	✔	✘[3]	✘[3]	✔

[1] Linux distributions don't all have support for the same named curves.

[2] Support for named curves was added to Windows CNG in Windows 10. For more information, see CNG Named Elliptic Curves. Named curves are not available in earlier versions of Windows, except for three curves in Windows 7.

[3] Exporting with explicit curve parameters requires OS library support, which is not available on Apple platforms or earlier versions of Windows.

[4] Android support for some curves depends on the Android version. Android distributors may choose to add or remove curves from their Android build as well.

ECDSA Native Interop

.NET exposes types to allow programs to interoperate with the OS libraries that the .NET cryptography code uses. The types involved don't translate between platforms and should only be directly used when necessary.

Expand table

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
ECDsaCng	✔	✘	✘	✘	✘

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
ECDsaOpenSsl	✗	✓	⚠*	✗	✗

* On macOS, ECDsaOpenSsl works if OpenSSL is installed in the system and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

ECDH

ECDH (Elliptic Curve Diffie-Hellman) key generation is done by the OS libraries and is subject to their size limitations and performance characteristics.

The ECDiffieHellman class supports the "raw" value of the ECDH computation as well as through the following key derivation functions:

- HASH(Z)
- HASH(prepend || Z || append)
- HMAC(key, Z)
- HMAC(key, prepend || Z || append)
- HMAC(Z, Z)
- HMAC(Z, prepend || Z || append)
- Tls11Prf(label, seed)

ECDH key curves are defined by the OS libraries and are subject to their limitations.

Expand table

Elliptic Curve	Windows 10	Windows 7 - 8.1	Linux	macOS	iOS, tvOS, MacCatalyst	Android
NIST P-256 (secp256r1)	✓	✓	✓	✓	✓	✓
NIST P-384 (secp384r1)	✓	✓	✓	✓	✓	✓
NIST P-521 (secp521r1)	✓	✓	✓	✓	✓	✓
Brainpool curves (as named curves)	✓	✗	⚠[1]	✗	✗	⚠[4]
Other named curves	⚠[2]	✗	⚠[1]	✗	✗	⚠[4]

Elliptic Curve	Windows 10	Windows 7 - 8.1	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Explicit curves	✔	✘	✔	✘	✘	✔
Export or import as explicit	✔	✘[3]	✔	✘[3]	✘[3]	✔

[1] Linux distributions don't all have support for the same named curves.

[2] Support for named curves was added to Windows CNG in Windows 10. For more information, see CNG Named Elliptic Curves. Named curves are not available in earlier versions of Windows, except for three curves in Windows 7.

[3] Exporting with explicit curve parameters requires OS library support, which is not available on Apple platforms or earlier versions of Windows.

[4] Android support for some curves depends on the Android version. Android distributors may choose to add or remove curves from their Android build as well.

ECDH native interop

.NET exposes types to allow programs to interoperate with the OS libraries that .NET uses. The types involved don't translate between platforms and should only be directly used when necessary.

Expand table

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
ECDiffieHellmanCng	✔	✘	✘	✘	✘
ECDiffieHellmanOpenSsl	✘	✔	⚠*	✘	✘

* On macOS, ECDiffieHellmanOpenSsl works if OpenSSL is installed and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

DSA

DSA (Digital Signature Algorithm) key generation is performed by the system libraries and is subject to their size limitations and performance characteristics.

Expand table

Function	Windows CNG	Linux	macOS	Windows CAPI	iOS, tvOS, MacCatalyst	Android
Key creation (<= 1024 bits)	✔	✔	✘	✔	✘	✔
Key creation (> 1024 bits)	✔	✔	✘	✘	✘	✔
Loading keys (<= 1024 bits)	✔	✔	✔	✔	✘	✔
Loading keys (> 1024 bits)	✔	✔	⚠*	✘	✘	✔
FIPS 186-2	✔	✔	✔	✔	✘	✔
FIPS 186-3 (SHA-2 signatures)	✔	✔	✘	✘	✘	✔

* macOS loads DSA keys bigger than 1024 bits, but the behavior of those keys is undefined. They don't behave according to FIPS 186-3.

DSA on Windows

- Windows CryptoAPI (CAPI) is used whenever new DSACryptoServiceProvider() is used.
- Windows Cryptography API Next Generation (CNG) is used whenever new DSACng() is used.
- The object returned by DSA.Create is internally powered by Windows CNG. This use of Windows CNG is an implementation detail and is subject to change.
- The GetDSAPublicKey extension method for X509Certificate2 returns a DSACng instance. This use of DSACng is an implementation detail and is subject to change.
- The GetDSAPrivateKey extension method for X509Certificate2 prefers an DSACng instance, but if DSACng can't open the key, DSACryptoServiceProvider will be attempted. The preferred provider is an implementation detail and is subject to change.

DSA native interop

.NET exposes types to allow programs to interoperate with the OS libraries that the .NET cryptography code uses. The types involved don't translate between platforms and should only be directly used when necessary.

Expand table

Type	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
DSACryptoServiceProvider	✔	⚠[1]	⚠[1]	✘	⚠[1]
DSACng	✔	✘	✘	✘	✘
DSAOpenSsl	✘	✔	⚠[2]	✘	✘

[1] On non-Windows, DSACryptoServiceProvider can be used for compatibility with existing programs. In that case, any method that requires system interop, such as opening a named key, throws a PlatformNotSupportedException.

[2] On macOS, DSAOpenSsl works if OpenSSL is installed and an appropriate libcrypto dylib can be found via dynamic library loading. If an appropriate library can't be found, exceptions will be thrown.

X.509 Certificates

The majority of support for X.509 certificates in .NET comes from OS libraries. To load a certificate into an X509Certificate2 or X509Certificate instance in .NET, the certificate must be loaded by the underlying OS library.

Read a PKCS12/PFX

Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Empty	✔	✔	✔	✔	✔
One certificate, no private key	✔	✔	✔	✔	✔
One certificate, with private key	✔	✔	✔	✔	✔
Multiple certificates, no private keys	✔	✔	✔	✔	✔
Multiple certificates, one private key	✔	✔	✔	✔	✔
Multiple certificates, multiple private keys	✔	✔	✔	✔	✔

Write a PKCS12/PFX

Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Empty	✔	✔	✔	✔	✔
One certificate, no private key	✔	✔	✔	✔	✔
One certificate, with private key	✔	✔	✔	✔	✔
Multiple certificates, no private keys	✔	✔	✔	✔	✔
Multiple certificates, one private key	✔	✔	✔	✔	✔
Multiple certificates, multiple private keys	✔	✔	✔	✔	✔
Ephemeral loading	✔	✔	✘	✔	✔

macOS can't load certificate private keys without a keychain object, which requires writing to disk. Keychains are created automatically for PFX loading, and are deleted when no longer in use. Since the X509KeyStorageFlags.EphemeralKeySet option means that the private key should not be written to disk, asserting that flag on macOS results in a PlatformNotSupportedException.

Write a PKCS7 certificate collection

Windows and Linux both emit DER-encoded PKCS7 blobs. macOS emits indefinite-length-CER-encoded PKCS7 blobs.

X509Store

On Windows, the X509Store class is a representation of the Windows Certificate Store APIs. Those APIs work the same in .NET Core and .NET 5 as they do in .NET Framework.

On non-Windows, the X509Store class is a projection of system trust decisions (read-only), user trust decisions (read-write), and user key storage (read-write).

The following tables show which scenarios are supported in each platform. For unsupported scenarios (✘ in the tables), a CryptographicException is thrown.

The My store

Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\My (ReadOnly)	✔	✔	✔	✔	✔
Open CurrentUser\My (ReadWrite)	✔	✔	✔	✔	✔
Open CurrentUser\My (ExistingOnly)	✔	⚠	✔	✔	✔
Open LocalMachine\My	✔	✘	✔	✔	✔

On Linux, stores are created on first write, and no user stores exist by default, so opening `CurrentUser\My` with `ExistingOnly` may fail.

On macOS, the `CurrentUser\My` store is the user's default keychain, which is `login.keychain` by default. The `LocalMachine\My` store is `System.keychain`.

The Root store

Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\Root (ReadOnly)	✔	✔	✔	✘	✔
Open CurrentUser\Root (ReadWrite)	✔	✔	✘	✘	✘
Open CurrentUser\Root (ExistingOnly)	✔	⚠	✔ (if ReadOnly)	✘	✔ (if ReadOnly)
Open LocalMachine\Root (ReadOnly)	✔	✔	✔	✘	✔
Open LocalMachine\Root (ReadWrite)	✔	✘	✘	✘	✘
Open LocalMachine\Root (ExistingOnly)	✔	⚠	✔ (if ReadOnly)	✘	✔ (if ReadOnly)

On Linux, the `LocalMachine\Root` store is an interpretation of the CA bundle in the default path for OpenSSL.

On macOS, the `CurrentUser\Root` store is an interpretation of the `SecTrustSettings` results for the user trust domain. The `LocalMachine\Root` store is an interpretation of the `SecTrustSettings` results for the admin and system trust domains.

The Intermediate store

Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\Intermediate (ReadOnly)	✔	✔	✔	✘	✘
Open CurrentUser\Intermediate (ReadWrite)	✔	✔	✘	✘	✘
Open CurrentUser\Intermediate (ExistingOnly)	✔	⚠	✔ (if ReadOnly)	✘	✘
Open LocalMachine\Intermediate (ReadOnly)	✔	✔	✔	✘	✘
Open LocalMachine\Intermediate (ReadWrite)	✔	✘	✘	✘	✘
Open LocalMachine\Intermediate (ExistingOnly)	✔	⚠	✔ (if ReadOnly)	✘	✘

On Linux, the `CurrentUser\Intermediate` store is used as a cache when downloading intermediate CAs by their Authority Information Access records on successful X509Chain builds. The `LocalMachine\Intermediate` store is an interpretation of the CA bundle in the default path for OpenSSL.

On macOS, the `CurrentUser\Intermediate` store is treated as a custom store. Certificates added to this store do not affect X.509 chain building.

The Disallowed store

Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open CurrentUser\Disallowed (ReadOnly)	✔	⚠	✔	✔	✔
Open CurrentUser\Disallowed (ReadWrite)	✔	⚠	✘	✘	✘
Open CurrentUser\Disallowed (ExistingOnly)	✔	⚠	✔ (if ReadOnly)	✔ (if ReadOnly)	✔ (if ReadOnly)
Open LocalMachine\Disallowed (ReadOnly)	✔	✘	✔	✔	✔
Open LocalMachine\Disallowed (ReadWrite)	✔	✘	✘	✘	✘
Open LocalMachine\Disallowed (ExistingOnly)	✔	✘	✔ (if ReadOnly)	✔ (if ReadOnly)	✔ (if ReadOnly)

On Linux, the `Disallowed` store is not used in chain building, and attempting to add contents to it results in a CryptographicException. A CryptographicException is thrown when opening the `Disallowed` store if it has already acquired contents.

On macOS, the CurrentUser\Disallowed and LocalMachine\Disallowed stores are interpretations of the appropriate SecTrustSettings results for certificates whose trust is set to `Always Deny`.

Nonexistent store

Expand table

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open non-existent store (ExistingOnly)	✘	✘	✘	✘	✘
Open CurrentUser non-existent store (ReadWrite)	✔	✔	⚠	✘	✘

Scenario	Windows	Linux	macOS	iOS, tvOS, MacCatalyst	Android
Open LocalMachine non-existent store (ReadWrite)	✔	✘	✘	✘	✘

On macOS, custom store creation with the X509Store API is supported only for `CurrentUser` location. It will create a new keychain with no password in the user's keychain directory (*~/Library/Keychains*). To create a keychain with password, a P/Invoke to `SecKeychainCreate` could be used. Similarly, `SecKeychainOpen` could be used to open keychains in different locations. The resulting `IntPtr` can be passed to new X509Store(IntPtr) to obtain a read/write-capable store, subject to the current user's permissions.

X509Chain

macOS doesn't support Offline CRL utilization, so `X509RevocationMode.Offline` is treated as `X509RevocationMode.Online`.

macOS doesn't support a user-initiated timeout on CRL (Certificate Revocation List) / OCSP (Online Certificate Status Protocol) / AIA (Authority Information Access) downloading, so `X509ChainPolicy.UrlRetrievalTimeout` is ignored.

Additional resources

- .NET Cryptography Model
- .NET Cryptographic Services
- Timing vulnerabilities with CBC-mode symmetric decryption using padding
- ASP.NET Core Data Protection

Overview of encryption, digital signatures, and hash algorithms in .NET

Article • 03/11/2022

This article provides an overview of the encryption methods and practices supported by .NET, including the ClickOnce manifests.

Introduction to cryptography

Public networks such as the Internet do not provide a means of secure communication between entities. Communication over such networks is susceptible to being read or even modified by unauthorized third parties. Cryptography helps protect data from being viewed, provides ways to detect whether data has been modified, and helps provide a secure means of communication over otherwise nonsecure channels. For example, data can be encrypted by using a cryptographic algorithm, transmitted in an encrypted state, and later decrypted by the intended party. If a third party intercepts the encrypted data, it will be difficult to decipher.

In .NET, the classes in the System.Security.Cryptography namespace manage many details of cryptography for you. Some are wrappers for operating system implementations, while others are purely managed implementations. You do not need to be an expert in cryptography to use these classes. When you create a new instance of one of the encryption algorithm classes, keys are autogenerated for ease of use, and default properties are as safe and secure as possible.

Cryptographic Primitives

In a typical situation where cryptography is used, two parties (Alice and Bob) communicate over a nonsecure channel. Alice and Bob want to ensure that their communication remains incomprehensible by anyone who might be listening. Furthermore, because Alice and Bob are in remote locations, Alice must make sure that the information she receives from Bob has not been modified by anyone during transmission. In addition, she must make sure that the information really does originate from Bob and not from someone who is impersonating Bob.

Cryptography is used to achieve the following goals:

- Confidentiality: To help protect a user's identity or data from being read.
- Data integrity: To help protect data from being changed.

- Authentication: To ensure that data originates from a particular party.
- Non-repudiation: To prevent a particular party from denying that they sent a message.

To achieve these goals, you can use a combination of algorithms and practices known as cryptographic primitives to create a cryptographic scheme. The following table lists the cryptographic primitives and their uses.

Expand table

Cryptographic primitive	Use
Secret-key encryption (symmetric cryptography)	Performs a transformation on data to keep it from being read by third parties. This type of encryption uses a single shared, secret key to encrypt and decrypt data.
Public-key encryption (asymmetric cryptography)	Performs a transformation on data to keep it from being read by third parties. This type of encryption uses a public/private key pair to encrypt and decrypt data.
Cryptographic signing	Helps verify that data originates from a specific party by creating a digital signature that is unique to that party. This process also uses hash functions.
Cryptographic hashes	Maps data from any length to a fixed-length byte sequence. Hashes are statistically unique; a different two-byte sequence will not hash to the same value.

Secret-Key Encryption

Secret-key encryption algorithms use a single secret key to encrypt and decrypt data. You must secure the key from access by unauthorized agents, because any party that has the key can use it to decrypt your data or encrypt their own data, claiming it originated from you.

Secret-key encryption is also referred to as symmetric encryption because the same key is used for encryption and decryption. Secret-key encryption algorithms are very fast (compared with public-key algorithms) and are well suited for performing cryptographic transformations on large streams of data. Asymmetric encryption algorithms such as RSA are limited mathematically in how much data they can encrypt. Symmetric encryption algorithms do not generally have those problems.

A type of secret-key algorithm called a block cipher is used to encrypt one block of data at a time. Block ciphers such as Data Encryption Standard (DES), TripleDES, and

Advanced Encryption Standard (AES) cryptographically transform an input block of *n* bytes into an output block of encrypted bytes. If you want to encrypt or decrypt a sequence of bytes, you have to do it block by block. Because *n* is small (8 bytes for DES and TripleDES; 16 bytes [the default], 24 bytes, or 32 bytes for AES), data values that are larger than *n* have to be encrypted one block at a time. Data values that are smaller than *n* have to be expanded to *n* in order to be processed.

One simple form of block cipher is called the electronic codebook (ECB) mode. ECB mode is not considered secure, because it does not use an initialization vector to initialize the first plaintext block. For a given secret key *k*, a simple block cipher that does not use an initialization vector will encrypt the same input block of plaintext into the same output block of ciphertext. Therefore, if you have duplicate blocks in your input plaintext stream, you will have duplicate blocks in your output ciphertext stream. These duplicate output blocks alert unauthorized users to the weak encryption used the algorithms that might have been employed, and the possible modes of attack. The ECB cipher mode is therefore quite vulnerable to analysis, and ultimately, key discovery.

The block cipher classes that are provided in the base class library use a default chaining mode called cipher-block chaining (CBC), although you can change this default if you want.

CBC ciphers overcome the problems associated with ECB ciphers by using an initialization vector (IV) to encrypt the first block of plaintext. Each subsequent block of plaintext undergoes a bitwise exclusive OR (`XOR`) operation with the previous ciphertext block before it is encrypted. Each ciphertext block is therefore dependent on all previous blocks. When this system is used, common message headers that might be known to an unauthorized user cannot be used to reverse-engineer a key.

One way to compromise data that is encrypted with a CBC cipher is to perform an exhaustive search of every possible key. Depending on the size of the key that is used to perform encryption, this kind of search is very time-consuming using even the fastest computers and is therefore infeasible. Larger key sizes are more difficult to decipher. Although encryption does not make it theoretically impossible for an adversary to retrieve the encrypted data, it does raise the cost of doing this. If it takes three months to perform an exhaustive search to retrieve data that is meaningful only for a few days, the exhaustive search method is impractical.

The disadvantage of secret-key encryption is that it presumes two parties have agreed on a key and IV, and communicated their values. The IV is not considered a secret and can be transmitted in plaintext with the message. However, the key must be kept secret from unauthorized users. Because of these problems, secret-key encryption is often

used together with public-key encryption to privately communicate the values of the key and IV.

Assuming that Alice and Bob are two parties who want to communicate over a nonsecure channel, they might use secret-key encryption as follows: Alice and Bob agree to use one particular algorithm (AES, for example) with a particular key and IV. Alice composes a message and creates a network stream (perhaps a named pipe or network email) on which to send the message. Next, she encrypts the text using the key and IV, and sends the encrypted message and IV to Bob over the intranet. Bob receives the encrypted text and decrypts it by using the IV and previously agreed upon key. If the transmission is intercepted, the interceptor cannot recover the original message, because they do not know the key. In this scenario, only the key must remain secret. In a real world scenario, either Alice or Bob generates a secret key and uses public-key (asymmetric) encryption to transfer the secret (symmetric) key to the other party. For more information about public-key encryption, see the next section.

.NET provides the following classes that implement secret-key encryption algorithms:

- Aes
- HMACSHA256, HMACSHA384 and HMACSHA512. (These are technically secret-key algorithms because they represent message authentication codes that are calculated by using a cryptographic hash function combined with a secret key. See Hash Values, later in this article.)

Public-Key Encryption

Public-key encryption uses a private key that must be kept secret from unauthorized users and a public key that can be made public to anyone. The public key and the private key are mathematically linked; data that is encrypted with the public key can be decrypted only with the private key, and data that is signed with the private key can be verified only with the public key. The public key can be made available to anyone; it is used for encrypting data to be sent to the keeper of the private key. Public-key cryptographic algorithms are also known as asymmetric algorithms because one key is required to encrypt data, and another key is required to decrypt data. A basic cryptographic rule prohibits key reuse, and both keys should be unique for each communication session. However, in practice, asymmetric keys are generally long-lived.

Two parties (Alice and Bob) might use public-key encryption as follows: First, Alice generates a public/private key pair. If Bob wants to send Alice an encrypted message, he asks her for her public key. Alice sends Bob her public key over a nonsecure network, and Bob uses this key to encrypt a message. Bob sends the encrypted message to Alice,

and she decrypts it by using her private key. If Bob received Alice's key over a nonsecure channel, such as a public network, Bob is open to a man-in-the-middle attack. Therefore, Bob must verify with Alice that he has a correct copy of her public key.

During the transmission of Alice's public key, an unauthorized agent might intercept the key. Furthermore, the same agent might intercept the encrypted message from Bob. However, the agent cannot decrypt the message with the public key. The message can be decrypted only with Alice's private key, which has not been transmitted. Alice does not use her private key to encrypt a reply message to Bob, because anyone with the public key could decrypt the message. If Alice wants to send a message back to Bob, she asks Bob for his public key and encrypts her message using that public key. Bob then decrypts the message using his associated private key.

In this scenario, Alice and Bob use public-key (asymmetric) encryption to transfer a secret (symmetric) key and use secret-key encryption for the remainder of their session.

The following list offers comparisons between public-key and secret-key cryptographic algorithms:

- Public-key cryptographic algorithms use a fixed buffer size, whereas secret-key cryptographic algorithms use a variable-length buffer.
- Public-key algorithms cannot be used to chain data together into streams the way secret-key algorithms can, because only small amounts of data can be encrypted. Therefore, asymmetric operations do not use the same streaming model as symmetric operations.
- Public-key encryption has a much larger keyspace (range of possible values for the key) than secret-key encryption. Therefore, public-key encryption is less susceptible to exhaustive attacks that try every possible key.
- Public keys are easy to distribute because they do not have to be secured, provided that some way exists to verify the identity of the sender.
- Some public-key algorithms (such as RSA and DSA, but not Diffie-Hellman) can be used to create digital signatures to verify the identity of the sender of data.
- Public-key algorithms are very slow compared with secret-key algorithms, and are not designed to encrypt large amounts of data. Public-key algorithms are useful only for transferring very small amounts of data. Typically, public-key encryption is used to encrypt a key and IV to be used by a secret-key algorithm. After the key and IV are transferred, secret-key encryption is used for the remainder of the session.

.NET provides the following classes that implement public-key algorithms:

- RSA
- ECDsa
- ECDiffieHellman
- DSA

RSA allows both encryption and signing, but DSA can be used only for signing. DSA is not as secure as RSA, and we recommend RSA. Diffie-Hellman can be used only for key generation. In general, public-key algorithms are more limited in their uses than private-key algorithms.

Digital Signatures

Public-key algorithms can also be used to form digital signatures. Digital signatures authenticate the identity of a sender (if you trust the sender's public key) and help protect the integrity of data. Using a public key generated by Alice, the recipient of Alice's data can verify that Alice sent it by comparing the digital signature to Alice's data and Alice's public key.

To use public-key cryptography to digitally sign a message, Alice first applies a hash algorithm to the message to create a message digest. The message digest is a compact and unique representation of data. Alice then encrypts the message digest with her private key to create her personal signature. Upon receiving the message and signature, Bob decrypts the signature using Alice's public key to recover the message digest and hashes the message using the same hash algorithm that Alice used. If the message digest that Bob computes exactly matches the message digest received from Alice, Bob is assured that the message came from the possessor of the private key and that the data has not been modified. If Bob trusts that Alice is the possessor of the private key, he knows that the message came from Alice.

> **Note**
>
> A signature can be verified by anyone because the sender's public key is common knowledge and is typically included in the digital signature format. This method does not retain the secrecy of the message; for the message to be secret, it must also be encrypted.

.NET provides the following classes that implement digital signature algorithms:

- RSA
- ECDsa
- DSA

Hash Values

Hash algorithms map binary values of an arbitrary length to smaller binary values of a fixed length, known as hash values. A hash value is a numerical representation of a piece of data. If you hash a paragraph of plaintext and change even one letter of the paragraph, a subsequent hash will produce a different value. If the hash is cryptographically strong, its value will change significantly. For example, if a single bit of a message is changed, a strong hash function may produce an output that differs by 50 percent. Many input values may hash to the same output value. However, it is computationally infeasible to find two distinct inputs that hash to the same value.

Two parties (Alice and Bob) could use a hash function to ensure message integrity. They would select a hash algorithm to sign their messages. Alice would write a message, and then create a hash of that message by using the selected algorithm. They would then follow one of the following methods:

- Alice sends the plaintext message and the hashed message (digital signature) to Bob. Bob receives and hashes the message and compares his hash value to the hash value that he received from Alice. If the hash values are identical, the message was not altered. If the values are not identical, the message was altered after Alice wrote it.

  Unfortunately, this method does not establish the authenticity of the sender. Anyone can impersonate Alice and send a message to Bob. They can use the same hash algorithm to sign their message, and all Bob can determine is that the message matches its signature. This is one form of a man-in-the-middle attack. For more information, see Cryptography Next Generation (CNG) Secure Communication Example.

- Alice sends the plaintext message to Bob over a nonsecure public channel. She sends the hashed message to Bob over a secure private channel. Bob receives the plaintext message, hashes it, and compares the hash to the privately exchanged hash. If the hashes match, Bob knows two things:

  - The message was not altered.

  - The sender of the message (Alice) is authentic.

For this system to work, Alice must hide her original hash value from all parties except Bob.

- Alice sends the plaintext message to Bob over a nonsecure public channel and places the hashed message on her publicly viewable Web site.

  This method prevents message tampering by preventing anyone from modifying the hash value. Although the message and its hash can be read by anyone, the hash value can be changed only by Alice. An attacker who wants to impersonate Alice would require access to Alice's Web site.

None of the previous methods will prevent someone from reading Alice's messages, because they are transmitted in plaintext. Full security typically requires digital signatures (message signing) and encryption.

.NET provides the following classes that implement hashing algorithms:

- SHA256.
- SHA384.
- SHA512.

.NET also provides MD5 and SHA1. But the MD5 and SHA-1 algorithms have been found to be insecure, and SHA-2 is now recommended instead. SHA-2 includes SHA256, SHA384, and SHA512.

Random Number Generation

Random number generation is integral to many cryptographic operations. For example, cryptographic keys need to be as random as possible so that it is infeasible to reproduce them. Cryptographic random number generators must generate output that is computationally infeasible to predict with a probability that is better than one half. Therefore, any method of predicting the next output bit must not perform better than random guessing. The classes in .NET use random number generators to generate cryptographic keys.

The RandomNumberGenerator class is an implementation of a random number generator algorithm.

ClickOnce Manifests

The following cryptography classes let you obtain and verify information about manifest signatures for applications that are deployed using ClickOnce technology:

- The ManifestSignatureInformation class obtains information about a manifest signature when you use its VerifySignature method overloads.
- You can use the ManifestKinds enumeration to specify which manifests to verify. The result of the verification is one of the SignatureVerificationResult enumeration values.
- The ManifestSignatureInformationCollection class provides a read-only collection of ManifestSignatureInformation objects of the verified signatures.

In addition, the following classes provide specific signature information:

- StrongNameSignatureInformation holds the strong name signature information for a manifest.
- AuthenticodeSignatureInformation represents the Authenticode signature information for a manifest.
- TimestampInformation contains information about the time stamp on an Authenticode signature.
- TrustStatus provides a simple way to check whether an Authenticode signature is trusted.

Cryptography Next Generation (CNG) Classes

The Cryptography Next Generation (CNG) classes provide a managed wrapper around the native CNG functions. (CNG is the replacement for CryptoAPI.) These classes have "Cng" as part of their names. Central to the CNG wrapper classes is the CngKey key container class, which abstracts the storage and use of CNG keys. This class lets you store a key pair or a public key securely and refer to it by using a simple string name. The elliptic curve-based ECDsaCng signature class and the ECDiffieHellmanCng encryption class can use CngKey objects.

The CngKey class is used for a variety of additional operations, including opening, creating, deleting, and exporting keys. It also provides access to the underlying key handle to use when calling native functions directly.

.NET also includes a variety of supporting CNG classes, such as the following:

- CngProvider maintains a key storage provider.

- CngAlgorithm maintains a CNG algorithm.
- CngProperty maintains frequently used key properties.

See also

- Cryptography Model - Describes how cryptography is implemented in the base class library.
- Cross-Platform Cryptography
- Timing vulnerabilities with CBC-mode symmetric decryption using padding
- ASP.NET Core Data Protection

Generate keys for encryption and decryption

Article • 08/12/2022

Creating and managing keys is an important part of the cryptographic process. Symmetric algorithms require the creation of a key and an initialization vector (IV). You must keep this key secret from anyone who shouldn't decrypt your data. The IV doesn't have to be secret but should be changed for each session. Asymmetric algorithms require the creation of a public key and a private key. The public key can be made known to anyone, but the decrypting party must only know the corresponding private key. This section describes how to generate and manage keys for both symmetric and asymmetric algorithms.

Symmetric Keys

The symmetric encryption classes supplied by .NET require a key and a new IV to encrypt and decrypt data. A new key and IV is automatically created when you create a new instance of one of the managed symmetric cryptographic classes using the parameterless `Create()` method. Anyone that you allow to decrypt your data must possess the same key and IV and use the same algorithm. Generally, a new key and IV should be created for every session, and neither the key nor the IV should be stored for use in a later session.

To communicate a symmetric key and IV to a remote party, you usually encrypt the symmetric key by using asymmetric encryption. Sending the key across an insecure network without encryption is unsafe because anyone who intercepts the key and IV can then decrypt your data.

The following example shows the creation of a new instance of the default implementation class for the Aes algorithm:

```
C#

Aes aes = Aes.Create();
```

The execution of the preceding code generates a new key and IV and sets them as values for the **Key** and **IV** properties, respectively.

Sometimes you might need to generate multiple keys. In this situation, you can create a new instance of a class that implements a symmetric algorithm. Then, create a new key

and IV by calling the `GenerateKey` and `GenerateIV` methods. The following code example illustrates how to create new keys and IVs after a new instance of the symmetric cryptographic class has been made:

```
C#
```

```
Aes aes = Aes.Create();
aes.GenerateIV();
aes.GenerateKey();
```

The execution of the preceding code creates a new instance of `Aes` and generates a key and IV. Another key and IV are created when the `GenerateKey` and `GenerateIV` methods are called.

Asymmetric Keys

.NET provides the RSA class for asymmetric encryption. When you use the parameterless `Create()` method to create a new instance, the RSA class creates a public/private key pair. Asymmetric keys can be either stored for use in multiple sessions or generated for one session only. While you can make the public key available, you must closely guard the private key.

A public/private key pair is generated when you create a new instance of an asymmetric algorithm class. After creating a new instance of the class, you can extract the key information using the ExportParameters method. This method returns an RSAParameters structure that holds the key information. The method also accepts a Boolean value that indicates whether to return only the public-key information or to return both the public-key and the private-key information.

You also can use other methods to extract the key information, such as:

- RSA.ExportRSAPublicKey
- RSA.ExportRSAPrivateKey
- AsymmetricAlgorithm.ExportSubjectPublicKeyInfo
- AsymmetricAlgorithm.ExportPkcs8PrivateKey
- AsymmetricAlgorithm.ExportEncryptedPkcs8PrivateKey

You can use the ImportParameters method to initialize an `RSA` instance to the value of an `RSAParameters` structure. Or you can use the RSA.Create(RSAParameters) method to create a new instance.

Never store asymmetric private keys verbatim or as plain text on the local computer. If you need to store a private key, you must use a key container. For more information about how to store a private key in a key container, see How to: Store Asymmetric Keys in a Key Container.

The following code example creates a new instance of the `RSA` class, creates a public/private key pair, and saves the public key information to an `RSAParameters` structure:

C#

```
//Generate a public/private key pair.
RSA rsa = RSA.Create();
//Save the public key information to an RSAParameters structure.
RSAParameters rsaKeyInfo = rsa.ExportParameters(false);
```

See also

- Encrypting Data
- Decrypting Data
- Cryptographic Services
- How to: Store Asymmetric Keys in a Key Container
- Cross-Platform Cryptography
- ASP.NET Core Data Protection

Store asymmetric keys in a key container

Article • 09/15/2021

Asymmetric private keys should never be stored verbatim or in plain text on the local computer. If you need to store a private key, use a key container. For more information on key containers, see Understanding machine-level and user-level RSA key containers.

> **Note**
>
> The code in this article applies to Windows and uses features not available in .NET Core 2.2 and earlier versions. For more information, see dotnet/runtime#23391.

Create an asymmetric key and save it in a key container

1. Create a new instance of a CspParameters class and pass the name that you want to call the key container to the CspParameters.KeyContainerName field.
2. Create a new instance of a class that derives from the AsymmetricAlgorithm class (usually RSACryptoServiceProvider or DSACryptoServiceProvider) and pass the previously created `CspParameters` object to its constructor.

> **Note**
>
> The creation and retrieval of an asymmetric key is one operation. If a key is not already in the container, it's created before being returned.
>
> - RSA.ToXmlString
> - DSA.ToXmlString

Delete the key from the key container

1. Create a new instance of a `CspParameters` class and pass the name that you want to call the key container to the CspParameters.KeyContainerName field.

2. Create a new instance of a class that derives from the AsymmetricAlgorithm class (usually `RSACryptoServiceProvider` or `DSACryptoServiceProvider`) and pass the previously created `CspParameters` object to its constructor.

3. Set the RSACryptoServiceProvider.PersistKeyInCsp or the DSACryptoServiceProvider.PersistKeyInCsp property of the class that derives from `AsymmetricAlgorithm` to `false` (`False` in Visual Basic).

4. Call the `Clear` method of the class that derives from `AsymmetricAlgorithm`. This method releases all resources of the class and clears the key container.

Example

The following example demonstrates how to create an asymmetric key, save it in a key container, retrieve the key at a later time, and delete the key from the container.

Notice that code in the `GenKey_SaveInContainer` method and the `GetKeyFromContainer` method is similar. When you specify a key container name for a CspParameters object and pass it to an AsymmetricAlgorithm object with the PersistKeyInCsp property or PersistKeyInCsp property set to `true`, the behavior is as follows:

- If a key container with the specified name does not exist, then one is created and the key is persisted.
- If a key container with the specified name does exist, then the key in the container is automatically loaded into the current AsymmetricAlgorithm object.

Therefore, the code in the `GenKey_SaveInContainer` method persists the key because it is run first, while the code in the `GetKeyFromContainer` method loads the key because it's run second.

C#

```
using System;
using System.Security.Cryptography;

public class StoreKey
{
    public static void Main()
    {
        try
        {
            // Create a key and save it in a container.
            GenKey_SaveInContainer("MyKeyContainer");

            // Retrieve the key from the container.
            GetKeyFromContainer("MyKeyContainer");
```

```
            // Delete the key from the container.
            DeleteKeyFromContainer("MyKeyContainer");

            // Create a key and save it in a container.
            GenKey_SaveInContainer("MyKeyContainer");

            // Delete the key from the container.
            DeleteKeyFromContainer("MyKeyContainer");
        }
        catch (CryptographicException e)
        {
            Console.WriteLine(e.Message);
        }
    }

    private static void GenKey_SaveInContainer(string containerName)
    {
        // Create the CspParameters object and set the key container
        // name used to store the RSA key pair.
        var parameters = new CspParameters
        {
            KeyContainerName = containerName
        };

        // Create a new instance of RSACryptoServiceProvider that accesses
        // the key container MyKeyContainerName.
        using var rsa = new RSACryptoServiceProvider(parameters);

        // Display the key information to the console.
        Console.WriteLine($"Key added to container: \n
{rsa.ToXmlString(true)}");
    }

    private static void GetKeyFromContainer(string containerName)
    {
        // Create the CspParameters object and set the key container
        // name used to store the RSA key pair.
        var parameters = new CspParameters
        {
            KeyContainerName = containerName
        };

        // Create a new instance of RSACryptoServiceProvider that accesses
        // the key container MyKeyContainerName.
        using var rsa = new RSACryptoServiceProvider(parameters);

        // Display the key information to the console.
        Console.WriteLine($"Key retrieved from container : \n
{rsa.ToXmlString(true)}");
    }

    private static void DeleteKeyFromContainer(string containerName)
    {
        // Create the CspParameters object and set the key container
```

```
        // name used to store the RSA key pair.
        var parameters = new CspParameters
        {
            KeyContainerName = containerName
        };

        // Create a new instance of RSACryptoServiceProvider that accesses
        // the key container.
        using var rsa = new RSACryptoServiceProvider(parameters)
        {
            // Delete the key entry in the container.
            PersistKeyInCsp = false
        };

        // Call Clear to release resources and delete the key from the
container.
        rsa.Clear();

        Console.WriteLine("Key deleted.");
    }
}
```

The output is as follows:

Console

```
Key added to container:
<RSAKeyValue> Key Information A</RSAKeyValue>
Key retrieved from container :
<RSAKeyValue> Key Information A</RSAKeyValue>
Key deleted.
Key added to container:
<RSAKeyValue> Key Information B</RSAKeyValue>
Key deleted.
```

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- Generating keys for encryption and decryption
- Encrypting data
- Decrypting data
- ASP.NET Core Data Protection

Encrypting data

Article • 11/18/2022

Symmetric encryption and asymmetric encryption are performed using different processes. Symmetric encryption is performed on streams and is therefore useful to encrypt large amounts of data. Asymmetric encryption is performed on a small number of bytes and is therefore useful only for small amounts of data.

Symmetric encryption

The managed symmetric cryptography classes are used with a special stream class called a CryptoStream that encrypts data read into the stream. The **CryptoStream** class is initialized with a managed stream class, a class that implements the ICryptoTransform interface (created from a class that implements a cryptographic algorithm), and a CryptoStreamMode enumeration that describes the type of access permitted to the **CryptoStream**. The **CryptoStream** class can be initialized using any class that derives from the Stream class, including FileStream, MemoryStream, and NetworkStream. Using these classes, you can perform symmetric encryption on a variety of stream objects.

The following example illustrates how to create a new instance of the default implementation class for the Aes algorithm. The instance is used to perform encryption on a **CryptoStream** class. In this example, the **CryptoStream** is initialized with a stream object called `fileStream` that can be any type of managed stream. The **CreateEncryptor** method from the **Aes** class is passed the key and IV that are used for encryption. In this case, the default key and IV generated from `aes` are used.

C#

```
Aes aes = Aes.Create();
CryptoStream cryptStream = new CryptoStream(
    fileStream, aes.CreateEncryptor(key, iv), CryptoStreamMode.Write);
```

After this code is executed, any data written to the **CryptoStream** object is encrypted using the AES algorithm.

The following example shows the entire process of creating a stream, encrypting the stream, writing to the stream, and closing the stream. This example creates a file stream that is encrypted using the **CryptoStream** class and the **Aes** class. Generated IV is written to the beginning of FileStream, so it can be read and used for decryption. Then a message is written to the encrypted stream with the StreamWriter class. While the same key can be used multiple times to encrypt and decrypt data, it is recommended to

generate a new random IV each time. This way the encrypted data is always different, even when plain text is the same.

C#

```
using System.Security.Cryptography;

try
{
    using (FileStream fileStream = new("TestData.txt",
FileMode.OpenOrCreate))
    {
        using (Aes aes = Aes.Create())
        {
            byte[] key =
            {
                0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
                0x09, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16
            };
            aes.Key = key;

            byte[] iv = aes.IV;
            fileStream.Write(iv, 0, iv.Length);

            using (CryptoStream cryptoStream = new(
                fileStream,
                aes.CreateEncryptor(),
                CryptoStreamMode.Write))
            {
                // By default, the StreamWriter uses UTF-8 encoding.
                // To change the text encoding, pass the desired encoding as
the second parameter.
                // For example, new StreamWriter(cryptoStream,
Encoding.Unicode).
                using (StreamWriter encryptWriter = new(cryptoStream))
                {
                    encryptWriter.WriteLine("Hello World!");
                }
            }
        }
    }

    Console.WriteLine("The file was encrypted.");
}
catch (Exception ex)
{
    Console.WriteLine($"The encryption failed. {ex}");
}
```

The code encrypts the stream using the AES symmetric algorithm, and writes IV and then encrypted "Hello World!" to the stream. If the code is successful, it creates an encrypted file named *TestData.txt* and displays the following text to the console:

Console

```
The file was encrypted.
```

You can decrypt the file by using the symmetric decryption example in Decrypting Data. That example and this example specify the same key.

However, if an exception is raised, the code displays the following text to the console:

Console

```
The encryption failed.
```

Asymmetric encryption

Asymmetric algorithms are usually used to encrypt small amounts of data such as the encryption of a symmetric key and IV. Typically, an individual performing asymmetric encryption uses the public key generated by another party. The RSA class is provided by .NET for this purpose.

The following example uses public key information to encrypt a symmetric key and IV. Two byte arrays are initialized that represents the public key of a third party. An RSAParameters object is initialized to these values. Next, the **RSAParameters** object (along with the public key it represents) is imported into an **RSA** instance using the RSA.ImportParameters method. Finally, the private key and IV created by an Aes class are encrypted. This example requires systems to have 128-bit encryption installed.

C#

```
using System;
using System.Security.Cryptography;

class Class1
{
    static void Main()
    {
        //Initialize the byte arrays to the public key information.
        byte[] modulus =
        {
            214,46,220,83,160,73,40,39,201,155,19,202,3,11,191,178,56,
            74,90,36,248,103,18,144,170,163,145,87,54,61,34,220,222,
            207,137,149,173,14,92,120,206,222,158,28,40,24,30,16,175,
            108,128,35,230,118,40,121,113,125,216,130,11,24,90,48,194,
            240,105,44,76,34,57,249,228,125,80,38,9,136,29,117,207,139,
            168,181,85,137,126,10,126,242,120,247,121,8,100,12,201,171,
            38,226,193,180,190,117,177,87,143,242,213,11,44,180,113,93,
```

```
            106,99,179,68,175,211,164,116,64,148,226,254,172,147
        };

        byte[] exponent = { 1, 0, 1 };

        //Create values to store encrypted symmetric keys.
        byte[] encryptedSymmetricKey;
        byte[] encryptedSymmetricIV;

        //Create a new instance of the RSA class.
        RSA rsa = RSA.Create();

        //Create a new instance of the RSAParameters structure.
        RSAParameters rsaKeyInfo = new RSAParameters();

        //Set rsaKeyInfo to the public key values.
        rsaKeyInfo.Modulus = modulus;
        rsaKeyInfo.Exponent = exponent;

        //Import key parameters into rsa.
        rsa.ImportParameters(rsaKeyInfo);

        //Create a new instance of the default Aes implementation class.
        Aes aes = Aes.Create();

        //Encrypt the symmetric key and IV.
        encryptedSymmetricKey = rsa.Encrypt(aes.Key,
RSAEncryptionPadding.Pkcs1);
        encryptedSymmetricIV = rsa.Encrypt(aes.IV,
RSAEncryptionPadding.Pkcs1);
    }
}
```

See also

- Generating keys for encryption and decryption
- Decrypting data
- Cryptographic services
- Cross-platform cryptography
- Timing vulnerabilities with CBC-mode symmetric decryption using padding
- ASP.NET Core data protection

Decrypting data

Article • 11/18/2022

Decryption is the reverse operation of encryption. For secret-key encryption, you must know both the key and IV that were used to encrypt the data. For public-key encryption, you must know either the public key (if the data was encrypted using the private key) or the private key (if the data was encrypted using the public key).

Symmetric decryption

The decryption of data encrypted with symmetric algorithms is similar to the process used to encrypt data with symmetric algorithms. The CryptoStream class is used with symmetric cryptography classes provided by .NET to decrypt data read from any managed stream object.

The following example illustrates how to create a new instance of the default implementation class for the Aes algorithm. The instance is used to perform decryption on a CryptoStream object. This example first creates a new instance of the Aes implementation class. It reads the initialization vector (IV) value from a managed stream variable, `fileStream`. Next it instantiates a CryptoStream object and initializes it to the value of the `fileStream` instance. The SymmetricAlgorithm.CreateDecryptor method from the Aes instance is passed the IV value and the same key that was used for encryption.

C#

```
Aes aes = Aes.Create();
CryptoStream cryptStream = new CryptoStream(
    fileStream, aes.CreateDecryptor(key, iv), CryptoStreamMode.Read);
```

The following example shows the entire process of creating a stream, decrypting the stream, reading from the stream, and closing the streams. A file stream object is created that reads a file named *TestData.txt*. The file stream is then decrypted using the **CryptoStream** class and the **Aes** class. This example specifies key value that is used in the symmetric encryption example for Encrypting Data. It does not show the code needed to encrypt and transfer these values.

C#

```
using System.Security.Cryptography;

try
```

```
{
    using (FileStream fileStream = new("TestData.txt", FileMode.Open))
    {
        using (Aes aes = Aes.Create())
        {
            byte[] iv = new byte[aes.IV.Length];
            int numBytesToRead = aes.IV.Length;
            int numBytesRead = 0;
            while (numBytesToRead > 0)
            {
                int n = fileStream.Read(iv, numBytesRead, numBytesToRead);
                if (n == 0) break;

                numBytesRead += n;
                numBytesToRead -= n;
            }

            byte[] key =
            {
                0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
                0x09, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16
            };

            using (CryptoStream cryptoStream = new(
               fileStream,
               aes.CreateDecryptor(key, iv),
               CryptoStreamMode.Read))
            {
                // By default, the StreamReader uses UTF-8 encoding.
                // To change the text encoding, pass the desired encoding as
the second parameter.
                // For example, new StreamReader(cryptoStream,
Encoding.Unicode).
                using (StreamReader decryptReader = new(cryptoStream))
                {
                    string decryptedMessage = await
decryptReader.ReadToEndAsync();
                    Console.WriteLine($"The decrypted original message:
{decryptedMessage}");
                }
            }
        }
    }
}
catch (Exception ex)
{
    Console.WriteLine($"The decryption failed. {ex}");
}
```

The preceding example uses the same key, and algorithm used in the symmetric encryption example for Encrypting Data. It decrypts the *TestData.txt* file that is created by that example and displays the original text on the console.

Asymmetric decryption

Typically, a party (party A) generates both a public and private key and stores the key either in memory or in a cryptographic key container. Party A then sends the public key to another party (party B). Using the public key, party B encrypts data and sends the data back to party A. After receiving the data, party A decrypts it using the private key that corresponds. Decryption will be successful only if party A uses the private key that corresponds to the public key Party B used to encrypt the data.

For information on how to store an asymmetric key in secure cryptographic key container and how to later retrieve the asymmetric key, see How to: Store Asymmetric Keys in a Key Container.

The following example illustrates the decryption of two arrays of bytes that represent a symmetric key and IV. For information on how to extract the asymmetric public key from the RSA object in a format that you can easily send to a third party, see Encrypting Data.

C#

```
//Create a new instance of the RSA class.
RSA rsa = RSA.Create();

// Export the public key information and send it to a third party.
// Wait for the third party to encrypt some data and send it back.

//Decrypt the symmetric key and IV.
symmetricKey = rsa.Decrypt(encryptedSymmetricKey,
RSAEncryptionPadding.Pkcs1);
symmetricIV = rsa.Decrypt(encryptedSymmetricIV ,
RSAEncryptionPadding.Pkcs1);
```

See also

- Generating keys for encryption and decryption
- Encrypting data
- Cryptographic services
- Cryptography model
- Cross-platform cryptography
- Timing vulnerabilities with CBC-mode symmetric decryption using padding
- ASP.NET Core data protection

Cryptographic Signatures

Article • 08/10/2022

Cryptographic digital signatures use public key algorithms to provide data integrity. When you sign data with a digital signature, someone else can verify the signature, and can prove that the data originated from you and was not altered after you signed it. For more information about digital signatures, see Cryptographic Services.

This topic explains how to generate and verify digital signatures using classes in the System.Security.Cryptography namespace.

Generate a signature

Digital signatures are usually applied to hash values that represent larger data. The following example applies a digital signature to a hash value. First, a new instance of the RSA class is created to generate a public/private key pair. Next, the RSA is passed to a new instance of the RSAPKCS1SignatureFormatter class. This transfers the private key to the RSAPKCS1SignatureFormatter, which actually performs the digital signing. Before you can sign the hash code, you must specify a hash algorithm to use. This example uses the `SHA256` algorithm. Finally, the CreateSignature method is called to perform the signing.

C#

```
using System.Security.Cryptography;
using System.Text;

using SHA256 alg = SHA256.Create();

byte[] data = Encoding.ASCII.GetBytes("Hello, from the .NET Docs!");
byte[] hash = alg.ComputeHash(data);

RSAParameters sharedParameters;
byte[] signedHash;

// Generate signature
using (RSA rsa = RSA.Create())
{
    sharedParameters = rsa.ExportParameters(false);

    RSAPKCS1SignatureFormatter rsaFormatter = new(rsa);
    rsaFormatter.SetHashAlgorithm(nameof(SHA256));

    signedHash = rsaFormatter.CreateSignature(hash);
}
```

```
// The sharedParameters, hash, and signedHash are used to later verify the
signature.
```

Verify a signature

To verify that data was signed by a particular party, you must have the following information:

- The public key of the party that signed the data.
- The digital signature.
- The data that was signed.
- The hash algorithm used by the signer.

To verify a signature signed by the RSAPKCS1SignatureFormatter class, use the RSAPKCS1SignatureDeformatter class. The RSAPKCS1SignatureDeformatter class must be supplied the public key of the signer. For RSA, you will need at a minimal the values of the RSAParameters.Modulus and the RSAParameters.Exponent to specify the public key. One way to achieve this is to call RSA.ExportParameters during signature creation and then call RSA.ImportParameters during the verification process. The party that generated the public/private key pair should provide these values. First create an RSA object to hold the public key that will verify the signature, and then initialize an RSAParameters structure to the modulus and exponent values that specify the public key.

The following code shows the sharing of an RSAParameters structure. The `RSA` responsible for creating the signature exports its parameters. The parameters are then imported into the new `RSA` instance that is responsible for verifying the signature.

The RSA instance is, in turn, passed to the constructor of an RSAPKCS1SignatureDeformatter to transfer the key.

The following example illustrates this process. In this example, imagine that `sharedParameters`, `hash`, and `signedHash` are provided by a remote party. The remote party has signed `hash` using the `SHA256` algorithm to produce the digital signature `signedHash`. The RSAPKCS1SignatureDeformatter.VerifySignature method verifies that the digital signature is valid and was used to sign `hash`.

C#

```
using System.Security.Cryptography;
using System.Text;

using SHA256 alg = SHA256.Create();
```

```
byte[] data = Encoding.ASCII.GetBytes("Hello, from the .NET Docs!");
byte[] hash = alg.ComputeHash(data);

RSAParameters sharedParameters;
byte[] signedHash;

// Generate signature
using (RSA rsa = RSA.Create())
{
    sharedParameters = rsa.ExportParameters(false);

    RSAPKCS1SignatureFormatter rsaFormatter = new(rsa);
    rsaFormatter.SetHashAlgorithm(nameof(SHA256));

    signedHash = rsaFormatter.CreateSignature(hash);
}

// Verify signature
using (RSA rsa = RSA.Create())
{
    rsa.ImportParameters(sharedParameters);

    RSAPKCS1SignatureDeformatter rsaDeformatter = new(rsa);
    rsaDeformatter.SetHashAlgorithm(nameof(SHA256));

    if (rsaDeformatter.VerifySignature(hash, signedHash))
    {
        Console.WriteLine("The signature is valid.");
    }
    else
    {
        Console.WriteLine("The signature is not valid.");
    }
}
```

This code fragment displays "`The signature is valid`" if the signature is valid and "`The signature is not valid`" if it's not.

See also

- Cryptographic Services
- Cryptography Model
- Cross-Platform Cryptography
- ASP.NET Core Data Protection

Ensuring Data Integrity with Hash Codes

Article • 01/03/2023

A hash value is a numeric value of a fixed length that uniquely identifies data. Hash values represent large amounts of data as much smaller numeric values, so they are used with digital signatures. You can sign a hash value more efficiently than signing the larger value. Hash values are also useful for verifying the integrity of data sent through insecure channels. The hash value of received data can be compared to the hash value of data as it was sent to determine whether the data was altered.

This topic describes how to generate and verify hash codes by using the classes in the System.Security.Cryptography namespace.

Generating a Hash

The hash classes can hash either an array of bytes or a stream object. The following example uses the SHA-256 hash algorithm to create a hash value for a string. The example uses Encoding.UTF8 to convert the string into an array of bytes that are hashed by using the SHA256 class. The hash value is then displayed to the console.

C#

```
using System;
using System.IO;
using System.Security.Cryptography;
using System.Text;

string messageString = "This is the original message!";

//Convert the string into an array of bytes.
byte[] messageBytes = Encoding.UTF8.GetBytes(messageString);

//Create the hash value from the array of bytes.
byte[] hashValue = SHA256.HashData(messageBytes);

//Display the hash value to the console.
Console.WriteLine(Convert.ToHexString(hashValue));
```

This code will display the following string to the console:

Console

```
67A1790DCA55B8803AD024EE28F616A284DF5DD7B8BA5F68B4B252A5E925AF79
```

Verifying a Hash

Data can be compared to a hash value to determine its integrity. Usually, data is hashed at a certain time and the hash value is protected in some way. At a later time, the data can be hashed again and compared to the protected value. If the hash values match, the data has not been altered. If the values do not match, the data has been corrupted. For this system to work, the protected hash must be encrypted or kept secret from all untrusted parties.

The following example compares the previous hash value of a string to a new hash value.

C#

```
using System;
using System.Linq;
using System.Security.Cryptography;
using System.Text;

//This hash value is produced from "This is the original message!"
//using SHA256.
byte[] sentHashValue =
Convert.FromHexString("67A1790DCA55B8803AD024EE28F616A284DF5DD7B8BA5F68B4B25
2A5E925AF79");

//This is the string that corresponds to the previous hash value.
string messageString = "This is the original message!";

//Convert the string into an array of bytes.
byte[] messageBytes = Encoding.UTF8.GetBytes(messageString);

//Create the hash value from the array of bytes.
byte[] compareHashValue = SHA256.HashData(messageBytes);

//Compare the values of the two byte arrays.
bool same = sentHashValue.SequenceEqual(compareHashValue);

//Display whether or not the hash values are the same.
if (same)
{
    Console.WriteLine("The hash codes match.");
}
else
{
    Console.WriteLine("The hash codes do not match.");
}
```

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- ASP.NET Core Data Protection

How to: Encrypt XML Elements with Symmetric Keys

Article • 09/15/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to encrypt an element within an XML document. XML Encryption allows you to store or transport sensitive XML, without worrying about the data being easily read. This procedure encrypts an XML element using the Advanced Encryption Standard (AES) algorithm.

For information about how to decrypt an XML element that was encrypted using this procedure, see How to: Decrypt XML Elements with Symmetric Keys.

When you use a symmetric algorithm like AES to encrypt XML data, you must use the same key to encrypt and decrypt the XML data. The example in this procedure assumes that the encrypted XML will be decrypted using the same key, and that the encrypting and decrypting parties agree on the algorithm and key to use. This example does not store or encrypt the AES key within the encrypted XML.

This example is appropriate for situations where a single application needs to encrypt data based on a session key stored in memory, or based on a cryptographically strong key derived from a password. For situations where two or more applications need to share encrypted XML data, consider using an encryption scheme based on an asymmetric algorithm or an X.509 certificate.

To encrypt an XML element with a symmetric key

1. Generate a symmetric key using the Aes class. This key will be used to encrypt the XML element.

   C#

   ```
   Aes? key = null;

   try
   {
       // Create a new AES key.
       key = Aes.Create();
   ```

2. Create an XmlDocument object by loading an XML file from disk. The XmlDocument object contains the XML element to encrypt.

   C#

```
// Load an XML document.
XmlDocument xmlDoc = new()
{
    PreserveWhitespace = true
};
xmlDoc.Load("test.xml");
```

3. Find the specified element in the XmlDocument object and create a new XmlElement object to represent the element you want to encrypt. In this example, the `"creditcard"` element is encrypted.

```
C#

XmlElement? elementToEncrypt = Doc.GetElementsByTagName(ElementName)[0]
as XmlElement;
```

4. Create a new instance of the EncryptedXml class and use it to encrypt the XmlElement with the symmetric key. The EncryptData method returns the encrypted element as an array of encrypted bytes.

```
C#

EncryptedXml eXml = new();

byte[] encryptedElement = eXml.EncryptData(elementToEncrypt, Key,
false);
```

5. Construct an EncryptedData object and populate it with the URL identifier of the XML Encryption element. This URL identifier lets a decrypting party know that the XML contains an encrypted element. You can use the XmlEncElementUrl field to specify the URL identifier.

```
C#

EncryptedData edElement = new()
{
    Type = EncryptedXml.XmlEncElementUrl
};
```

6. Create an EncryptionMethod object that is initialized to the URL identifier of the cryptographic algorithm used to generate the key. Pass the EncryptionMethod object to the EncryptionMethod property.

```
C#
```

```
string? encryptionMethod;
if (Key is Aes)
{
    encryptionMethod = EncryptedXml.XmlEncAES256Url;
}
else
{
    // Throw an exception if the transform is not AES
    throw new CryptographicException("The specified algorithm is not
supported or not recommended for XML Encryption.");
}

edElement.EncryptionMethod = new EncryptionMethod(encryptionMethod);
```

7. Add the encrypted element data to the EncryptedData object.

C#

```
edElement.CipherData.CipherValue = encryptedElement;
```

8. Replace the element from the original XmlDocument object with the EncryptedData element.

C#

```
EncryptedXml.ReplaceElement(elementToEncrypt, edElement, false);
```

Example

XML

```
<root>
    <creditcard>
        <number>19834209</number>
        <expiry>02/02/2002</expiry>
    </creditcard>
</root>
```

C#

```
using System;
using System.Xml;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;

namespace CSCrypto
```

```
{
    class Program
    {
        static void Main(string[] args)
        {
            Aes? key = null;

            try
            {
                // Create a new AES key.
                key = Aes.Create();
                // Load an XML document.
                XmlDocument xmlDoc = new()
                {
                    PreserveWhitespace = true
                };
                xmlDoc.Load("test.xml");

                // Encrypt the "creditcard" element.
                Encrypt(xmlDoc, "creditcard", key);

                Console.WriteLine("The element was encrypted");

                Console.WriteLine(xmlDoc.InnerXml);

                Decrypt(xmlDoc, key);

                Console.WriteLine("The element was decrypted");

                Console.WriteLine(xmlDoc.InnerXml);
            }
            catch (Exception e)
            {
                Console.WriteLine(e.Message);
            }
            finally
            {
                key?.Clear();
            }
        }

        public static void Encrypt(XmlDocument Doc, string ElementName,
SymmetricAlgorithm Key)
        {
            // Check the arguments.
            ArgumentNullException.ThrowIfNull(Doc);
            ArgumentNullException.ThrowIfNull(ElementName);
            ArgumentNullException.ThrowIfNull(Key);

            ////////////////////////////////////////////////
            // Find the specified element in the XmlDocument
            // object and create a new XmlElement object.
            ////////////////////////////////////////////////
            XmlElement? elementToEncrypt =
Doc.GetElementsByTagName(ElementName)[0] as XmlElement;
```

```
            // Throw an XmlException if the element was not found.
            if (elementToEncrypt == null)
            {
                throw new XmlException("The specified element was not
found");
            }

            ////////////////////////////////////////////////
            // Create a new instance of the EncryptedXml class
            // and use it to encrypt the XmlElement with the
            // symmetric key.
            ////////////////////////////////////////////////

            EncryptedXml eXml = new();

            byte[] encryptedElement = eXml.EncryptData(elementToEncrypt,
Key, false);
            ////////////////////////////////////////////////
            // Construct an EncryptedData object and populate
            // it with the desired encryption information.
            ////////////////////////////////////////////////

            EncryptedData edElement = new()
            {
                Type = EncryptedXml.XmlEncElementUrl
            };

            // Create an EncryptionMethod element so that the
            // receiver knows which algorithm to use for decryption.
            // Determine what kind of algorithm is being used and
            // supply the appropriate URL to the EncryptionMethod element.

            string? encryptionMethod;
            if (Key is Aes)
            {
                encryptionMethod = EncryptedXml.XmlEncAES256Url;
            }
            else
            {
                // Throw an exception if the transform is not AES
                throw new CryptographicException("The specified algorithm is
not supported or not recommended for XML Encryption.");
            }

            edElement.EncryptionMethod = new
EncryptionMethod(encryptionMethod);

            // Add the encrypted element data to the
            // EncryptedData object.
            edElement.CipherData.CipherValue = encryptedElement;

            ////////////////////////////////////////////////////
            // Replace the element from the original XmlDocument
            // object with the EncryptedData element.
            ////////////////////////////////////////////////////
```

```
            EncryptedXml.ReplaceElement(elementToEncrypt, edElement, false);
        }

        public static void Decrypt(XmlDocument Doc, SymmetricAlgorithm Alg)
        {
            // Check the arguments.
            ArgumentNullException.ThrowIfNull(Doc);
            ArgumentNullException.ThrowIfNull(Alg);

            // Find the EncryptedData element in the XmlDocument.
            XmlElement? encryptedElement =
Doc.GetElementsByTagName("EncryptedData")[0] as XmlElement;

            // If the EncryptedData element was not found, throw an
exception.
            if (encryptedElement == null)
            {
                throw new XmlException("The EncryptedData element was not
found.");
            }

            // Create an EncryptedData object and populate it.
            EncryptedData edElement = new();
            edElement.LoadXml(encryptedElement);

            // Create a new EncryptedXml object.
            EncryptedXml exml = new();

            // Decrypt the element using the symmetric key.
            byte[] rgbOutput = exml.DecryptData(edElement, Alg);

            // Replace the encryptedData element with the plaintext XML
element.
            exml.ReplaceData(encryptedElement, rgbOutput);
        }
    }
}
```

Compiling the Code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.
- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.
- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET Security

Never store a cryptographic key in plaintext or transfer a key between machines in plaintext. Instead, use a secure key container to store cryptographic keys.

When you are done using a cryptographic key, clear it from memory by setting each byte to zero or by calling the Clear method of the managed cryptography class.

See also

- Cryptography Model - Describes how cryptography is implemented in the base class library.
- Cryptographic Services
- Cross-Platform Cryptography
- System.Security.Cryptography.Xml
- How to: Decrypt XML Elements with Symmetric Keys
- ASP.NET Core Data Protection

How to: Decrypt XML Elements with Symmetric Keys

Article • 09/15/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to encrypt an element within an XML document. XML Encryption allows you to store or transport sensitive XML, without worrying about the data being easily read. This code example decrypts an XML element using the Advanced Encryption Standard (AES) algorithm.

For information about how to encrypt an XML element using this procedure, see How to: Encrypt XML Elements with Symmetric Keys.

When you use a symmetric algorithm like AES to encrypt XML data, you must use the same key to encrypt and decrypt the XML data. The example in this procedure assumes that the encrypted XML was encrypted using the same key, and that the encrypting and decrypting parties agree on the algorithm and key to use. This example does not store or encrypt the AES key within the encrypted XML.

This example is appropriate for situations where a single application needs to encrypt data based on a session key stored in memory, or based on a cryptographically strong key derived from a password. For situations where two or more applications need to share encrypted XML data, consider using an encryption scheme based on an asymmetric algorithm or an X.509 certificate.

To decrypt an XML element with a symmetric key

1. Encrypt an XML element with the previously generated key using the techniques described in How to: Encrypt XML Elements with Symmetric Keys.

2. Find the <`EncryptedData`> element (defined by the XML Encryption standard) in an XmlDocument object that contains the encrypted XML and create a new XmlElement object to represent that element.

   C#

   ```
   XmlElement? encryptedElement =
   Doc.GetElementsByTagName("EncryptedData")[0] as XmlElement;
   ```

3. Create an EncryptedData object by loading the raw XML data from the previously created XmlElement object.

```
C#

EncryptedData edElement = new();
edElement.LoadXml(encryptedElement);
```

4. Create a new EncryptedXml object and use it to decrypt the XML data using the same key that was used for encryption.

```
C#

EncryptedXml exml = new();

// Decrypt the element using the symmetric key.
byte[] rgbOutput = exml.DecryptData(edElement, Alg);
```

5. Replace the encrypted element with the newly decrypted plaintext element within the XML document.

```
C#

exml.ReplaceData(encryptedElement, rgbOutput);
```

Example

This example assumes that a file named `"test.xml"` exists in the same directory as the compiled program. It also assumes that `"test.xml"` contains a `"creditcard"` element. You can place the following XML into a file called `test.xml` and use it with this example.

```
XML

<root>
    <creditcard>
        <number>19834209</number>
        <expiry>02/02/2002</expiry>
    </creditcard>
</root>
```

```
C#

using System;
using System.Xml;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;

namespace CSCrypto
```

```
{
    class Program
    {
        static void Main(string[] args)
        {
            Aes? key = null;

            try
            {
                // Create a new AES key.
                key = Aes.Create();
                // Load an XML document.
                XmlDocument xmlDoc = new()
                {
                    PreserveWhitespace = true
                };
                xmlDoc.Load("test.xml");

                // Encrypt the "creditcard" element.
                Encrypt(xmlDoc, "creditcard", key);

                Console.WriteLine("The element was encrypted");

                Console.WriteLine(xmlDoc.InnerXml);

                Decrypt(xmlDoc, key);

                Console.WriteLine("The element was decrypted");

                Console.WriteLine(xmlDoc.InnerXml);
            }
            catch (Exception e)
            {
                Console.WriteLine(e.Message);
            }
            finally
            {
                key?.Clear();
            }
        }

        public static void Encrypt(XmlDocument Doc, string ElementName,
SymmetricAlgorithm Key)
        {
            // Check the arguments.
            ArgumentNullException.ThrowIfNull(Doc);
            ArgumentNullException.ThrowIfNull(ElementName);
            ArgumentNullException.ThrowIfNull(Key);

            ////////////////////////////////////////////////
            // Find the specified element in the XmlDocument
            // object and create a new XmlElement object.
            ////////////////////////////////////////////////
            XmlElement? elementToEncrypt =
Doc.GetElementsByTagName(ElementName)[0] as XmlElement;
```

```
            // Throw an XmlException if the element was not found.
            if (elementToEncrypt == null)
            {
                throw new XmlException("The specified element was not
found");
            }

            ////////////////////////////////////////////////
            // Create a new instance of the EncryptedXml class
            // and use it to encrypt the XmlElement with the
            // symmetric key.
            ////////////////////////////////////////////////

            EncryptedXml eXml = new();

            byte[] encryptedElement = eXml.EncryptData(elementToEncrypt,
Key, false);
            ////////////////////////////////////////////////
            // Construct an EncryptedData object and populate
            // it with the desired encryption information.
            ////////////////////////////////////////////////

            EncryptedData edElement = new()
            {
                Type = EncryptedXml.XmlEncElementUrl
            };

            // Create an EncryptionMethod element so that the
            // receiver knows which algorithm to use for decryption.
            // Determine what kind of algorithm is being used and
            // supply the appropriate URL to the EncryptionMethod element.

            string? encryptionMethod;
            if (Key is Aes)
            {
                encryptionMethod = EncryptedXml.XmlEncAES256Url;
            }
            else
            {
                // Throw an exception if the transform is not AES
                throw new CryptographicException("The specified algorithm is
not supported or not recommended for XML Encryption.");
            }

            edElement.EncryptionMethod = new
EncryptionMethod(encryptionMethod);

            // Add the encrypted element data to the
            // EncryptedData object.
            edElement.CipherData.CipherValue = encryptedElement;

            ////////////////////////////////////////////////////
            // Replace the element from the original XmlDocument
            // object with the EncryptedData element.
            ////////////////////////////////////////////////////
```

```
            EncryptedXml.ReplaceElement(elementToEncrypt, edElement, false);
        }

        public static void Decrypt(XmlDocument Doc, SymmetricAlgorithm Alg)
        {
            // Check the arguments.
            ArgumentNullException.ThrowIfNull(Doc);
            ArgumentNullException.ThrowIfNull(Alg);

            // Find the EncryptedData element in the XmlDocument.
            XmlElement? encryptedElement =
Doc.GetElementsByTagName("EncryptedData")[0] as XmlElement;

            // If the EncryptedData element was not found, throw an
exception.
            if (encryptedElement == null)
            {
                throw new XmlException("The EncryptedData element was not
found.");
            }

            // Create an EncryptedData object and populate it.
            EncryptedData edElement = new();
            edElement.LoadXml(encryptedElement);

            // Create a new EncryptedXml object.
            EncryptedXml exml = new();

            // Decrypt the element using the symmetric key.
            byte[] rgbOutput = exml.DecryptData(edElement, Alg);

            // Replace the encryptedData element with the plaintext XML
element.
            exml.ReplaceData(encryptedElement, rgbOutput);
        }
    }
}
```

Compiling the Code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.
- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.
- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET Security

Never store a cryptographic key in plaintext or transfer a key between machines in plaintext.

When you are done using a symmetric cryptographic key, clear it from memory by setting each byte to zero or by calling the Clear method of the managed cryptography class.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- System.Security.Cryptography.Xml
- How to: Encrypt XML Elements with Symmetric Keys
- ASP.NET Core Data Protection

Encrypt XML elements with asymmetric keys

Article • 09/15/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to encrypt an element within an XML document. XML Encryption is a standard way to exchange or store encrypted XML data, without worrying about the data being easily read. For more information about the XML Encryption standard, see the World Wide Web Consortium (W3C) specification for XML Encryption located at https://www.w3.org/TR/xmldsig-core/.

You can use XML Encryption to replace any XML element or document with an `<EncryptedData>` element that contains the encrypted XML data. The `<EncryptedData>` element can also contain sub elements that include information about the keys and processes used during encryption. XML Encryption allows a document to contain multiple encrypted elements and allows an element to be encrypted multiple times. The code example in this procedure shows how to create an `<EncryptedData>` element along with several other sub elements that you can use later during decryption.

This example encrypts an XML element using two keys. It generates an RSA public/private key pair and saves the key pair to a secure key container. The example then creates a separate session key using the Advanced Encryption Standard (AES) algorithm. The example uses the AES session key to encrypt the XML document and then uses the RSA public key to encrypt the AES session key. Finally, the example saves the encrypted AES session key and the encrypted XML data to the XML document within a new `<EncryptedData>` element.

To decrypt the XML element, you retrieve the RSA private key from the key container, use it to decrypt the session key, and then use the session key to decrypt the document. For more information about how to decrypt an XML element that was encrypted using this procedure, see How to: Decrypt XML Elements with Asymmetric Keys.

This example is appropriate for situations where multiple applications need to share encrypted data or where an application needs to save encrypted data between the times that it runs.

To encrypt an XML element with an asymmetric key

1. Create a CspParameters object and specify the name of the key container.

```
```

```
C#

CspParameters cspParams = new CspParameters();
cspParams.KeyContainerName = "XML_ENC_RSA_KEY";
```

2. Generate an asymmetric key using the RSACryptoServiceProvider class. The key is automatically saved to the key container when you pass the CspParameters object to the constructor of the RSACryptoServiceProvider class. This key will be used to encrypt the AES session key and can be retrieved later to decrypt it.

```
C#

RSACryptoServiceProvider rsaKey = new
RSACryptoServiceProvider(cspParams);
```

3. Create an XmlDocument object by loading an XML file from disk. The XmlDocument object contains the XML element to encrypt.

```
C#

// Create an XmlDocument object.
XmlDocument xmlDoc = new XmlDocument();

// Load an XML file into the XmlDocument object.
try
{
    xmlDoc.PreserveWhitespace = true;
    xmlDoc.Load("test.xml");
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
}
```

4. Find the specified element in the XmlDocument object and create a new XmlElement object to represent the element you want to encrypt. In this example, the `"creditcard"` element is encrypted.

```
C#

XmlElement? elementToEncrypt =
Doc.GetElementsByTagName(ElementToEncrypt)[0] as XmlElement;

// Throw an XmlException if the element was not found.
if (elementToEncrypt == null)
{
```

```
    throw new XmlException("The specified element was not found");
}
```

5. Create a new session key using the Aes class. This key will encrypt the XML element, and then be encrypted itself and placed in the XML document.

```
C#

// Create an AES key.
sessionKey = Aes.Create();
```

6. Create a new instance of the EncryptedXml class and use it to encrypt the specified element using the session key. The EncryptData method returns the encrypted element as an array of encrypted bytes.

```
C#

EncryptedXml eXml = new EncryptedXml();

byte[] encryptedElement = eXml.EncryptData(elementToEncrypt,
sessionKey, false);
```

7. Construct an EncryptedData object and populate it with the URL identifier of the encrypted XML element. This URL identifier lets a decrypting party know that the XML contains an encrypted element. You can use the XmlEncElementUrl field to specify the URL identifier. The plaintext XML element will be replaced by an `<EncryptedData>` element encapsulated by this EncryptedData object.

```
C#

EncryptedData edElement = new EncryptedData();
edElement.Type = EncryptedXml.XmlEncElementUrl;
edElement.Id = EncryptionElementID;
```

8. Create an EncryptionMethod object that is initialized to the URL identifier of the cryptographic algorithm used to generate the session key. Pass the EncryptionMethod object to the EncryptionMethod property.

```
C#

edElement.EncryptionMethod = new
EncryptionMethod(EncryptedXml.XmlEncAES256Url);
```

9. Create an EncryptedKey object to contain the encrypted session key. Encrypt the session key, add it to the EncryptedKey object, and enter a session key name and key identifier URL.

```
C#

EncryptedKey ek = new EncryptedKey();

byte[] encryptedKey = EncryptedXml.EncryptKey(sessionKey.Key, Alg,
false);

ek.CipherData = new CipherData(encryptedKey);

ek.EncryptionMethod = new
EncryptionMethod(EncryptedXml.XmlEncRSA15Url);
```

10. Create a new DataReference object that maps the encrypted data to a particular session key. This optional step allows you to easily specify that multiple parts of an XML document were encrypted by a single key.

```
C#

DataReference dRef = new DataReference();

// Specify the EncryptedData URI.
dRef.Uri = "#" + EncryptionElementID;

// Add the DataReference to the EncryptedKey.
ek.AddReference(dRef);
```

11. Add the encrypted key to the EncryptedData object.

```
C#

edElement.KeyInfo.AddClause(new KeyInfoEncryptedKey(ek));
```

12. Create a new KeyInfo object to specify the name of the RSA key. Add it to the EncryptedData object. This helps the decrypting party identify the correct asymmetric key to use when decrypting the session key.

```
C#


// Create a new KeyInfoName element.
KeyInfoName kin = new KeyInfoName();

// Specify a name for the key.
```

```
kin.Value = KeyName;

// Add the KeyInfoName element to the
// EncryptedKey object.
ek.KeyInfo.AddClause(kin);
```

13. Add the encrypted element data to the EncryptedData object.

```
C#

edElement.CipherData.CipherValue = encryptedElement;
```

14. Replace the element from the original XmlDocument object with the EncryptedData element.

```
C#

EncryptedXml.ReplaceElement(elementToEncrypt, edElement, false);
```

15. Save the XmlDocument object.

```
C#

xmlDoc.Save("test.xml");
```

Example

This example assumes that a file named `"test.xml"` exists in the same directory as the compiled program. It also assumes that `"test.xml"` contains a `"creditcard"` element. You can place the following XML into a file called `test.xml` and use it with this example.

```
XML

<root>
    <creditcard>
        <number>19834209</number>
        <expiry>02/02/2002</expiry>
    </creditcard>
</root>
```

```
C#


using System;
```

```
using System.Xml;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;
using System.Runtime.Versioning;

[SupportedOSPlatform("windows")]
class Program
{
    static void Main(string[] args)
    {
        // Create an XmlDocument object.
        XmlDocument xmlDoc = new XmlDocument();

        // Load an XML file into the XmlDocument object.
        try
        {
            xmlDoc.PreserveWhitespace = true;
            xmlDoc.Load("test.xml");
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }

        // Create a new CspParameters object to specify
        // a key container.
        CspParameters cspParams = new CspParameters();
        cspParams.KeyContainerName = "XML_ENC_RSA_KEY";

        // Create a new RSA key and save it in the container.  This key will
encrypt
        // a symmetric key, which will then be encrypted in the XML
document.
        RSACryptoServiceProvider rsaKey = new
RSACryptoServiceProvider(cspParams);

        try
        {
            // Encrypt the "creditcard" element.
            Encrypt(xmlDoc, "creditcard", "EncryptedElement1", rsaKey,
"rsaKey");

            // Save the XML document.
            xmlDoc.Save("test.xml");

            // Display the encrypted XML to the console.
            Console.WriteLine("Encrypted XML:");
            Console.WriteLine();
            Console.WriteLine(xmlDoc.OuterXml);
            Decrypt(xmlDoc, rsaKey, "rsaKey");
            xmlDoc.Save("test.xml");
            // Display the encrypted XML to the console.
            Console.WriteLine();
            Console.WriteLine("Decrypted XML:");
            Console.WriteLine();
```

```
            Console.WriteLine(xmlDoc.OuterXml);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        finally
        {
            // Clear the RSA key.
            rsaKey.Clear();
        }

        Console.ReadLine();
    }

    public static void Encrypt(XmlDocument Doc, string ElementToEncrypt,
string EncryptionElementID, RSA Alg, string KeyName)
    {
        // Check the arguments.
        if (Doc == null)
            throw new ArgumentNullException("Doc");
        if (ElementToEncrypt == null)
            throw new ArgumentNullException("ElementToEncrypt");
        if (EncryptionElementID == null)
            throw new ArgumentNullException("EncryptionElementID");
        if (Alg == null)
            throw new ArgumentNullException("Alg");
        if (KeyName == null)
            throw new ArgumentNullException("KeyName");

        ////////////////////////////////////////////////
        // Find the specified element in the XmlDocument
        // object and create a new XmlElement object.
        ////////////////////////////////////////////////
        XmlElement? elementToEncrypt =
Doc.GetElementsByTagName(ElementToEncrypt)[0] as XmlElement;

        // Throw an XmlException if the element was not found.
        if (elementToEncrypt == null)
        {
            throw new XmlException("The specified element was not found");
        }
        Aes? sessionKey = null;

        try
        {
            //////////////////////////////////////////////////
            // Create a new instance of the EncryptedXml class
            // and use it to encrypt the XmlElement with the
            // a new random symmetric key.
            //////////////////////////////////////////////////

            // Create an AES key.
            sessionKey = Aes.Create();
```

```
            EncryptedXml eXml = new EncryptedXml();

            byte[] encryptedElement = eXml.EncryptData(elementToEncrypt,
sessionKey, false);
            ////////////////////////////////////////////////
            // Construct an EncryptedData object and populate
            // it with the desired encryption information.
            ////////////////////////////////////////////////

            EncryptedData edElement = new EncryptedData();
            edElement.Type = EncryptedXml.XmlEncElementUrl;
            edElement.Id = EncryptionElementID;
            // Create an EncryptionMethod element so that the
            // receiver knows which algorithm to use for decryption.

            edElement.EncryptionMethod = new
EncryptionMethod(EncryptedXml.XmlEncAES256Url);
            // Encrypt the session key and add it to an EncryptedKey
element.
            EncryptedKey ek = new EncryptedKey();

            byte[] encryptedKey = EncryptedXml.EncryptKey(sessionKey.Key,
Alg, false);

            ek.CipherData = new CipherData(encryptedKey);

            ek.EncryptionMethod = new
EncryptionMethod(EncryptedXml.XmlEncRSA15Url);

            // Create a new DataReference element
            // for the KeyInfo element.  This optional
            // element specifies which EncryptedData
            // uses this key.  An XML document can have
            // multiple EncryptedData elements that use
            // different keys.
            DataReference dRef = new DataReference();

            // Specify the EncryptedData URI.
            dRef.Uri = "#" + EncryptionElementID;

            // Add the DataReference to the EncryptedKey.
            ek.AddReference(dRef);
            // Add the encrypted key to the
            // EncryptedData object.

            edElement.KeyInfo.AddClause(new KeyInfoEncryptedKey(ek));
            // Set the KeyInfo element to specify the
            // name of the RSA key.


            // Create a new KeyInfoName element.
            KeyInfoName kin = new KeyInfoName();

            // Specify a name for the key.
            kin.Value = KeyName;
```

```
            // Add the KeyInfoName element to the
            // EncryptedKey object.
            ek.KeyInfo.AddClause(kin);
            // Add the encrypted element data to the
            // EncryptedData object.
            edElement.CipherData.CipherValue = encryptedElement;
            //////////////////////////////////////////////////
            // Replace the element from the original XmlDocument
            // object with the EncryptedData element.
            //////////////////////////////////////////////////
            EncryptedXml.ReplaceElement(elementToEncrypt, edElement, false);
        }
        finally
        {
            sessionKey?.Clear();
        }
    }

    public static void Decrypt(XmlDocument Doc, RSA Alg, string KeyName)
    {
        // Check the arguments.
        if (Doc == null)
            throw new ArgumentNullException("Doc");
        if (Alg == null)
            throw new ArgumentNullException("Alg");
        if (KeyName == null)
            throw new ArgumentNullException("KeyName");

        // Create a new EncryptedXml object.
        EncryptedXml exml = new EncryptedXml(Doc);

        // Add a key-name mapping.
        // This method can only decrypt documents
        // that present the specified key name.
        exml.AddKeyNameMapping(KeyName, Alg);

        // Decrypt the element.
        exml.DecryptDocument();
    }
}
```

Compiling the code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.
- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.

- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET security

Never store a symmetric cryptographic key in plaintext or transfer a symmetric key between machines in plaintext. Additionally, never store or transfer the private key of an asymmetric key pair in plaintext. For more information about symmetric and asymmetric cryptographic keys, see Generating Keys for Encryption and Decryption.

> **Tip**
>
> For development, use **Secret Manager** for secure secret storage. In production, consider a product like **Azure Key Vault**.

Never embed a key directly into your source code. Embedded keys can be easily read from an assembly using the Ildasm.exe (IL Disassembler) or by opening the assembly in a text editor such as Notepad.

When you are done using a cryptographic key, clear it from memory by setting each byte to zero or by calling the Clear method of the managed cryptography class. Cryptographic keys can sometimes be read from memory by a debugger or read from a hard drive if the memory location is paged to disk.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography- System.Security.Cryptography.Xml
- How to: Decrypt XML Elements with Asymmetric Keys
- ASP.NET Core Data Protection

How to: Decrypt XML Elements with Asymmetric Keys

Article • 09/15/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to encrypt and decrypt an element within an XML document. XML Encryption is a standard way to exchange or store encrypted XML data, without worrying about the data being easily read. For more information about the XML Encryption standard, see the World Wide Web Consortium (W3C) recommendation XML Signature Syntax and Processing.

> **Note**
>
> The code in this article applies to Windows.

The example in this procedure decrypts an XML element that was encrypted using the methods described in How to: Encrypt XML Elements with Asymmetric Keys. It finds an <`EncryptedData`> element, decrypts the element, and then replaces the element with the original plaintext XML element.

This example decrypts an XML element using two keys. It retrieves a previously generated RSA private key from a key container, and then uses the RSA key to decrypt a session key stored in the <`EncryptedKey`> element of the <`EncryptedData`> element. The example then uses the session key to decrypt the XML element.

This example is appropriate for situations where multiple applications have to share encrypted data or where an application has to save encrypted data between the times that it runs.

To decrypt an XML element with an asymmetric key

1. Create a CspParameters object and specify the name of the key container.

   C#

   ```
   CspParameters cspParams = new CspParameters();
   cspParams.KeyContainerName = "XML_ENC_RSA_KEY";
   ```

2. Retrieve a previously generated asymmetric key from the container using the RSACryptoServiceProvider object. The key is automatically retrieved from the key

container when you pass the CspParameters object to the RSACryptoServiceProvider constructor.

```
C#

RSACryptoServiceProvider rsaKey = new
RSACryptoServiceProvider(cspParams);
```

3. Create a new EncryptedXml object to decrypt the document.

```
C#

// Create a new EncryptedXml object.
EncryptedXml exml = new EncryptedXml(Doc);
```

4. Add a key/name mapping to associate the RSA key with the element within the document that should be decrypted. You must use the same name for the key that you used when you encrypted the document. Note that this name is separate from the name used to identify the key in the key container specified in step 1.

```
C#

exml.AddKeyNameMapping(KeyName, Alg);
```

5. Call the DecryptDocument method to decrypt the <`EncryptedData`> element. This method uses the RSA key to decrypt the session key and automatically uses the session key to decrypt the XML element. It also automatically replaces the <`EncryptedData`> element with the original plaintext.

```
C#

exml.DecryptDocument();
```

6. Save the XML document.

```
C#

xmlDoc.Save("test.xml");
```

Example

This example assumes that a file named `test.xml` exists in the same directory as the compiled program. It also assumes that `test.xml` contains an XML element that was encrypted using the techniques described in How to: Encrypt XML Elements with Asymmetric Keys.

C#

```
using System;
using System.Xml;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;
using System.Runtime.Versioning;

[SupportedOSPlatform("windows")]
class Program
{
    static void Main(string[] args)
    {
        // Create an XmlDocument object.
        XmlDocument xmlDoc = new XmlDocument();

        // Load an XML file into the XmlDocument object.
        try
        {
            xmlDoc.PreserveWhitespace = true;
            xmlDoc.Load("test.xml");
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        CspParameters cspParams = new CspParameters();
        cspParams.KeyContainerName = "XML_ENC_RSA_KEY";

        // Get the RSA key from the key container.  This key will decrypt
        // a symmetric key that was imbedded in the XML document.
        RSACryptoServiceProvider rsaKey = new
RSACryptoServiceProvider(cspParams);

        try
        {

            // Decrypt the elements.
            Decrypt(xmlDoc, rsaKey, "rsaKey");

            // Save the XML document.
            xmlDoc.Save("test.xml");

            // Display the encrypted XML to the console.
            Console.WriteLine();
            Console.WriteLine("Decrypted XML:");
            Console.WriteLine();
```

```
            Console.WriteLine(xmlDoc.OuterXml);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
        finally
        {
            // Clear the RSA key.
            rsaKey.Clear();
        }

        Console.ReadLine();
    }

    public static void Decrypt(XmlDocument Doc, RSA Alg, string KeyName)
    {
        // Check the arguments.
        if (Doc == null)
            throw new ArgumentNullException("Doc");
        if (Alg == null)
            throw new ArgumentNullException("Alg");
        if (KeyName == null)
            throw new ArgumentNullException("KeyName");
        // Create a new EncryptedXml object.
        EncryptedXml exml = new EncryptedXml(Doc);

        // Add a key-name mapping.
        // This method can only decrypt documents
        // that present the specified key name.
        exml.AddKeyNameMapping(KeyName, Alg);

        // Decrypt the element.
        exml.DecryptDocument();
    }
}
```

Compiling the Code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.
- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.
- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET Security

Never store a symmetric cryptographic key in plaintext or transfer a symmetric key between machines in plaintext. Additionally, never store or transfer the private key of an asymmetric key pair in plaintext. For more information about symmetric and asymmetric cryptographic keys, see Generating Keys for Encryption and Decryption.

Never embed a key directly into your source code. Embedded keys can be easily read from an assembly by using Ildasm.exe (IL Disassembler) or by opening the assembly in a text editor such as Notepad.

When you are done using a cryptographic key, clear it from memory by setting each byte to zero or by calling the Clear method of the managed cryptography class. Cryptographic keys can sometimes be read from memory by a debugger or read from a hard drive if the memory location is paged to disk.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- System.Security.Cryptography.Xml
- How to: Encrypt XML Elements with Asymmetric Keys
- ASP.NET Core Data Protection

How to: Encrypt XML Elements with X.509 Certificates

Article • 09/15/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to encrypt an element within an XML document. XML Encryption is a standard way to exchange or store encrypted XML data, without worrying about the data being easily read. For more information about the XML Encryption standard, see the World Wide Web Consortium (W3C) specification for XML Encryption located at https://www.w3.org/TR/xmldsig-core/.

You can use XML Encryption to replace any XML element or document with an <`EncryptedData`> element that contains the encrypted XML data. The <`EncryptedData`> element can contain sub elements that include information about the keys and processes used during encryption. XML Encryption allows a document to contain multiple encrypted elements and allows an element to be encrypted multiple times. The code example in this procedure shows you how to create an <`EncryptedData`> element along with several other sub elements that you can use later during decryption.

This example encrypts an XML element using two keys. The example programmatically retrieves a certificate and uses it to encrypt an XML element using the Encrypt method. Internally, the Encrypt method creates a separate session key and uses it to encrypt the XML document. This method encrypts the session key and saves it along with the encrypted XML within a new <`EncryptedData`> element.

To decrypt the XML element, call the DecryptDocument method, which automatically retrieves the X.509 certificate from the store and performs the necessary decryption. For more information about how to decrypt an XML element that was encrypted using this procedure, see How to: Decrypt XML Elements with X.509 Certificates.

This example is appropriate for situations where multiple applications need to share encrypted data or where an application needs to save encrypted data between the times that it runs.

To encrypt an XML element with an X.509 certificate

To run this example, you need to create a test certificate and save it in a certificate store. Instructions for that task are provided only for the Windows Certificate Creation Tool (Makecert.exe).

1. Use Makecert.exe to generate a test X.509 certificate and place it in the local user store. You must generate an exchange key and you must make the key exportable. Run the following command:

```
Console

makecert -r -pe -n "CN=XML_ENC_TEST_CERT" -b 01/01/2020 -e 01/01/2025 -
sky exchange -ss my
```

2. Create an X509Store object and initialize it to open the current user store.

```
C#

X509Store store = new X509Store(StoreLocation.CurrentUser);
```

3. Open the store in read-only mode.

```
C#

store.Open(OpenFlags.ReadOnly);
```

4. Initialize an X509Certificate2Collection with all of the certificates in the store.

```
C#

X509Certificate2Collection certCollection = store.Certificates;
```

5. Enumerate through the certificates in the store and find the certificate with the appropriate name. In this example, the certificate is named `"CN=XML_ENC_TEST_CERT"`.

```
C#

X509Certificate2 cert = null;

// Loop through each certificate and find the certificate
// with the appropriate name.
foreach (X509Certificate2 c in certCollection)
{
    if (c.Subject == "CN=XML_ENC_TEST_CERT")
    {
        cert = c;

        break;
```

```
    }
}
```

6. Close the store after the certificate is located.

```
C#

store.Close();
```

7. Create an XmlDocument object by loading an XML file from disk. The XmlDocument object contains the XML element to encrypt.

```
C#

XmlDocument xmlDoc = new XmlDocument();
```

8. Find the specified element in the XmlDocument object and create a new XmlElement object to represent the element you want to encrypt. In this example, the `"creditcard"` element is encrypted.

```
C#

XmlElement elementToEncrypt =
Doc.GetElementsByTagName(ElementToEncrypt)[0] as XmlElement;
```

9. Create a new instance of the EncryptedXml class and use it to encrypt the specified element using the X.509 certificate. The Encrypt method returns the encrypted element as an EncryptedData object.

```
C#

EncryptedXml eXml = new EncryptedXml();

// Encrypt the element.
EncryptedData edElement = eXml.Encrypt(elementToEncrypt, Cert);
```

10. Replace the element from the original XmlDocument object with the EncryptedData element.

```
C#

EncryptedXml.ReplaceElement(elementToEncrypt, edElement, false);
```

11. Save the XmlDocument object.

C#

```
xmlDoc.Save("test.xml");
```

Example

This example assumes that a file named `"test.xml"` exists in the same directory as the compiled program. It also assumes that `"test.xml"` contains a `"creditcard"` element. You can place the following XML into a file called `test.xml` and use it with this example.

XML

```
<root>
    <creditcard>
        <number>19834209</number>
        <expiry>02/02/2002</expiry>
    </creditcard>
</root>
```

C#

```
using System;
using System.Xml;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;
using System.Security.Cryptography.X509Certificates;

class Program
{
    static void Main(string[] args)
    {
        try
        {
            // Create an XmlDocument object.
            XmlDocument xmlDoc = new XmlDocument();

            // Load an XML file into the XmlDocument object.
            xmlDoc.PreserveWhitespace = true;
            xmlDoc.Load("test.xml");

            // Open the X.509 "Current User" store in read only mode.
            X509Store store = new X509Store(StoreLocation.CurrentUser);
            store.Open(OpenFlags.ReadOnly);

            // Place all certificates in an X509Certificate2Collection
object.
            X509Certificate2Collection certCollection = store.Certificates;

            X509Certificate2 cert = null;
```

```
            // Loop through each certificate and find the certificate
            // with the appropriate name.
            foreach (X509Certificate2 c in certCollection)
            {
                if (c.Subject == "CN=XML_ENC_TEST_CERT")
                {
                    cert = c;

                    break;
                }
            }

            if (cert == null)
            {
                throw new CryptographicException("The X.509 certificate
could not be found.");
            }

            // Close the store.
            store.Close();

            // Encrypt the "creditcard" element.
            Encrypt(xmlDoc, "creditcard", cert);

            // Save the XML document.
            xmlDoc.Save("test.xml");

            // Display the encrypted XML to the console.
            Console.WriteLine("Encrypted XML:");
            Console.WriteLine();
            Console.WriteLine(xmlDoc.OuterXml);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }

    public static void Encrypt(XmlDocument Doc, string ElementToEncrypt,
X509Certificate2 Cert)
    {
        // Check the arguments.
        if (Doc == null)
            throw new ArgumentNullException("Doc");
        if (ElementToEncrypt == null)
            throw new ArgumentNullException("ElementToEncrypt");
        if (Cert == null)
            throw new ArgumentNullException("Cert");

        ////////////////////////////////////////////////
        // Find the specified element in the XmlDocument
        // object and create a new XmlElement object.
        ////////////////////////////////////////////////
```

```
        XmlElement elementToEncrypt =
Doc.GetElementsByTagName(ElementToEncrypt)[0] as XmlElement;
        // Throw an XmlException if the element was not found.
        if (elementToEncrypt == null)
        {
            throw new XmlException("The specified element was not found");
        }

        //////////////////////////////////////////////////
        // Create a new instance of the EncryptedXml class
        // and use it to encrypt the XmlElement with the
        // X.509 Certificate.
        //////////////////////////////////////////////////

        EncryptedXml eXml = new EncryptedXml();

        // Encrypt the element.
        EncryptedData edElement = eXml.Encrypt(elementToEncrypt, Cert);

        //////////////////////////////////////////////////
        // Replace the element from the original XmlDocument
        // object with the EncryptedData element.
        //////////////////////////////////////////////////
        EncryptedXml.ReplaceElement(elementToEncrypt, edElement, false);
    }
}
```

Compiling the Code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.
- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.
- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET Security

The X.509 certificate used in this example is for test purposes only. Applications should use an X.509 certificate generated by a trusted certificate authority.

See also

- Cryptography Model

- Cryptographic Services
- Cross-Platform Cryptography
- System.Security.Cryptography.Xml
- How to: Decrypt XML Elements with X.509 Certificates
- ASP.NET Core Data Protection

How to: Decrypt XML Elements with X.509 Certificates

Article • 09/15/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to encrypt and decrypt an element within an XML document. XML Encryption is a standard way to exchange or store encrypted XML data, without worrying about the data being easily read. For more information about the XML Encryption standard, see the World Wide Web Consortium (W3C) specification for XML Encryption located at https://www.w3.org/TR/xmldsig-core/.

This example decrypts an XML element that was encrypted using the methods described in: How to: Encrypt XML Elements with X.509 Certificates. It finds an <`EncryptedData`> element, decrypts the element, and then replaces the element with the original plaintext XML element.

The code example in this procedure decrypts an XML element using an X.509 certificate from the local certificate store of the current user account. The example uses the DecryptDocument method to automatically retrieve the X.509 certificate and decrypt a session key stored in the <`EncryptedKey`> element of the <`EncryptedData`> element. The DecryptDocument method then automatically uses the session key to decrypt the XML element.

This example is appropriate for situations where multiple applications need to share encrypted data or where an application needs to save encrypted data between the times that it runs.

To decrypt an XML element with an X.509 certificate

1. Create an XmlDocument object by loading an XML file from disk. The XmlDocument object contains the XML element to decrypt.

   C#

   ```
   XmlDocument xmlDoc = new XmlDocument();
   ```

2. Create a new EncryptedXml object by passing the XmlDocument object to the constructor.

   C#

```
EncryptedXml exml = new EncryptedXml(Doc);
```

3. Decrypt the XML document using the DecryptDocument method.

```
C#

exml.DecryptDocument();
```

4. Save the XmlDocument object.

```
C#

xmlDoc.Save("test.xml");
```

Example

This example assumes that a file named `"test.xml"` exists in the same directory as the compiled program. It also assumes that `"test.xml"` contains a `"creditcard"` element. You can place the following XML into a file called `test.xml` and use it with this example.

```
XML

<root>
    <creditcard>
        <number>19834209</number>
        <expiry>02/02/2002</expiry>
    </creditcard>
</root>
```

```
C#

using System;
using System.Xml;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;
using System.Security.Cryptography.X509Certificates;

class Program
{
    static void Main(string[] args)
    {
        try
        {
            // Create an XmlDocument object.
            XmlDocument xmlDoc = new XmlDocument();
```

```
            // Load an XML file into the XmlDocument object.
            xmlDoc.PreserveWhitespace = true;
            xmlDoc.Load("test.xml");

            // Decrypt the document.
            Decrypt(xmlDoc);

            // Save the XML document.
            xmlDoc.Save("test.xml");

            // Display the decrypted XML to the console.
            Console.WriteLine("Decrypted XML:");
            Console.WriteLine();
            Console.WriteLine(xmlDoc.OuterXml);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }

    public static void Decrypt(XmlDocument Doc)
    {
        // Check the arguments.
        if (Doc == null)
            throw new ArgumentNullException("Doc");

        // Create a new EncryptedXml object.
        EncryptedXml exml = new EncryptedXml(Doc);

        // Decrypt the XML document.
        exml.DecryptDocument();
    }
}
```

Compiling the Code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.
- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.
- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET Security

The X.509 certificate used in this example is for test purposes only. Applications should use an X.509 certificate generated by a trusted certificate authority.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- System.Security.Cryptography.Xml
- How to: Encrypt XML Elements with X.509 Certificates
- ASP.NET Core Data Protection

How to: Sign XML Documents with Digital Signatures

Article • 09/15/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to sign an XML document or part of an XML document with a digital signature. XML digital signatures (XMLDSIG) allow you to verify that data was not altered after it was signed. For more information about the XMLDSIG standard, see the World Wide Web Consortium (W3C) recommendation XML Signature Syntax and Processing.

> **Note**
>
> The code in this article applies to Windows.

The code example in this procedure demonstrates how to digitally sign an entire XML document and attach the signature to the document in a <`Signature`> element. The example creates an RSA signing key, adds the key to a secure key container, and then uses the key to digitally sign an XML document. The key can then be retrieved to verify the XML digital signature, or can be used to sign another XML document.

For information about how to verify an XML digital signature that was created using this procedure, see How to: Verify the Digital Signatures of XML Documents.

To digitally sign an XML document

1. Create a CspParameters object and specify the name of the key container.

   C#

   ```
   CspParameters cspParams = new()
   {
       KeyContainerName = "XML_DSIG_RSA_KEY"
   };
   ```

2. Generate an asymmetric key using the RSACryptoServiceProvider class. The key is automatically saved to the key container when you pass the CspParameters object to the constructor of the RSACryptoServiceProvider class. This key will be used to sign the XML document.

   C#

```
RSACryptoServiceProvider rsaKey = new(cspParams);
```

3. Create an XmlDocument object by loading an XML file from disk. The XmlDocument object contains the XML element to encrypt.

```
C#

XmlDocument xmlDoc = new()
{
    // Load an XML file into the XmlDocument object.
    PreserveWhitespace = true
};
xmlDoc.Load("test.xml");
```

4. Create a new SignedXml object and pass the XmlDocument object to it.

```
C#

SignedXml signedXml = new(xmlDoc)
{
```

5. Add the signing RSA key to the SignedXml object.

```
C#

    SigningKey = rsaKey
};
```

6. Create a Reference object that describes what to sign. To sign the entire document, set the Uri property to `""`.

```
C#

// Create a reference to be signed.
Reference reference = new()
{
    Uri = ""
};
```

7. Add an XmlDsigEnvelopedSignatureTransform object to the Reference object. A transformation allows the verifier to represent the XML data in the identical manner that the signer used. XML data can be represented in different ways, so this step is vital to verification.

```
```

```
C#

XmlDsigEnvelopedSignatureTransform env = new();
reference.AddTransform(env);
```

8. Add the Reference object to the SignedXml object.

```
C#

signedXml.AddReference(reference);
```

9. Compute the signature by calling the ComputeSignature method.

```
C#

signedXml.ComputeSignature();
```

10. Retrieve the XML representation of the signature (a <Signature> element) and save it to a new XmlElement object.

```
C#

XmlElement xmlDigitalSignature = signedXml.GetXml();
```

11. Append the element to the XmlDocument object.

```
C#

xmlDoc.DocumentElement?.AppendChild(xmlDoc.ImportNode(xmlDigitalSignatu
re, true));
```

12. Save the document.

```
C#

xmlDoc.Save("test.xml");
```

Example

This example assumes that a file named `test.xml` exists in the same directory as the compiled program. You can place the following XML into a file called `test.xml` and use it with this example.

XML

```
<root>
    <creditcard>
        <number>19834209</number>
        <expiry>02/02/2002</expiry>
    </creditcard>
</root>
```

C#

```
using System;
using System.Runtime.Versioning;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;
using System.Xml;

[SupportedOSPlatform("Windows")]
public class SignXML
{
    public static void Main(String[] args)
    {
        try
        {
            // Create a new CspParameters object to specify
            // a key container.
            CspParameters cspParams = new()
            {
                KeyContainerName = "XML_DSIG_RSA_KEY"
            };

            // Create a new RSA signing key and save it in the container.
            RSACryptoServiceProvider rsaKey = new(cspParams);

            // Create a new XML document.
            XmlDocument xmlDoc = new()
            {
                // Load an XML file into the XmlDocument object.
                PreserveWhitespace = true
            };
            xmlDoc.Load("test.xml");

            // Sign the XML document.
            SignXml(xmlDoc, rsaKey);

            Console.WriteLine("XML file signed.");

            // Save the document.
            xmlDoc.Save("test.xml");
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
```

```
        }
    }

    // Sign an XML file.
    // This document cannot be verified unless the verifying
    // code has the key with which it was signed.
    public static void SignXml(XmlDocument xmlDoc, RSA rsaKey)
    {
        // Check arguments.
        if (xmlDoc == null)
            throw new ArgumentException(null, nameof(xmlDoc));
        if (rsaKey == null)
            throw new ArgumentException(null, nameof(rsaKey));

        // Create a SignedXml object.
        SignedXml signedXml = new(xmlDoc)
        {

            // Add the key to the SignedXml document.
            SigningKey = rsaKey
        };

        // Create a reference to be signed.
        Reference reference = new()
        {
            Uri = ""
        };

        // Add an enveloped transformation to the reference.
        XmlDsigEnvelopedSignatureTransform env = new();
        reference.AddTransform(env);

        // Add the reference to the SignedXml object.
        signedXml.AddReference(reference);

        // Compute the signature.
        signedXml.ComputeSignature();

        // Get the XML representation of the signature and save
        // it to an XmlElement object.
        XmlElement xmlDigitalSignature = signedXml.GetXml();

        // Append the element to the XML document.

xmlDoc.DocumentElement?.AppendChild(xmlDoc.ImportNode(xmlDigitalSignature,
true));
    }
}
```

Compiling the Code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.
- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.
- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET Security

Never store or transfer the private key of an asymmetric key pair in plaintext. For more information about symmetric and asymmetric cryptographic keys, see Generating Keys for Encryption and Decryption.

Never embed a private key directly into your source code. Embedded keys can be easily read from an assembly using the Ildasm.exe (IL Disassembler) or by opening the assembly in a text editor such as Notepad.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- System.Security.Cryptography.Xml
- How to: Verify the Digital Signatures of XML Documents
- ASP.NET Core Data Protection

How to: Verify the Digital Signatures of XML Documents

Article • 09/29/2021

You can use the classes in the System.Security.Cryptography.Xml namespace to verify XML data signed with a digital signature. XML digital signatures (XMLDSIG) allow you to verify that data was not altered after it was signed. For more information about the XMLDSIG standard, see the World Wide Web Consortium (W3C) specification at https://www.w3.org/TR/xmldsig-core/ .

> **Note**
>
> The code in this article applies to Windows.

The code example in this procedure demonstrates how to verify an XML digital signature contained in a <`Signature`> element. The example retrieves an RSA public key from a key container and then uses the key to verify the signature.

For information about how to create a digital signature that can be verified using this technique, see How to: Sign XML Documents with Digital Signatures.

To verify the digital signature of an XML document

1. To verify the document, you must use the same asymmetric key that was used for signing. Create a CspParameters object and specify the name of the key container that was used for signing.

   C#

   ```
   CspParameters cspParams = new()
   {
       KeyContainerName = "XML_DSIG_RSA_KEY"
   };
   ```

2. Retrieve the public key using the RSACryptoServiceProvider class. The key is automatically loaded from the key container by name when you pass the CspParameters object to the constructor of the RSACryptoServiceProvider class.

   C#

```
RSACryptoServiceProvider rsaKey = new(cspParams);
```

3. Create an XmlDocument object by loading an XML file from disk. The XmlDocument object contains the signed XML document to verify.

```
C#

XmlDocument xmlDoc = new()
{
    // Load an XML file into the XmlDocument object.
    PreserveWhitespace = true
};
xmlDoc.Load("test.xml");
```

4. Create a new SignedXml object and pass the XmlDocument object to it.

```
C#

SignedXml signedXml = new(xmlDoc);
```

5. Find the <`signature`> element and create a new XmlNodeList object.

```
C#

XmlNodeList nodeList = xmlDoc.GetElementsByTagName("Signature");
```

6. Load the XML of the first <`signature`> element into the SignedXml object.

```
C#

signedXml.LoadXml((XmlElement?)nodeList[0]);
```

7. Check the signature using the CheckSignature method and the RSA public key. This method returns a Boolean value that indicates success or failure.

```
C#

return signedXml.CheckSignature(key);
```

Example

This example assumes that a file named `"test.xml"` exists in the same directory as the compiled program. The `"test.xml"` file must be signed using the techniques described in How to: Sign XML Documents with Digital Signatures.

C#

```
using System;
using System.Runtime.Versioning;
using System.Security.Cryptography;
using System.Security.Cryptography.Xml;
using System.Xml;

[SupportedOSPlatform("Windows")]
public class VerifyXML
{
    public static void Main(string[] args)
    {
        try
        {
            // Create a new CspParameters object to specify
            // a key container.
            CspParameters cspParams = new()
            {
                KeyContainerName = "XML_DSIG_RSA_KEY"
            };

            // Create a new RSA signing key and save it in the container.
            RSACryptoServiceProvider rsaKey = new(cspParams);

            // Create a new XML document.
            XmlDocument xmlDoc = new()
            {
                // Load an XML file into the XmlDocument object.
                PreserveWhitespace = true
            };
            xmlDoc.Load("test.xml");

            // Verify the signature of the signed XML.
            Console.WriteLine("Verifying signature...");
            bool result = VerifyXml(xmlDoc, rsaKey);

            // Display the results of the signature verification to
            // the console.
            if (result)
            {
                Console.WriteLine("The XML signature is valid.");
            }
            else
            {
                Console.WriteLine("The XML signature is not valid.");
            }
        }
        catch (Exception e)
```

```
        {
            Console.WriteLine(e.Message);
        }
    }

    // Verify the signature of an XML file against an asymmetric
    // algorithm and return the result.
    public static bool VerifyXml(XmlDocument xmlDoc, RSA key)
    {
        // Check arguments.
        if (xmlDoc == null)
             throw new ArgumentException(null, nameof(xmlDoc));
        if (key == null)
            throw new ArgumentException(null, nameof(key));

        // Create a new SignedXml object and pass it
        // the XML document class.
        SignedXml signedXml = new(xmlDoc);

        // Find the "Signature" node and create a new
        // XmlNodeList object.
        XmlNodeList nodeList = xmlDoc.GetElementsByTagName("Signature");

        // Throw an exception if no signature was found.
        if (nodeList.Count <= 0)
        {
            throw new CryptographicException("Verification failed: No
Signature was found in the document.");
        }

        // This example only supports one signature for
        // the entire XML document.  Throw an exception
        // if more than one signature was found.
        if (nodeList.Count >= 2)
        {
            throw new CryptographicException("Verification failed: More that
one signature was found for the document.");
        }

        // Load the first <signature> node.
        signedXml.LoadXml((XmlElement?)nodeList[0]);

        // Check the signature and return the result.
        return signedXml.CheckSignature(key);
    }
}
```

Compiling the Code

- In a project that targets .NET Framework, include a reference to `System.Security.dll`.

- In a project that targets .NET Core or .NET 5, install NuGet package System.Security.Cryptography.Xml.
- Include the following namespaces: System.Xml, System.Security.Cryptography, and System.Security.Cryptography.Xml.

.NET Security

Never store or transfer the private key of an asymmetric key pair in plaintext. For more information about symmetric and asymmetric cryptographic keys, see Generating Keys for Encryption and Decryption.

Never embed a private key directly into your source code. Embedded keys can be easily read from an assembly using the Ildasm.exe (IL Disassembler) or by opening the assembly in a text editor such as Notepad.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- System.Security.Cryptography.Xml
- How to: Sign XML Documents with Digital Signatures
- ASP.NET Core Data Protection

How to: Use Data Protection

Article • 01/31/2023

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see ASP.NET Core Data Protection.

.NET provides access to the data protection API (DPAPI), which allows you to encrypt data using information from the current user account or computer. When you use the DPAPI, you alleviate the difficult problem of explicitly generating and storing a cryptographic key.

Use the ProtectedData class to encrypt a copy of an array of bytes. You can specify that data encrypted by the current user account can be decrypted only by the same user account, or you can specify that data encrypted by the current user account can be decrypted by any account on the computer. See the DataProtectionScope enumeration for a detailed description of ProtectedData options.

Encrypt data to a file or stream using data protection

1. Create random entropy.
2. Call the static Protect method while passing an array of bytes to encrypt, the entropy, and the data protection scope.
3. Write the encrypted data to a file or stream.

To decrypt data from a file or stream using data protection

1. Read the encrypted data from a file or stream.
2. Call the static Unprotect method while passing an array of bytes to decrypt and the data protection scope.

Example

The following code example demonstrates two forms of encryption and decryption. First, the code example encrypts and then decrypts an in-memory array of bytes. Next, the code example encrypts a copy of a byte array, saves it to a file, loads the data back from the file, and then decrypts the data. The example displays the original data, the encrypted data, and the decrypted data.

C#

```
using System;
using System.IO;
using System.Text;
using System.Security.Cryptography;

public class MemoryProtectionSample
{
    public static void Main() => Run();

    public static void Run()
    {
        try
        {
            ////////////////////////////////
            //
            // Memory Encryption - ProtectedMemory
            //
            ////////////////////////////////

            // Create the original data to be encrypted (The data length
should be a multiple of 16).
            byte[] toEncrypt =
UnicodeEncoding.ASCII.GetBytes("ThisIsSomeData16");

            Console.WriteLine($"Original data:
{UnicodeEncoding.ASCII.GetString(toEncrypt)}");
            Console.WriteLine("Encrypting...");

            // Encrypt the data in memory.
            EncryptInMemoryData(toEncrypt, MemoryProtectionScope.SameLogon);

            Console.WriteLine($"Encrypted data:
{UnicodeEncoding.ASCII.GetString(toEncrypt)}");
            Console.WriteLine("Decrypting...");

            // Decrypt the data in memory.
            DecryptInMemoryData(toEncrypt, MemoryProtectionScope.SameLogon);

            Console.WriteLine($"Decrypted data:
{UnicodeEncoding.ASCII.GetString(toEncrypt)}");

            ////////////////////////////////
            //
            // Data Encryption - ProtectedData
            //
```

```
            ////////////////////////////
            // Create the original data to be encrypted
            toEncrypt = UnicodeEncoding.ASCII.GetBytes("This is some data of
any length.");

            // Create a file.
            FileStream fStream = new FileStream("Data.dat",
FileMode.OpenOrCreate);

            // Create some random entropy.
            byte[] entropy = CreateRandomEntropy();

            Console.WriteLine();
            Console.WriteLine($"Original data:
{UnicodeEncoding.ASCII.GetString(toEncrypt)}");
            Console.WriteLine("Encrypting and writing to disk...");

            // Encrypt a copy of the data to the stream.
            int bytesWritten = EncryptDataToStream(toEncrypt, entropy,
DataProtectionScope.CurrentUser, fStream);

            fStream.Close();

            Console.WriteLine("Reading data from disk and decrypting...");

            // Open the file.
            fStream = new FileStream("Data.dat", FileMode.Open);

            // Read from the stream and decrypt the data.
            byte[] decryptData = DecryptDataFromStream(entropy,
DataProtectionScope.CurrentUser, fStream, bytesWritten);

            fStream.Close();

            Console.WriteLine($"Decrypted data:
{UnicodeEncoding.ASCII.GetString(decryptData)}");
        }
        catch (Exception e)
        {
            Console.WriteLine($"ERROR: {e.Message}");
        }
    }

    public static void EncryptInMemoryData(byte[] Buffer,
MemoryProtectionScope Scope )
    {
        if (Buffer == null)
            throw new ArgumentNullException(nameof(Buffer));
        if (Buffer.Length <= 0)
            throw new ArgumentException("The buffer length was 0.",
nameof(Buffer));

        // Encrypt the data in memory. The result is stored in the same
array as the original data.
```

```
        ProtectedMemory.Protect(Buffer, Scope);
    }

    public static void DecryptInMemoryData(byte[] Buffer,
MemoryProtectionScope Scope)
    {
        if (Buffer == null)
            throw new ArgumentNullException(nameof(Buffer));
        if (Buffer.Length <= 0)
            throw new ArgumentException("The buffer length was 0.",
nameof(Buffer));

        // Decrypt the data in memory. The result is stored in the same
array as the original data.
        ProtectedMemory.Unprotect(Buffer, Scope);
    }

    public static byte[] CreateRandomEntropy()
    {
        // Create a byte array to hold the random value.
        byte[] entropy = new byte[16];

        // Create a new instance of the RNGCryptoServiceProvider.
        // Fill the array with a random value.
        new RNGCryptoServiceProvider().GetBytes(entropy);

        // Return the array.
        return entropy;
    }

    public static int EncryptDataToStream(byte[] Buffer, byte[] Entropy,
DataProtectionScope Scope, Stream S)
    {
        if (Buffer == null)
            throw new ArgumentNullException(nameof(Buffer));
        if (Buffer.Length <= 0)
            throw new ArgumentException("The buffer length was 0.",
nameof(Buffer));
        if (Entropy == null)
            throw new ArgumentNullException(nameof(Entropy));
        if (Entropy.Length <= 0)
            throw new ArgumentException("The entropy length was 0.",
nameof(Entropy));
        if (S == null)
            throw new ArgumentNullException(nameof(S));

        int length = 0;

        // Encrypt the data and store the result in a new byte array. The
original data remains unchanged.
        byte[] encryptedData = ProtectedData.Protect(Buffer, Entropy,
Scope);

        // Write the encrypted data to a stream.
        if (S.CanWrite && encryptedData != null)
```

```
        {
            S.Write(encryptedData, 0, encryptedData.Length);

            length = encryptedData.Length;
        }

        // Return the length that was written to the stream.
        return length;
    }

    public static byte[] DecryptDataFromStream(byte[] Entropy,
DataProtectionScope Scope, Stream S, int Length)
    {
        if (S == null)
            throw new ArgumentNullException(nameof(S));
        if (Length <= 0 )
            throw new ArgumentException("The given length was 0.",
nameof(Length));
        if (Entropy == null)
            throw new ArgumentNullException(nameof(Entropy));
        if (Entropy.Length <= 0)
            throw new ArgumentException("The entropy length was 0.",
nameof(Entropy));

        byte[] inBuffer = new byte[Length];
        byte[] outBuffer;

        // Read the encrypted data from a stream.
        if (S.CanRead)
        {
            S.Read(inBuffer, 0, Length);

            outBuffer = ProtectedData.Unprotect(inBuffer, Entropy, Scope);
        }
        else
        {
            throw new IOException("Could not read the stream.");
        }

        // Return the decrypted data
        return outBuffer;
    }
}
```

Compiling the Code

This example compiles and runs only when targeting .NET Framework and running on Windows.

- Include a reference to `System.Security.dll`.

- Include the System, System.IO, System.Security.Cryptography, and System.Text namespace.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- ProtectedMemory
- ProtectedData
- ASP.NET Core Data Protection

How to: Access Hardware Encryption Devices

Article • 09/15/2021

> **Note**
>
> This article applies to Windows.

You can use the CspParameters class to access hardware encryption devices. For example, you can use this class to integrate your application with a smart card, a hardware random number generator, or a hardware implementation of a particular cryptographic algorithm.

The CspParameters class creates a cryptographic service provider (CSP) that accesses a properly installed hardware encryption device. You can verify the availability of a CSP by inspecting the following registry key using the Registry Editor (Regedit.exe): HKEY_LOCAL_MACHINE\Software\Microsoft\Cryptography\Defaults\Provider.

To sign data using a key card

1. Create a new instance of the CspParameters class, passing the integer provider type and the provider name to the constructor.
2. Pass the appropriate flags to the Flags property of the newly created CspParameters object. For example, pass the UseDefaultKeyContainer flag.
3. Create a new instance of an AsymmetricAlgorithm class (for example, the RSACryptoServiceProvider class), passing the CspParameters object to the constructor.
4. Sign your data using one of the `Sign` methods and verify your data using one of the `Verify` methods.

To generate a random number using a hardware random number generator

1. Create a new instance of the CspParameters class, passing the integer provider type and the provider name to the constructor.

2. Create a new instance of the RNGCryptoServiceProvider, passing the CspParameters object to the constructor.

3. Create a random value using the GetBytes or GetNonZeroBytes method.

Example

The following code example demonstrates how to sign data using a smart card. The code example creates a CspParameters object that exposes a smart card, and then initializes an RSACryptoServiceProvider object using the CSP. The code example then signs and verifies some data.

Due to collision problems with SHA1, we recommend SHA256 or better.

C#

```
using System;
using System.Runtime.Versioning;
using System.Security.Cryptography;

namespace SmartCardSign
{
    [SupportedOSPlatform("windows")]
    class SCSign
    {
        static void Main(string[] args)
        {
            // To identify the Smart Card CryptoGraphic Providers on your
            // computer, use the Microsoft Registry Editor (Regedit.exe).
            // The available Smart Card CryptoGraphic Providers are listed
            // in
HKEY_LOCAL_MACHINE\Software\Microsoft\Cryptography\Defaults\Provider.

            // Create a new CspParameters object that identifies a
            // Smart Card CryptoGraphic Provider.
            // The 1st parameter comes from
HKEY_LOCAL_MACHINE\Software\Microsoft\Cryptography\Defaults\Provider Types.
            // The 2nd parameter comes from
HKEY_LOCAL_MACHINE\Software\Microsoft\Cryptography\Defaults\Provider.
            CspParameters csp = new CspParameters(1, "Schlumberger
Cryptographic Service Provider");
            csp.Flags = CspProviderFlags.UseDefaultKeyContainer;

            // Initialize an RSACryptoServiceProvider object using
            // the CspParameters object.
            RSACryptoServiceProvider rsa = new
RSACryptoServiceProvider(csp);

            // Create some data to sign.
            byte[] data = new byte[] { 0, 1, 2, 3, 4, 5, 6, 7 };
```

```
            Console.WriteLine("Data         : " +
BitConverter.ToString(data));

            // Sign the data using the Smart Card CryptoGraphic Provider.
            byte[] sig = rsa.SignData(data, "SHA1");

            Console.WriteLine("Signature    : " +
BitConverter.ToString(sig));

            // Verify the data using the Smart Card CryptoGraphic Provider.
            bool verified = rsa.VerifyData(data, "SHA1", sig);

            Console.WriteLine("Verified     : " + verified);
        }
    }
}
```

Compiling the Code

- Include the System and System.Security.Cryptography namespaces.
- You must have a smart card reader and drivers installed on your computer.
- You must initialize the CspParameters object using information specific to your card reader. For more information, see the documentation of your card reader.

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- ASP.NET Core Data Protection

Walkthrough: Create a Cryptographic Application

Article • 12/02/2021

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see ASP.NET Core Data Protection.

This walkthrough demonstrates how to encrypt and decrypt the contents of a file. The code examples are designed for a Windows Forms application. This application does not demonstrate real-world scenarios, such as using smart cards. Instead, it demonstrates the fundamentals of encryption and decryption.

This walkthrough uses the following guidelines for encryption:

- Use the Aes class, a symmetric algorithm, to encrypt and decrypt data by using its automatically generated Key and IV.
- Use the RSA asymmetric algorithm to encrypt and decrypt the key to the data encrypted by Aes. Asymmetric algorithms are best used for smaller amounts of data, such as a key.

  > **Note**
  >
  > If you want to protect data on your computer instead of exchanging encrypted content with other people, consider using the ProtectedData class.

The following table summarizes the cryptographic tasks in this topic.

Expand table

Task	Description
Create a Windows Forms application	Lists the controls that are required to run the application.
Declare global objects	Declares string path variables, the CspParameters, and the RSACryptoServiceProvider to have global context of the Form class.

Task	Description
Create an asymmetric key	Creates an asymmetric public and private key-value pair and assigns it a key container name.
Encrypt a file	Displays a dialog box to select a file for encryption and encrypts the file.
Decrypt a file	Displays a dialog box to select an encrypted file for decryption and decrypts the file.
Get a private key	Gets the full key pair using the key container name.
Export a public key	Saves the key to an XML file with only public parameters.
Import a public key	Loads the key from an XML file into the key container.
Test the application	Lists procedures for testing this application.

Prerequisites

You need the following components to complete this walkthrough:

- References to the System.IO and System.Security.Cryptography namespaces.

Create a Windows Forms application

Most of the code examples in this walkthrough are designed to be event handlers for button controls. The following table lists the controls required for the sample application and their required names to match the code examples.

Expand table

Control	Name	Text property (as needed)
Button	`buttonEncryptFile`	Encrypt File
Button	`buttonDecryptFile`	Decrypt File
Button	`buttonCreateAsmKeys`	Create Keys
Button	`buttonExportPublicKey`	Export Public Key
Button	`buttonImportPublicKey`	Import Public Key
Button	`buttonGetPrivateKey`	Get Private Key
Label	`label1`	Key not set

Control	Name	Text property (as needed)
OpenFileDialog	`_encryptOpenFileDialog`	
OpenFileDialog	`_decryptOpenFileDialog`	

Double-click the buttons in the Visual Studio designer to create their event handlers.

Declare global objects

Add the following code as part of the declaration of the class Form1. Edit the string variables for your environment and preferences.

C#

```
// Declare CspParameters and RsaCryptoServiceProvider
// objects with global scope of your Form class.
readonly CspParameters _cspp = new CspParameters();
RSACryptoServiceProvider _rsa;

// Path variables for source, encryption, and
// decryption folders. Must end with a backslash.
const string EncrFolder = @"c:\Encrypt\";
const string DecrFolder = @"c:\Decrypt\";
const string SrcFolder = @"c:\docs\";

// Public key file
const string PubKeyFile = @"c:\encrypt\rsaPublicKey.txt";

// Key container name for
// private/public key value pair.
const string KeyName = "Key01";
```

Create an asymmetric key

This task creates an asymmetric key that encrypts and decrypts the Aes key. This key was used to encrypt the content and it displays the key container name on the label control.

Add the following code as the `Click` event handler for the `Create Keys` button (`buttonCreateAsmKeys_Click`).

C#

```
private void buttonCreateAsmKeys_Click(object sender, EventArgs e)
{
    // Stores a key pair in the key container.
    _cspp.KeyContainerName = KeyName;
```

```
    _rsa = new RSACryptoServiceProvider(_cspp)
    {
        PersistKeyInCsp = true
    };

    label1.Text = _rsa.PublicOnly
        ? $"Key: {_cspp.KeyContainerName} - Public Only"
        : $"Key: {_cspp.KeyContainerName} - Full Key Pair";
}
```

Encrypt a file

This task involves two methods: the event handler method for the `Encrypt File` button (`buttonEncryptFile_Click`) and the `EncryptFile` method. The first method displays a dialog box for selecting a file and passes the file name to the second method, which performs the encryption.

The encrypted content, key, and IV are all saved to one FileStream, which is referred to as the encryption package.

The `EncryptFile` method does the following:

1. Creates a Aes symmetric algorithm to encrypt the content.
2. Creates an RSACryptoServiceProvider object to encrypt the Aes key.
3. Uses a CryptoStream object to read and encrypt the FileStream of the source file, in blocks of bytes, into a destination FileStream object for the encrypted file.
4. Determines the lengths of the encrypted key and IV, and creates byte arrays of their length values.
5. Writes the Key, IV, and their length values to the encrypted package.

The encryption package uses the following format:

- Key length, bytes 0 - 3
- IV length, bytes 4 - 7
- Encrypted key
- IV
- Cipher text

You can use the lengths of the key and IV to determine the starting points and lengths of all parts of the encryption package, which can then be used to decrypt the file.

Add the following code as the `Click` event handler for the `Encrypt File` button (`buttonEncryptFile_Click`).

```
```

C#

```
private void buttonEncryptFile_Click(object sender, EventArgs e)
{
    if (_rsa is null)
    {
        MessageBox.Show("Key not set.");
    }
    else
    {
        // Display a dialog box to select a file to encrypt.
        _encryptOpenFileDialog.InitialDirectory = SrcFolder;
        if (_encryptOpenFileDialog.ShowDialog() == DialogResult.OK)
        {
            string fName = _encryptOpenFileDialog.FileName;
            if (fName != null)
            {
                // Pass the file name without the path.
                EncryptFile(new FileInfo(fName));
            }
        }
    }
}
```

Add the following `EncryptFile` method to the form.

C#

```
private void EncryptFile(FileInfo file)
{
    // Create instance of Aes for
    // symmetric encryption of the data.
    Aes aes = Aes.Create();
    ICryptoTransform transform = aes.CreateEncryptor();

    // Use RSACryptoServiceProvider to
    // encrypt the AES key.
    // rsa is previously instantiated:
    //    rsa = new RSACryptoServiceProvider(cspp);
    byte[] keyEncrypted = _rsa.Encrypt(aes.Key, false);

    // Create byte arrays to contain
    // the length values of the key and IV.
    int lKey = keyEncrypted.Length;
    byte[] LenK = BitConverter.GetBytes(lKey);
    int lIV = aes.IV.Length;
    byte[] LenIV = BitConverter.GetBytes(lIV);

    // Write the following to the FileStream
    // for the encrypted file (outFs):
    // - length of the key
    // - length of the IV
    // - encrypted key
```

```
    // - the IV
    // - the encrypted cipher content

    // Change the file's extension to ".enc"
    string outFile =
        Path.Combine(EncrFolder, Path.ChangeExtension(file.Name, ".enc"));

    using (var outFs = new FileStream(outFile, FileMode.Create))
    {
        outFs.Write(LenK, 0, 4);
        outFs.Write(LenIV, 0, 4);
        outFs.Write(keyEncrypted, 0, lKey);
        outFs.Write(aes.IV, 0, lIV);

        // Now write the cipher text using
        // a CryptoStream for encrypting.
        using (var outStreamEncrypted =
            new CryptoStream(outFs, transform, CryptoStreamMode.Write))
        {
            // By encrypting a chunk at
            // a time, you can save memory
            // and accommodate large files.
            int count = 0;
            int offset = 0;

            // blockSizeBytes can be any arbitrary size.
            int blockSizeBytes = aes.BlockSize / 8;
            byte[] data = new byte[blockSizeBytes];
            int bytesRead = 0;

            using (var inFs = new FileStream(file.FullName, FileMode.Open))
            {
                do
                {
                    count = inFs.Read(data, 0, blockSizeBytes);
                    offset += count;
                    outStreamEncrypted.Write(data, 0, count);
                    bytesRead += blockSizeBytes;
                } while (count > 0);
            }
            outStreamEncrypted.FlushFinalBlock();
        }
    }
}
```

Decrypt a file

This task involves two methods, the event handler method for the `Decrypt File` button (`buttonDecryptFile_Click`), and the `DecryptFile` method. The first method displays a dialog box for selecting a file and passes its file name to the second method, which performs the decryption.

The `Decrypt` method does the following:

1. Creates an Aes symmetric algorithm to decrypt the content.
2. Reads the first eight bytes of the FileStream of the encrypted package into byte arrays to obtain the lengths of the encrypted key and the IV.
3. Extracts the key and IV from the encryption package into byte arrays.
4. Creates an RSACryptoServiceProvider object to decrypt the Aes key.
5. Uses a CryptoStream object to read and decrypt the cipher text section of the FileStream encryption package, in blocks of bytes, into the FileStream object for the decrypted file. When this is finished, the decryption is completed.

Add the following code as the `Click` event handler for the `Decrypt File` button.

C#

```
private void buttonDecryptFile_Click(object sender, EventArgs e)
{
    if (_rsa is null)
    {
        MessageBox.Show("Key not set.");
    }
    else
    {
        // Display a dialog box to select the encrypted file.
        _decryptOpenFileDialog.InitialDirectory = EncrFolder;
        if (_decryptOpenFileDialog.ShowDialog() == DialogResult.OK)
        {
            string fName = _decryptOpenFileDialog.FileName;
            if (fName != null)
            {
                DecryptFile(new FileInfo(fName));
            }
        }
    }
}
```

Add the following `DecryptFile` method to the form.

C#

```
private void DecryptFile(FileInfo file)
{
    // Create instance of Aes for
    // symmetric decryption of the data.
    Aes aes = Aes.Create();

    // Create byte arrays to get the length of
    // the encrypted key and IV.
    // These values were stored as 4 bytes each
```

```
    // at the beginning of the encrypted package.
    byte[] LenK = new byte[4];
    byte[] LenIV = new byte[4];

    // Construct the file name for the decrypted file.
    string outFile =
        Path.ChangeExtension(file.FullName.Replace("Encrypt", "Decrypt"),
".txt");

    // Use FileStream objects to read the encrypted
    // file (inFs) and save the decrypted file (outFs).
    using (var inFs = new FileStream(file.FullName, FileMode.Open))
    {
        inFs.Seek(0, SeekOrigin.Begin);
        inFs.Read(LenK, 0, 3);
        inFs.Seek(4, SeekOrigin.Begin);
        inFs.Read(LenIV, 0, 3);

        // Convert the lengths to integer values.
        int lenK = BitConverter.ToInt32(LenK, 0);
        int lenIV = BitConverter.ToInt32(LenIV, 0);

        // Determine the start position of
        // the cipher text (startC)
        // and its length(lenC).
        int startC = lenK + lenIV + 8;
        int lenC = (int)inFs.Length - startC;

        // Create the byte arrays for
        // the encrypted Aes key,
        // the IV, and the cipher text.
        byte[] KeyEncrypted = new byte[lenK];
        byte[] IV = new byte[lenIV];

        // Extract the key and IV
        // starting from index 8
        // after the length values.
        inFs.Seek(8, SeekOrigin.Begin);
        inFs.Read(KeyEncrypted, 0, lenK);
        inFs.Seek(8 + lenK, SeekOrigin.Begin);
        inFs.Read(IV, 0, lenIV);

        Directory.CreateDirectory(DecrFolder);
        // Use RSACryptoServiceProvider
        // to decrypt the AES key.
        byte[] KeyDecrypted = _rsa.Decrypt(KeyEncrypted, false);

        // Decrypt the key.
        ICryptoTransform transform = aes.CreateDecryptor(KeyDecrypted, IV);

        // Decrypt the cipher text from
        // from the FileSteam of the encrypted
        // file (inFs) into the FileStream
        // for the decrypted file (outFs).
        using (var outFs = new FileStream(outFile, FileMode.Create))
```

```
        {
            int count = 0;
            int offset = 0;

            // blockSizeBytes can be any arbitrary size.
            int blockSizeBytes = aes.BlockSize / 8;
            byte[] data = new byte[blockSizeBytes];

            // By decrypting a chunk a time,
            // you can save memory and
            // accommodate large files.

            // Start at the beginning
            // of the cipher text.
            inFs.Seek(startC, SeekOrigin.Begin);
            using (var outStreamDecrypted =
                new CryptoStream(outFs, transform, CryptoStreamMode.Write))
            {
                do
                {
                    count = inFs.Read(data, 0, blockSizeBytes);
                    offset += count;
                    outStreamDecrypted.Write(data, 0, count);
                } while (count > 0);

                outStreamDecrypted.FlushFinalBlock();
            }
        }
    }
}
```

Export a public key

This task saves the key created by the `Create Keys` button to a file. It exports only the public parameters.

This task simulates the scenario of Alice giving Bob her public key so that he can encrypt files for her. He and others who have that public key will not be able to decrypt them because they do not have the full key pair with private parameters.

Add the following code as the `Click` event handler for the `Export Public Key` button (`buttonExportPublicKey_Click`).

C#

```
void buttonExportPublicKey_Click(object sender, EventArgs e)
{
    // Save the public key created by the RSA
    // to a file. Caution, persisting the
```

```
    // key to a file is a security risk.
    Directory.CreateDirectory(EncrFolder);
    using (var sw = new StreamWriter(PubKeyFile, false))
    {
        sw.Write(_rsa.ToXmlString(false));
    }
}
```

Import a public key

This task loads the key with only public parameters, as created by the `Export Public Key` button, and sets it as the key container name.

This task simulates the scenario of Bob loading Alice's key with only public parameters so he can encrypt files for her.

Add the following code as the `Click` event handler for the `Import Public Key` button (`buttonImportPublicKey_Click`).

C#

```
void buttonImportPublicKey_Click(object sender, EventArgs e)
{
    using (var sr = new StreamReader(PubKeyFile))
    {
        _cspp.KeyContainerName = KeyName;
        _rsa = new RSACryptoServiceProvider(_cspp);

        string keytxt = sr.ReadToEnd();
        _rsa.FromXmlString(keytxt);
        _rsa.PersistKeyInCsp = true;

        label1.Text = _rsa.PublicOnly
            ? $"Key: {_cspp.KeyContainerName} - Public Only"
            : $"Key: {_cspp.KeyContainerName} - Full Key Pair";
    }
}
```

Get a private key

This task sets the key container name to the name of the key created by using the `Create Keys` button. The key container will contain the full key pair with private parameters.

This task simulates the scenario of Alice using her private key to decrypt files encrypted by Bob.

Add the following code as the `Click` event handler for the `Get Private Key` button (`buttonGetPrivateKey_Click`).

C#

```
private void buttonGetPrivateKey_Click(object sender, EventArgs e)
{
    _cspp.KeyContainerName = KeyName;
    _rsa = new RSACryptoServiceProvider(_cspp)
    {
        PersistKeyInCsp = true
    };

    label1.Text = _rsa.PublicOnly
        ? $"Key: {_cspp.KeyContainerName} - Public Only"
        : $"Key: {_cspp.KeyContainerName} - Full Key Pair";
}
```

Test the application

After you have built the application, perform the following testing scenarios.

To create keys, encrypt, and decrypt

1. Click the `Create Keys` button. The label displays the key name and shows that it is a full key pair.
2. Click the `Export Public Key` button. Note that exporting the public key parameters does not change the current key.
3. Click the `Encrypt File` button and select a file.
4. Click the `Decrypt File` button and select the file just encrypted.
5. Examine the file just decrypted.
6. Close the application and restart it to test retrieving persisted key containers in the next scenario.

To encrypt using the public key

1. Click the `Import Public Key` button. The label displays the key name and shows that it is public only.
2. Click the `Encrypt File` button and select a file.
3. Click the `Decrypt File` button and select the file just encrypted. This will fail because you must have the private key to decrypt.

This scenario demonstrates having only the public key to encrypt a file for another person. Typically that person would give you only the public key and withhold the private key for decryption.

To decrypt using the private key

1. Click the `Get Private Key` button. The label displays the key name and shows whether it is the full key pair.
2. Click the `Decrypt File` button and select the file just encrypted. This will be successful because you have the full key pair to decrypt.

See also

- Cryptography Model - Describes how cryptography is implemented in the base class library.
- Cryptographic Services
- Cross-Platform Cryptography
- ASP.NET Core Data Protection

Secure coding guidelines

Article • 09/15/2021

Most application code can simply use the infrastructure implemented by .NET. In some cases, additional application-specific security is required, built either by extending the security system or by using new ad hoc methods.

Using .NET enforced permissions and other enforcement in your code, you should erect barriers to prevent malicious code from accessing information that you don't want it to have or performing other undesirable actions. Additionally, you must strike a balance between security and usability in all the expected scenarios using trusted code.

This overview describes the different ways code can be designed to work with the security system.

Securing resource access

When designing and writing your code, you need to protect and limit the access that code has to resources, especially when using or invoking code of unknown origin. So, keep in mind the following techniques to ensure your code is secure:

- Do not use Code Access Security (CAS).
- Do not use partial trusted code.
- Do not use the AllowPartiallyTrustedCaller attribute (APTCA).
- Do not use .NET Remoting.
- Do not use Distributed Component Object Model (DCOM).
- Do not use binary formatters.

Code Access Security and Security-Transparent Code are not supported as a security boundary with partially trusted code. We advise against loading and executing code of unknown origins without putting alternative security measures in place. The alternative security measures are:

- Virtualization
- AppContainers
- Operating system (OS) users and permissions

- Hyper-V containers

Security-neutral code

Security-neutral code does nothing explicit with the security system. It runs with whatever permissions it receives. Although applications that fail to catch security exceptions associated with protected operations (such as using files, networking, and so on) can result in an unhandled exception, security-neutral code still takes advantage of the security technologies in .NET.

A security-neutral library has special characteristics that you should understand. Suppose your library provides API elements that use files or call unmanaged code. If your code doesn't have the corresponding permission, it won't run as described. However, even if the code has the permission, any application code that calls it must have the same permission in order to work. If the calling code doesn't have the right permission, a SecurityException appears as a result of the code access security stack walk.

Application code that isn't a reusable component

If your code is part of an application that won't be called by other code, security is simple and special coding might not be required. However, remember that malicious code can call your code. While code access security might stop malicious code from accessing resources, such code could still read values of your fields or properties that might contain sensitive information.

Additionally, if your code accepts user input from the Internet or other unreliable sources, you must be careful about malicious input.

Managed wrapper to native code implementation

Typically in this scenario, some useful functionality is implemented in native code that you want to make available to managed code. Managed wrappers are easy to write using either platform invoke or COM interop. However, if you do this, callers of your wrappers must have unmanaged code rights in order to succeed. Under default policy, this means that code downloaded from an intranet or the Internet won't work with the wrappers.

Instead of giving unmanaged code rights to all applications that use these wrappers, it's better to give these rights only to the wrapper code. If the underlying functionality exposes no resources and the implementation is likewise safe, the wrapper only needs to assert its rights, which enables any code to call through it. When resources are involved, security coding should be the same as the library code case described in the next section. Because the wrapper is potentially exposing callers to these resources, careful verification of the safety of the native code is necessary and is the wrapper's responsibility.

Library code that exposes protected resources

The following approach is the most powerful and hence potentially dangerous (if done incorrectly) for security coding: your library serves as an interface for other code to access certain resources that aren't otherwise available, just as the .NET classes enforce permissions for the resources they use. Wherever you expose a resource, your code must first demand the permission appropriate to the resource (that is, it must perform a security check) and then typically assert its rights to perform the actual operation.

See also

- Securing State Data
- Security and User Input
- Security and Race Conditions
- Security and On-the-Fly Code Generation
- Role-Based Security
- ASP.NET Core Security

Securing State Data

Article • 09/15/2021

Applications that handle sensitive data or make any kind of security decisions need to keep that data under their own control and cannot allow other potentially malicious code to access the data directly. The best way to protect data in memory is to declare the data as private or internal (with scope limited to the same assembly) variables. However, even this data is subject to access you should be aware of:

- Using reflection mechanisms, highly trusted code that can reference your object can get and set private members.
- Using serialization, highly trusted code can effectively get and set private members if it can access the corresponding data in the serialized form of the object.
- Under debugging, this data can be read.

Make sure none of your own methods or properties exposes these values unintentionally.

See also

- Secure Coding Guidelines
- ASP.NET Core Security

Security and User Input

Article • 09/15/2021

User data, which is any kind of input (data from a Web request or URL, input to controls of a Microsoft Windows Forms application, and so on), can adversely influence code because often that data is used directly as parameters to call other code. This situation is analogous to malicious code calling your code with strange parameters, and the same precautions should be taken. User input is actually harder to make safe because there is no stack frame to trace the presence of the potentially untrusted data.

These are among the subtlest and hardest security bugs to find because, although they can exist in code that is seemingly unrelated to security, they are a gateway to pass bad data through to other code. To look for these bugs, follow any kind of input data, imagine what the range of possible values might be, and consider whether the code seeing this data can handle all those cases. You can fix these bugs through range checking and rejecting any input the code cannot handle.

Some important considerations involving user data include the following:

- Any user data in a server response runs in the context of the server's site on the client. If your Web server takes user data and inserts it into the returned Web page, it might, for example, include a **<script>** tag and run as if from the server.
- Remember that the client can request any URL.
- Consider tricky or invalid paths:
  - ..\ , extremely long paths.
  - Use of wild card characters (*).
  - Token expansion (%token%).
  - Strange forms of paths with special meaning.
  - Alternate file system stream names such as `filename::$DATA`.
  - Short versions of file names such as `longfi~1` for `longfilename`.
- Remember that Eval(userdata) can do anything.
- Be wary of late binding to a name that includes some user data.
- If you are dealing with Web data, consider the various forms of escapes that are permissible, including:

    - Hexadecimal escapes (%nn).
    - Unicode escapes (%nnn).
    - Overlong UTF-8 escapes (%nn%nn).
    - Double escapes (%nn becomes %mmnn, where %mm is the escape for '%').
- Be wary of user names that might have more than one canonical format. For example, you can often use either the MYDOMAIN*username* form or the *username*@mydomain.example.com form.

See also

- Secure Coding Guidelines
- ASP.NET Core Security

Security and Race Conditions

Article • 09/15/2021

Another area of concern is the potential for security holes exploited by race conditions. There are several ways in which this might happen. The subtopics that follow outline some of the major pitfalls that the developer must avoid.

Race Conditions in the Dispose Method

If a class's **Dispose** method (for more information, see Garbage Collection) is not synchronized, it is possible that cleanup code inside **Dispose** can be run more than once, as shown in the following example.

C#

```
void Dispose()
{
    if (myObj != null)
    {
        Cleanup(myObj);
        myObj = null;
    }
}
```

Because this **Dispose** implementation is not synchronized, it is possible for `Cleanup` to be called by first one thread and then a second thread before `_myObj` is set to **null**. Whether this is a security concern depends on what happens when the `Cleanup` code runs. A major issue with unsynchronized **Dispose** implementations involves the use of resource handles such as files. Improper disposal can cause the wrong handle to be used, which often leads to security vulnerabilities.

Race Conditions in Constructors

In some applications, it might be possible for other threads to access class members before their class constructors have completely run. You should review all class constructors to make sure that there are no security issues if this should happen, or synchronize threads if necessary.

Race Conditions with Cached Objects

Code that caches security information or uses the code access security Assert operation might also be vulnerable to race conditions if other parts of the class are not appropriately synchronized, as shown in the following example.

C#

```
void SomeSecureFunction()
{
    if (SomeDemandPasses())
    {
        fCallersOk = true;
        DoOtherWork();
        fCallersOk = false;
    }
}
void DoOtherWork()
{
    if (fCallersOK)
    {
        DoSomethingTrusted();
    }
    else
    {
        DemandSomething();
        DoSomethingTrusted();
    }
}
```

If there are other paths to `DoOtherWork` that can be called from another thread with the same object, an untrusted caller can slip past a demand.

If your code caches security information, make sure that you review it for this vulnerability.

Race Conditions in Finalizers

Race conditions can also occur in an object that references a static or unmanaged resource that it then frees in its finalizer. If multiple objects share a resource that is manipulated in a class's finalizer, the objects must synchronize all access to that resource.

See also

- Secure Coding Guidelines
- ASP.NET Core Security

Security and On-the-Fly Code Generation

Article • 09/15/2021

Some libraries operate by generating code and running it to perform some operation for the caller. The basic problem is generating code on behalf of lesser-trust code and running it at a higher trust. The problem worsens when the caller can influence code generation, so you must ensure that only code you consider safe is generated.

You need to know exactly what code you are generating at all times. This means that you must have strict controls on any values that you get from a user, be they quote-enclosed strings (which should be escaped so they cannot include unexpected code elements), identifiers (which should be checked to verify that they are valid identifiers), or anything else. Identifiers can be dangerous because a compiled assembly can be modified so that its identifiers contain strange characters, which will probably break it (although this is rarely a security vulnerability).

It is recommended that you generate code with reflection emit, which often helps you avoid many of these problems.

When you compile the code, consider whether there is some way a malicious program could modify it. Is there a small window of time during which malicious code can change source code on disk before the compiler reads it or before your code loads the .dll file? If so, you must protect the directory containing these files, using an Access Control List in the file system, as appropriate.

See also

- Secure Coding Guidelines
- ASP.NET Core Security

Timing vulnerabilities with CBC-mode symmetric decryption using padding

Article • 09/08/2022

Microsoft believes that it's no longer safe to decrypt data encrypted with the Cipher-Block-Chaining (CBC) mode of symmetric encryption when verifiable padding has been applied without first ensuring the integrity of the ciphertext, except for very specific circumstances. This judgement is based on currently known cryptographic research.

Introduction

A padding oracle attack is a type of attack against encrypted data that allows the attacker to decrypt the contents of the data, without knowing the key.

An oracle refers to a "tell" which gives an attacker information about whether the action they're executing is correct or not. Imagine playing a board or card game with a child. When their face lights up with a big smile because they think they're about to make a good move, that's an oracle. You, as the opponent, can use this oracle to plan your next move appropriately.

Padding is a specific cryptographic term. Some ciphers, which are the algorithms used to encrypt your data, work on blocks of data where each block is a fixed size. If the data you want to encrypt isn't the right size to fill the blocks, your data is padded until it does. Many forms of padding require that padding to always be present, even if the original input was of the right size. This allows the padding to always be safely removed upon decryption.

Putting the two things together, a software implementation with a padding oracle reveals whether decrypted data has valid padding. The oracle could be something as simple as returning a value that says "Invalid padding" or something more complicated like taking a measurably different time to process a valid block as opposed to an invalid block.

Block-based ciphers have another property, called the mode, which determines the relationship of data in the first block to the data in the second block, and so on. One of the most commonly used modes is CBC. CBC introduces an initial random block, known as the Initialization Vector (IV), and combines the previous block with the result of static encryption to make it such that encrypting the same message with the same key doesn't always produce the same encrypted output.

An attacker can use a padding oracle, in combination with how CBC data is structured, to send slightly changed messages to the code that exposes the oracle, and keep sending data until the oracle tells them the data is correct. From this response, the attacker can decrypt the message byte by byte.

Modern computer networks are of such high quality that an attacker can detect very small (less than 0.1 ms) differences in execution time on remote systems. Applications that are assuming that a successful decryption can only happen when the data wasn't tampered with may be vulnerable to attack from tools that are designed to observe differences in successful and unsuccessful decryption. While this timing difference may be more significant in some languages or libraries than others, it's now believed that this is a practical threat for all languages and libraries when the application's response to failure is taken into account.

This attack relies on the ability to change the encrypted data and test the result with the oracle. The only way to fully mitigate the attack is to detect changes to the encrypted data and refuse to perform any actions on it. The standard way to do this is to create a signature for the data and validate that signature before any operations are performed. The signature must be verifiable, it cannot be created by the attacker, otherwise they'd change the encrypted data, then compute a new signature based on the changed data. One common type of appropriate signature is known as a keyed-hash message authentication code (HMAC). An HMAC differs from a checksum in that it takes a secret key, known only to the person producing the HMAC and to the person validating it. Without possession of the key, you can't produce a correct HMAC. When you receive your data, you'd take the encrypted data, independently compute the HMAC using the secret key you and the sender share, then compare the HMAC they sent against the one you computed. This comparison must be constant time, otherwise you've added another detectable oracle, allowing a different type of attack.

In summary, to use padded CBC block ciphers safely, you must combine them with an HMAC (or another data integrity check) that you validate using a constant time comparison before trying to decrypt the data. Since all altered messages take the same amount time to produce a response, the attack is prevented.

Who is vulnerable

This vulnerability applies to both managed and native applications that are performing their own encryption and decryption. This includes, for example:

- An application that encrypts a cookie for later decryption on the server.
- A database application that provides the ability for users to insert data into a table whose columns are later decrypted.

- A data transfer application that relies on encryption using a shared key to protect the data in transit.
- An application that encrypts and decrypts messages "inside" the TLS tunnel.

Note that using TLS alone may not protect you in these scenarios.

A vulnerable application:

- Decrypts data using the CBC cipher mode with a verifiable padding mode, such as PKCS#7 or ANSI X.923.
- Performs the decryption without having performed a data integrity check (via a MAC or an asymmetric digital signature).

This also applies to applications built on top of abstractions over top of these primitives, such as the Cryptographic Message Syntax (PKCS#7/CMS) EnvelopedData structure.

Related areas of concern

Research has led Microsoft to be further concerned about CBC messages that are padded with ISO 10126-equivalent padding when the message has a well-known or predictable footer structure. For example, content prepared under the rules of the W3C XML Encryption Syntax and Processing Recommendation (xmlenc, EncryptedXml). While the W3C guidance to sign the message then encrypt was considered appropriate at the time, Microsoft now recommends always doing encrypt-then-sign.

Application developers should always be mindful of verifying the applicability of an asymmetric signature key, as there's no inherent trust relationship between an asymmetric key and an arbitrary message.

Details

Historically, there has been consensus that it's important to both encrypt and authenticate important data, using means such as HMAC or RSA signatures. However, there has been less clear guidance as to how to sequence the encryption and authentication operations. Due to the vulnerability detailed in this article, Microsoft's guidance is now to always use the "encrypt-then-sign" paradigm. That is, first encrypt data using a symmetric key, then compute a MAC or asymmetric signature over the ciphertext (encrypted data). When decrypting data, perform the reverse. First, confirm the MAC or signature of the ciphertext, then decrypt it.

A class of vulnerabilities known as "padding oracle attacks" have been known to exist for over 10 years. These vulnerabilities allow an attacker to decrypt data encrypted by

symmetric block algorithms, such as AES and 3DES, using no more than 4096 attempts per block of data. These vulnerabilities make use of the fact that block ciphers are most frequently used with verifiable padding data at the end. It was found that if an attacker can tamper with ciphertext and find out whether the tampering caused an error in the format of the padding at the end, the attacker can decrypt the data.

Initially, practical attacks were based on services that would return different error codes based on whether padding was valid, such as the ASP.NET vulnerability MS10-070. However, Microsoft now believes that it's practical to conduct similar attacks using only the differences in timing between processing valid and invalid padding.

Provided that the encryption scheme employs a signature and that the signature verification is performed with a fixed runtime for a given length of data (irrespective of the contents), the data integrity can be verified without emitting any information to an attacker via a side channel. Since the integrity check rejects any tampered messages, the padding oracle threat is mitigated.

Guidance

First and foremost, Microsoft recommends that any data that has confidentiality needs be transmitted over Transport Layer Security (TLS), the successor to Secure Sockets Layer (SSL).

Next, analyze your application to:

- Understand precisely what encryption you're performing and what encryption is being provided by the platforms and APIs you're using.
- Be certain that each usage at each layer of a symmetric block cipher algorithm, such as AES and 3DES, in CBC mode incorporate the use of a secret-keyed data integrity check (an asymmetric signature, an HMAC, or to change the cipher mode to an authenticated encryption (AE) mode such as GCM or CCM).

Based on the current research, it's generally believed that when the authentication and encryption steps are performed independently for non-AE modes of encryption, authenticating the ciphertext (encrypt-then-sign) is the best general option. However, there's no one-size-fits-all correct answer to cryptography and this generalization isn't as good as directed advice from a professional cryptographer.

Applications that are unable to change their messaging format but perform unauthenticated CBC decryption are encouraged to try to incorporate mitigations such as:

- Decrypt without allowing the decryptor to verify or remove padding:

    - Any padding that was applied still needs to be removed or ignored, you're moving the burden into your application.
    - The benefit is that the padding verification and removal can be incorporated into other application data verification logic. If the padding verification and data verification can be done in constant time, the threat is reduced.
    - Since the interpretation of the padding changes the perceived message length, there may still be timing information emitted from this approach.
- Change the decryption padding mode to ISO10126:
    - ISO10126 decryption padding is compatible with both PKCS7 encryption padding and ANSIX923 encryption padding.
    - Changing the mode reduces the padding oracle knowledge to 1 byte instead of the entire block. However, if the content has a well-known footer, such as a closing XML element, related attacks can continue to attack the rest of the message.
    - This also doesn't prevent plaintext recovery in situations where the attacker can coerce the same plaintext to be encrypted multiple times with a different message offset.
- Gate the evaluation of a decryption call to dampen the timing signal:
    - The computation of hold time must have a minimum in excess of the maximum amount of time the decryption operation would take for any data segment that contains padding.
    - Time computations should be done according to the guidance in Acquiring high-resolution time stamps, not by using Environment.TickCount (subject to roll-over/overflow) or subtracting two system timestamps (subject to NTP adjustment errors).
    - Time computations must be inclusive of the decryption operation including all potential exceptions in managed or C++ applications, not just padded onto the end.
    - If success or failure has been determined yet, the timing gate needs to return failure when it expires.
- Services that are performing unauthenticated decryption should have monitoring in place to detect that a flood of "invalid" messages has come through.
    - Bear in mind that this signal carries both false positives (legitimately corrupted data) and false negatives (spreading out the attack over a sufficiently long time to evade detection).

Finding vulnerable code - native applications

For programs built against the Windows Cryptography: Next Generation (CNG) library:

- The decryption call is to BCryptDecrypt, specifying the `BCRYPT_BLOCK_PADDING` flag.
- The key handle has been initialized by calling BCryptSetProperty with BCRYPT_CHAINING_MODE set to `BCRYPT_CHAIN_MODE_CBC`.
  - Since `BCRYPT_CHAIN_MODE_CBC` is the default, affected code may not have assigned any value for `BCRYPT_CHAINING_MODE`.

For programs built against the older Windows Cryptographic API:

- The decryption call is to CryptDecrypt with `Final=TRUE`.
- The key handle has been initialized by calling CryptSetKeyParam with KP_MODE set to `CRYPT_MODE_CBC`.
  - Since `CRYPT_MODE_CBC` is the default, affected code may not have assigned any value for `KP_MODE`.

Finding vulnerable code - managed applications

- The decryption call is to the CreateDecryptor() or CreateDecryptor(Byte[], Byte[]) methods on System.Security.Cryptography.SymmetricAlgorithm.
  - This includes the following derived types within the .NET, but may also include third-party types:
    - Aes
    - AesCng
    - AesCryptoServiceProvider
    - AesManaged
    - DES
    - DESCryptoServiceProvider
    - RC2
    - RC2CryptoServiceProvider
    - Rijndael
    - RijndaelManaged
    - TripleDES
    - TripleDESCng
    - TripleDESCryptoServiceProvider
- The SymmetricAlgorithm.Padding property was set to PaddingMode.PKCS7, PaddingMode.ANSIX923, or PaddingMode.ISO10126.
  - Since PaddingMode.PKCS7 is the default, affected code may never have assigned the SymmetricAlgorithm.Padding property.
- The SymmetricAlgorithm.Mode property was set to CipherMode.CBC

- Since CipherMode.CBC is the default, affected code may never have assigned the SymmetricAlgorithm.Mode property.

Finding vulnerable code - cryptographic message syntax

An unauthenticated CMS EnvelopedData message whose encrypted content uses the CBC mode of AES (2.16.840.1.101.3.4.1.2, 2.16.840.1.101.3.4.1.22, 2.16.840.1.101.3.4.1.42), DES (1.3.14.3.2.7), 3DES (1.2.840.113549.3.7) or RC2 (1.2.840.113549.3.2) is vulnerable, as well as messages using any other block cipher algorithms in CBC mode.

While stream ciphers aren't susceptible to this particular vulnerability, Microsoft recommends always authenticating the data over inspecting the ContentEncryptionAlgorithm value.

For managed applications, a CMS EnvelopedData blob can be detected as any value that is passed to System.Security.Cryptography.Pkcs.EnvelopedCms.Decode(Byte[]).

For native applications, a CMS EnvelopedData blob can be detected as any value provided to a CMS handle via CryptMsgUpdate whose resulting CMSG_TYPE_PARAM is `CMSG_ENVELOPED` and/or the CMS handle is later sent a `CMSG_CTRL_DECRYPT` instruction via CryptMsgControl.

Vulnerable code example - managed

This method reads a cookie and decrypts it and no data integrity check is visible. Therefore, the contents of a cookie that is read by this method can be attacked by the user who received it, or by any attacker who has obtained the encrypted cookie value.

C#

```
private byte[] DecryptCookie(string cookieName)
{
    HttpCookie cookie = Request.Cookies[cookieName];

    if (cookie == null)
    {
        return null;
    }

    using (ICryptoTransform decryptor = _aes.CreateDecryptor())
    using (MemoryStream memoryStream = new MemoryStream())
    using (CryptoStream cryptoStream =
        new CryptoStream(memoryStream, decryptor, CryptoStreamMode.Write))
```

```
    {
        byte[] readCookie = Convert.FromBase64String(cookie.Value);
        cryptoStream.Write(readCookie, 0, readCookie.Length);
        cryptoStream.FlushFinalBlock();
        return memoryStream.ToArray();
    }
}
```

Example code following recommended practices - managed

The following sample code uses a non-standard message format of

`cipher_algorithm_id || hmac_algorithm_id || hmac_tag || iv || ciphertext`

where the `cipher_algorithm_id` and `hmac_algorithm_id` algorithm identifiers are application-local (non-standard) representations of those algorithms. These identifiers may make sense in other parts of your existing messaging protocol instead of as a bare concatenated bytestream.

This example also uses a single master key to derive both an encryption key and an HMAC key. This is provided both as a convenience for turning a singly-keyed application into a dual-keyed application, and to encourage keeping the two keys as different values. It further guarantees that the HMAC key and encryption key can't get out of synchronization.

This sample doesn't accept a Stream for either encryption or decryption. The current data format makes one-pass encrypt difficult because the `hmac_tag` value precedes the ciphertext. However, this format was chosen because it keeps all of the fixed-size elements at the beginning to keep the parser simpler. With this data format, one-pass decrypt is possible, though an implementer is cautioned to call GetHashAndReset and verify the result before calling TransformFinalBlock. If streaming encryption is important, then a different AE mode may be required.

C#

```
// ==++==
//
//   Copyright (c) Microsoft Corporation.  All rights reserved.
//
//   Shared under the terms of the Microsoft Public License,
//   https://opensource.org/licenses/MS-PL
//
// ==--==
```

```
using System;
using System.Diagnostics;
using System.IO;
using System.Runtime.CompilerServices;
using System.Security.Cryptography;

namespace Microsoft.Examples.Cryptography
{
    public enum AeCipher : byte
    {
        Unknown,
        Aes256CbcPkcs7,
    }

    public enum AeMac : byte
    {
        Unknown,
        HMACSHA256,
        HMACSHA384,
    }

    /// <summary>
    /// Provides extension methods to make HashAlgorithm look like .NET
Core's
    /// IncrementalHash
    /// </summary>
    internal static class IncrementalHashExtensions
    {
        public static void AppendData(this HashAlgorithm hash, byte[] data)
        {
            hash.TransformBlock(data, 0, data.Length, null, 0);
        }

        public static void AppendData(
            this HashAlgorithm hash,
            byte[] data,
            int offset,
            int length)
        {
            hash.TransformBlock(data, offset, length, null, 0);
        }

        public static byte[] GetHashAndReset(this HashAlgorithm hash)
        {
            hash.TransformFinalBlock(Array.Empty<byte>(), 0, 0);
            return hash.Hash;
        }
    }

    public static partial class AuthenticatedEncryption
    {
        /// <summary>
        /// Use <paramref name="masterKey"/> to derive two keys (one cipher,
one HMAC)
        /// to provide authenticated encryption for <paramref
```

```
name="message"/>.
        /// </summary>
        /// <param name="masterKey">The master key from which other keys
derive.</param>
        /// <param name="message">The message to encrypt</param>
        /// <returns>
        /// A concatenation of
        /// [cipher algorithm+chainmode+padding][mac algorithm][authtag][IV]
[ciphertext],
        /// suitable to be passed to <see cref="Decrypt"/>.
        /// </returns>
        /// <remarks>
        /// <paramref name="masterKey"/> should be a 128-bit (or bigger)
value generated
        /// by a secure random number generator, such as the one returned
from
        /// <see cref="RandomNumberGenerator.Create()"/>.
        /// This implementation chooses to block deficient inputs by length,
but does not
        /// make any attempt at discerning the randomness of the key.
        ///
        /// If the master key is being input by a prompt (like a
password/passphrase)
        /// then it should be properly turned into keying material via a Key
Derivation
        /// Function like PBKDF2, represented by Rfc2898DeriveBytes. A
'password' should
        /// never be simply turned to bytes via an Encoding class and used
as a key.
        /// </remarks>
        public static byte[] Encrypt(byte[] masterKey, byte[] message)
        {
            if (masterKey == null)
                throw new ArgumentNullException(nameof(masterKey));
            if (masterKey.Length < 16)
                throw new ArgumentOutOfRangeException(
                    nameof(masterKey),
                    "Master Key must be at least 128 bits (16 bytes)");
            if (message == null)
                throw new ArgumentNullException(nameof(message));

            // First, choose an encryption scheme.
            AeCipher aeCipher = AeCipher.Aes256CbcPkcs7;

            // Second, choose an authentication (message integrity) scheme.
            //
            // In this example we use the master key length to change from
HMACSHA256 to
            // HMACSHA384, but that is completely arbitrary. This mostly
represents a
            // "cryptographic needs change over time" scenario.
            AeMac aeMac = masterKey.Length < 48 ? AeMac.HMACSHA256 :
AeMac.HMACSHA384;

            // It's good to be able to identify what choices were made when
```

```
a message was
            // encrypted, so that the message can later be decrypted. This
allows for
            // future versions to add support for new encryption schemes,
but still be
            // able to read old data. A practice known as "cryptographic
agility".
            //
            // This is similar in practice to PKCS#7 messaging, but this
uses a
            // private-scoped byte rather than a public-scoped Object
IDentifier (OID).
            // Please note that the scheme in this example adheres to no
particular
            // standard, and is unlikely to survive to a more complete
implementation in
            // the .NET Framework.
            //
            // You may be well-served by prepending a version number byte to
this
            // message, but may want to avoid the value 0x30 (the leading
byte value for
            // DER-encoded structures such as X.509 certificates and PKCS#7
messages).
            byte[] algorithmChoices = { (byte)aeCipher, (byte)aeMac };
            byte[] iv;
            byte[] cipherText;
            byte[] tag;

            // Using our algorithm choices, open an HMAC (as an
authentication tag
            // generator) and a SymmetricAlgorithm which use different keys
each derived
            // from the same master key.
            //
            // A custom implementation may very well have distinctly managed
secret keys
            // for the MAC and cipher, this example merely demonstrates the
master to
            // derived key methodology to encourage key separation from the
MAC and
            // cipher keys.
            using (HMAC tagGenerator = GetMac(aeMac, masterKey))
            {
                using (SymmetricAlgorithm cipher = GetCipher(aeCipher,
masterKey))
                using (ICryptoTransform encryptor =
cipher.CreateEncryptor())
                {
                    // Since no IV was provided, a random one has been
generated
                    // during the call to CreateEncryptor.
                    //
                    // But note that it only does the auto-generation once.
If the cipher
```

```
                    // object were used again, a call to GenerateIV would
have been
                    // required.
                    iv = cipher.IV;

                    cipherText = Transform(encryptor, message, 0,
message.Length);
                }

                // The IV and ciphertext both need to be included in the MAC
to prevent
                // tampering.
                //
                // By including the algorithm identifiers, we have
technically moved from
                // simple Authenticated Encryption (AE) to Authenticated
Encryption with
                // Additional Data (AEAD). By including the algorithm
identifiers in the
                // MAC, it becomes harder for an attacker to change them as
an attempt to
                // perform a downgrade attack.
                //
                // If you've added a data format version field, it can also
be included
                // in the MAC to further inhibit an attacker's options for
confusing the
                // data processor into believing the tampered message is
valid.
                tagGenerator.AppendData(algorithmChoices);
                tagGenerator.AppendData(iv);
                tagGenerator.AppendData(cipherText);
                tag = tagGenerator.GetHashAndReset();
            }

            // Build the final result as the concatenation of everything
except the keys.
            int totalLength =
                algorithmChoices.Length +
                tag.Length +
                iv.Length +
                cipherText.Length;

            byte[] output = new byte[totalLength];
            int outputOffset = 0;

            Append(algorithmChoices, output, ref outputOffset);
            Append(tag, output, ref outputOffset);
            Append(iv, output, ref outputOffset);
            Append(cipherText, output, ref outputOffset);

            Debug.Assert(outputOffset == output.Length);
            return output;
        }
```

```
        /// <summary>
        /// Reads a message produced by <see cref="Encrypt"/> after
verifying it hasn't
        /// been tampered with.
        /// </summary>
        /// <param name="masterKey">The master key from which other keys
derive.</param>
        /// <param name="cipherText">
        /// The output of <see cref="Encrypt"/>: a concatenation of a cipher
ID, mac ID,
        /// authTag, IV, and cipherText.
        /// </param>
        /// <returns>The decrypted content.</returns>
        /// <exception cref="ArgumentNullException">
        /// <paramref name="masterKey"/> is <c>null</c>.
        /// </exception>
        /// <exception cref="ArgumentNullException">
        /// <paramref name="cipherText"/> is <c>null</c>.
        /// </exception>
        /// <exception cref="CryptographicException">
        /// <paramref name="cipherText"/> identifies unknown algorithms, is
not long
        /// enough, fails a data integrity check, or fails to decrypt.
        /// </exception>
        /// <remarks>
        /// <paramref name="masterKey"/> should be a 128-bit (or larger)
value
        /// generated by a secure random number generator, such as the one
returned from
        /// <see cref="RandomNumberGenerator.Create()"/>. This
implementation chooses to
        /// block deficient inputs by length, but doesn't make any attempt
at
        /// discerning the randomness of the key.
        ///
        /// If the master key is being input by a prompt (like a
password/passphrase),
        /// then it should be properly turned into keying material via a Key
Derivation
        /// Function like PBKDF2, represented by Rfc2898DeriveBytes. A
'password' should
        /// never be simply turned to bytes via an Encoding class and used
as a key.
        /// </remarks>
        public static byte[] Decrypt(byte[] masterKey, byte[] cipherText)
        {
            // This example continues the .NET practice of throwing
exceptions for
            // failures. If you consider message tampering to be normal (and
thus
            // "not exceptional") behavior, you may like the signature
            // bool Decrypt(byte[] messageKey, byte[] cipherText, out byte[]
message)
            // better.
            if (masterKey == null)
```

```
                throw new ArgumentNullException(nameof(masterKey));
            if (masterKey.Length < 16)
                throw new ArgumentOutOfRangeException(
                    nameof(masterKey),
                    "Master Key must be at least 128 bits (16 bytes)");
            if (cipherText == null)
                throw new ArgumentNullException(nameof(cipherText));

            // The format of this message is assumed to be public, so
there's no harm in
            // saying ahead of time that the message makes no sense.
            if (cipherText.Length < 2)
            {
                throw new CryptographicException();
            }

            // Use the message algorithm headers to determine what cipher
algorithm and
            // MAC algorithm are going to be used. Since the same Key
Derivation
            // Functions (KDFs) are being used in Decrypt as Encrypt, the
keys are also
            // the same.
            AeCipher aeCipher = (AeCipher)cipherText[0];
            AeMac aeMac = (AeMac)cipherText[1];

            using (SymmetricAlgorithm cipher = GetCipher(aeCipher,
masterKey))
            using (HMAC tagGenerator = GetMac(aeMac, masterKey))
            {
                int blockSizeInBytes = cipher.BlockSize / 8;
                int tagSizeInBytes = tagGenerator.HashSize / 8;
                int headerSizeInBytes = 2;
                int tagOffset = headerSizeInBytes;
                int ivOffset = tagOffset + tagSizeInBytes;
                int cipherTextOffset = ivOffset + blockSizeInBytes;
                int cipherTextLength = cipherText.Length - cipherTextOffset;
                int minLen = cipherTextOffset + blockSizeInBytes;

                // Again, the minimum length is still assumed to be public
knowledge,
                // nothing has leaked out yet. The minimum length couldn't
just be calculated
                // without reading the header.
                if (cipherText.Length < minLen)
                {
                    throw new CryptographicException();
                }

                // It's very important that the MAC be calculated and
verified before
                // proceeding to decrypt the ciphertext, as this prevents
any sort of
                // information leaking out to an attacker.
                //
```

```
                // Don't include the tag in the calculation, though.

                // First, everything before the tag (the cipher and MAC
algorithm ids)
                tagGenerator.AppendData(cipherText, 0, tagOffset);

                // Skip the data before the tag and the tag, then read
everything that
                // remains.
                tagGenerator.AppendData(
                    cipherText,
                    tagOffset + tagSizeInBytes,
                    cipherText.Length - tagSizeInBytes - tagOffset);

                byte[] generatedTag = tagGenerator.GetHashAndReset();

                // The time it took to get to this point has so far been a
function only
                // of the length of the data, or of non-encrypted values
(e.g. it could
                // take longer to prepare the *key* for the HMACSHA384 MAC
than the
                // HMACSHA256 MAC, but the algorithm choice wasn't a
secret).
                //
                // If the verification of the authentication tag aborts as
soon as a
                // difference is found in the byte arrays then your program
may be
                // acting as a timing oracle which helps an attacker to
brute-force the
                // right answer for the MAC. So, it's very important that
every possible
                // "no" (2^256-1 of them for HMACSHA256) be evaluated in as
close to the
                // same amount of time as possible. For this, we call
CryptographicEquals
                if (!CryptographicEquals(
                    generatedTag,
                    0,
                    cipherText,
                    tagOffset,
                    tagSizeInBytes))
                {
                    // Assuming every tampered message (of the same length)
took the same
                    // amount of time to process, we can now safely say
                    // "this data makes no sense" without giving anything
away.
                    throw new CryptographicException();
                }

                // Restore the IV into the symmetricAlgorithm instance.
                byte[] iv = new byte[blockSizeInBytes];
                Buffer.BlockCopy(cipherText, ivOffset, iv, 0, iv.Length);
```

```
                cipher.IV = iv;

                using (ICryptoTransform decryptor =
cipher.CreateDecryptor())
                {
                    return Transform(
                        decryptor,
                        cipherText,
                        cipherTextOffset,
                        cipherTextLength);
                }
            }
        }

        private static byte[] Transform(
            ICryptoTransform transform,
            byte[] input,
            int inputOffset,
            int inputLength)
        {
            // Many of the implementations of ICryptoTransform report true
for
            // CanTransformMultipleBlocks, and when the entire message is
available in
            // one shot this saves on the allocation of the CryptoStream and
the
            // intermediate structures it needs to properly chunk the
message into blocks
            // (since the underlying stream won't always return the number
of bytes
            // needed).
            if (transform.CanTransformMultipleBlocks)
            {
                return transform.TransformFinalBlock(input, inputOffset,
inputLength);
            }

            // If our transform couldn't do multiple blocks at once, let
CryptoStream
            // handle the chunking.
            using (MemoryStream messageStream = new MemoryStream())
            using (CryptoStream cryptoStream =
                new CryptoStream(messageStream, transform,
CryptoStreamMode.Write))
            {
                cryptoStream.Write(input, inputOffset, inputLength);
                cryptoStream.FlushFinalBlock();
                return messageStream.ToArray();
            }
        }

        /// <summary>
        /// Open a properly configured <see cref="SymmetricAlgorithm"/>
conforming to the
        /// scheme identified by <paramref name="aeCipher"/>.
```

```
        /// </summary>
        /// <param name="aeCipher">The cipher mode to open.</param>
        /// <param name="masterKey">The master key from which other keys
derive.</param>
        /// <returns>
        /// A SymmetricAlgorithm object with the right key, cipher mode, and
padding
        /// mode; or <c>null</c> on unknown algorithms.
        /// </returns>
        private static SymmetricAlgorithm GetCipher(AeCipher aeCipher,
byte[] masterKey)
        {
            SymmetricAlgorithm symmetricAlgorithm;

            switch (aeCipher)
            {
                case AeCipher.Aes256CbcPkcs7:
                    symmetricAlgorithm = Aes.Create();
                    // While 256-bit, CBC, and PKCS7 are all the default
values for these
                    // properties, being explicit helps comprehension more
than it hurts
                    // performance.
                    symmetricAlgorithm.KeySize = 256;
                    symmetricAlgorithm.Mode = CipherMode.CBC;
                    symmetricAlgorithm.Padding = PaddingMode.PKCS7;
                    break;
                default:
                    // An algorithm we don't understand
                    throw new CryptographicException();
            }

            // Instead of using the master key directly, derive a key for
our chosen
            // HMAC algorithm based upon the master key.
            //
            // Since none of the symmetric encryption algorithms currently
in .NET
            // support key sizes greater than 256-bit, we can use HMACSHA256
with
            // NIST SP 800-108 5.1 (Counter Mode KDF) to derive a value that
is
            // no smaller than the key size, then Array.Resize to trim it
down as
            // needed.

            using (HMAC hmac = new HMACSHA256(masterKey))
            {
                // i=1, Label=ASCII(cipher)
                byte[] cipherKey = hmac.ComputeHash(
                    new byte[] { 1, 99, 105, 112, 104, 101, 114 });

                // Resize the array to the desired keysize. KeySize is in
bits,
                // and Array.Resize wants the length in bytes.
```

```
                Array.Resize(ref cipherKey, symmetricAlgorithm.KeySize / 8);

                symmetricAlgorithm.Key = cipherKey;
            }

            return symmetricAlgorithm;
        }

        /// <summary>
        /// Open a properly configured <see cref="HMAC"/> conforming to the
scheme
        /// identified by <paramref name="aeMac"/>.
        /// </summary>
        /// <param name="aeMac">The message authentication mode to open.
</param>
        /// <param name="masterKey">The master key from which other keys
derive.</param>
        /// <returns>
        /// An HMAC object with the proper key, or <c>null</c> on unknown
algorithms.
        /// </returns>
        private static HMAC GetMac(AeMac aeMac, byte[] masterKey)
        {
            HMAC hmac;

            switch (aeMac)
            {
                case AeMac.HMACSHA256:
                    hmac = new HMACSHA256();
                    break;
                case AeMac.HMACSHA384:
                    hmac = new HMACSHA384();
                    break;
                default:
                    // An algorithm we don't understand
                    throw new CryptographicException();
            }

            // Instead of using the master key directly, derive a key for
our chosen
            // HMAC algorithm based upon the master key.
            // Since the output size of the HMAC is the same as the ideal
key size for
            // the HMAC, we can use the master key over a fixed input once
to perform
            // NIST SP 800-108 5.1 (Counter Mode KDF):
            hmac.Key = masterKey;

            // i=1, Context=ASCII(MAC)
            byte[] newKey = hmac.ComputeHash(new byte[] { 1, 77, 65, 67 });

            hmac.Key = newKey;
            return hmac;
        }
```

```
        // A simple helper method to ensure that the offset (writePos)
always moves
        // forward with new data.
        private static void Append(byte[] newData, byte[] combinedData, ref
int writePos)
        {
            Buffer.BlockCopy(newData, 0, combinedData, writePos,
newData.Length);
            writePos += newData.Length;
        }

        /// <summary>
        /// Compare the contents of two arrays in an amount of time which is
only
        /// dependent on <paramref name="length"/>.
        /// </summary>
        /// <param name="a">An array to compare to <paramref name="b"/>.
</param>
        /// <param name="aOffset">
        /// The starting position within <paramref name="a"/> for
comparison.
        /// </param>
        /// <param name="b">An array to compare to <paramref name="a"/>.
</param>
        /// <param name="bOffset">
        /// The starting position within <paramref name="b"/> for
comparison.
        /// </param>
        /// <param name="length">
        /// The number of bytes to compare between <paramref name="a"/> and
        /// <paramref name="b"/>.</param>
        /// <returns>
        /// <c>true</c> if both <paramref name="a"/> and <paramref
name="b"/> have
        /// sufficient length for the comparison and all of the applicable
values are the
        /// same in both arrays; <c>false</c> otherwise.
        /// </returns>
        /// <remarks>
        /// An "insufficient data" <c>false</c> response can happen early,
but otherwise
        /// a <c>true</c> or <c>false</c> response take the same amount of
time.
        /// </remarks>
        [MethodImpl(MethodImplOptions.NoInlining |
MethodImplOptions.NoOptimization)]
        private static bool CryptographicEquals(
            byte[] a,
            int aOffset,
            byte[] b,
            int bOffset,
            int length)
        {
            Debug.Assert(a != null);
            Debug.Assert(b != null);
```

```
            Debug.Assert(length >= 0);

            int result = 0;

            if (a.Length - aOffset < length || b.Length - bOffset < length)
            {
                return false;
            }

            unchecked
            {
                for (int i = 0; i < length; i++)
                {
                    // Bitwise-OR of subtraction has been found to have the
most
                    // stable execution time.
                    //
                    // This cannot overflow because bytes are 1 byte in
length, and
                    // result is 4 bytes.
                    // The OR propagates all set bytes, so the differences
are only
                    // present in the lowest byte.
                    result = result | (a[i + aOffset] - b[i + bOffset]);
                }
            }

            return result == 0;
        }
    }
}
```

See also

- Cryptography Model
- Cryptographic Services
- Cross-Platform Cryptography
- ASP.NET Core Data Protection

System.Security.Cryptography.RSACryptoServiceProvider class

Article • 01/09/2024

This article provides supplementary remarks to the reference documentation for this API.

The RSACryptoServiceProvider class is the default implementation of RSA.

The RSACryptoServiceProvider supports key sizes from 384 bits to 16384 bits in increments of 8 bits if you have the Microsoft Enhanced Cryptographic Provider installed. It supports key sizes from 384 bits to 512 bits in increments of 8 bits if you have the Microsoft Base Cryptographic Provider installed.

Valid key sizes are dependent on the cryptographic service provider (CSP) that is used by the RSACryptoServiceProvider instance. Windows CSPs enable keys sizes of 384 to 16384 bits for Windows versions prior to Windows 8.1, and key sizes of 512 to 16384 bits for Windows 8.1. For more information, see CryptGenKey function in the Windows documentation.

Interoperation with the Microsoft Cryptographic API (CAPI)

Unlike the RSA implementation in unmanaged CAPI, the RSACryptoServiceProvider class reverses the order of an encrypted array of bytes after encryption and before decryption. By default, data encrypted by the RSACryptoServiceProvider class cannot be decrypted by the CAPI `CryptDecrypt` function and data encrypted by the CAPI `CryptEncrypt` method cannot be decrypted by the RSACryptoServiceProvider class.

If you do not compensate for the reverse ordering when interoperating between APIs, the RSACryptoServiceProvider class throws a CryptographicException.

To interoperate with CAPI, you must manually reverse the order of encrypted bytes before the encrypted data interoperates with another API. You can easily reverse the order of a managed byte array by calling the Array.Reverse method.

System.Security.Cryptography.RSAParameters structure

Article • 01/09/2024

This article provides supplementary remarks to the reference documentation for this API.

The RSAParameters structure represents the standard parameters for the RSA algorithm.

The RSA class exposes an ExportParameters method that enables you to retrieve the raw RSA key in the form of an RSAParameters structure.

To understand the contents of this structure, it helps to be familiar with how the RSA algorithm works. The next section discusses the algorithm briefly.

RSA algorithm

To generate a key pair, you start by creating two large prime numbers named p and q. These numbers are multiplied and the result is called n. Because p and q are both prime numbers, the only factors of n are 1, p, q, and n.

If we consider only numbers that are less than n, the count of numbers that are relatively prime to n, that is, have no factors in common with n, equals (p - 1)(q - 1).

Now you choose a number e, which is relatively prime to the value you calculated. The public key is now represented as {e, n}.

To create the private key, you must calculate d, which is a number such that (d)(e) mod (p - 1)(q - 1) = 1. In accordance with the Euclidean algorithm, the private key is now {d, n}.

Encryption of plaintext m to ciphertext c is defined as c = (m ^ e) mod n. Decryption would then be defined as m = (c ^ d) mod n.

Summary of fields

Section A.1.2 of the PKCS #1: RSA Cryptography Standard defines a format for RSA private keys.

The following table summarizes the fields of the RSAParameters structure. The third column provides the corresponding field in section A.1.2 of PKCS #1: RSA Cryptography

Standard .

Expand table

RSAParameters field	Contains	Corresponding PKCS #1 field
D	d, the private exponent	privateExponent
DP	d mod (p - 1)	exponent1
DQ	d mod (q - 1)	exponent2
Exponent	e, the public exponent	publicExponent
InverseQ	(InverseQ)(q) = 1 mod p	coefficient
Modulus	n	modulus
P	p	prime1
Q	q	prime2

The security of RSA derives from the fact that, given the public key { e, n }, it is computationally infeasible to calculate d, either directly or by factoring n into p and q. Therefore, any part of the key related to d, p, or q must be kept secret. If you call ExportParameters and ask for only the public key information, this is why you will receive only Exponent and Modulus. The other fields are available only if you have access to the private key, and you request it.

RSAParameters is not encrypted in any way, so you must be careful when you use it with the private key information. All members of RSAParameters are serialized. If anyone can derive or intercept the private key parameters, the key and all the information encrypted or signed with it are compromised.

System.Security.SecureString class

Article • 01/09/2024

> **Important**
>
> We recommend that you don't use the `SecureString` class for new development on .NET (Core) or when you migrate existing code to .NET (Core). For more information, see SecureString shouldn't be used.

This article provides supplementary remarks to the reference documentation for this API.

SecureString is a string type that provides a measure of security. It tries to avoid storing potentially sensitive strings in process memory as plain text. (For limitations, however, see the How secure is SecureString? section.) The value of an instance of SecureString is automatically protected using a mechanism supported by the underlying platform when the instance is initialized or when the value is modified. Your application can render the instance immutable and prevent further modification by invoking the MakeReadOnly method.

The maximum length of a SecureString instance is 65,536 characters.

> **Important**
>
> This type implements the IDisposable interface. When you have finished using an instance of the type, you should dispose of it either directly or indirectly. To dispose of the type directly, call its Dispose method in a `try`/`catch` block. To dispose of it indirectly, use a language construct such as `using` (in C#) or `Using` (in Visual Basic). For more information, see the "Using an Object that Implements IDisposable" section in the IDisposable interface topic.

The SecureString class and its members are not visible to COM. For more information, see ComVisibleAttribute.

String versus SecureString

An instance of the System.String class is both immutable and, when no longer needed, cannot be programmatically scheduled for garbage collection; that is, the instance is read-only after it is created, and it is not possible to predict when the instance will be

deleted from computer memory. Because System.String instances are immutable, operations that appear to modify an existing instance actually create a copy of it to manipulate. Consequently, if a String object contains sensitive information such as a password, credit card number, or personal data, there is a risk the information could be revealed after it is used because your application cannot delete the data from computer memory.

A SecureString object is similar to a String object in that it has a text value. However, the value of a SecureString object is pinned in memory, may use a protection mechanism, such as encryption, provided by the underlying operating system, can be modified until your application marks it as read-only, and can be deleted from computer memory either by your application calling the Dispose method or by the .NET garbage collector.

For a discussion of the limitations of the SecureString class, see the How secure is SecureString? section.

SecureString operations

The SecureString class includes members that allow you to do the following:

Instantiate a SecureString object You instantiate a SecureString object by calling its parameterless constructor.

Add characters to a SecureString object You can add a single character at a time to a SecureString object by calling its AppendChar or InsertAt method.

> **Important**
>
> A **SecureString** object should never be constructed from a **String**, because the sensitive data is already subject to the memory persistence consequences of the immutable **String** class. The best way to construct a **SecureString** object is from a character-at-a-time unmanaged source, such as the **Console.ReadKey** method.

Remove characters from a SecureString object You can replace an individual character by calling the SetAt method, remove an individual character by calling the RemoveAt method, or remove all characters from the SecureString instance by calling the Clear method.

Make the SecureString object read-only Once you have defined the string that the SecureString object represents, you call its MakeReadOnly method to make the string read-only.

Get information about the SecureString object The SecureString class has only two members that provide information about the string: its Length property, which indicates the number of UTF16-encoded code units in the string; and the IsReadOnly, method, which indicates whether the instance is read-only.

Release the memory allocated to the SecureString instance Because SecureString implements the IDisposable interface, you release its memory by calling the Dispose method.

The SecureString class has no members that inspect, compare, or convert the value of a SecureString. The absence of such members helps protect the value of the instance from accidental or malicious exposure. Use appropriate members of the System.Runtime.InteropServices.Marshal class, such as the SecureStringToBSTR method, to manipulate the value of a SecureString object.

The .NET Class Library commonly uses SecureString instances in the following ways:

- To provide password information to a process by using the ProcessStartInfo structure or by calling an overload of the Process.Start method that has a parameter of type SecureString.
- To provide network password information by calling a NetworkCredential class constructor that has a parameter of type SecureString or by using the NetworkCredential.SecurePassword property.
- To provide password information for SQL Server Authentication by calling the SqlCredential.SqlCredential constructor or retrieving the value of the SqlCredential.Password property.
- To pass a string to unmanaged code. For more information, see the SecureString and interop section.

SecureString and interop

Because the operating system does not directly support SecureString, you must convert the value of the SecureString object to the required string type before passing the string to a native method. The Marshal class has five methods that do this:

- Marshal.SecureStringToBSTR, which converts the SecureString string value to a binary string (BSTR) recognized by COM.
- Marshal.SecureStringToCoTaskMemAnsi and Marshal.SecureStringToGlobalAllocAnsi, which copy the SecureString string value to an ANSI string in unmanaged memory.

- Marshal.SecureStringToCoTaskMemUnicode and Marshal.SecureStringToGlobalAllocUnicode, which copy the SecureString string value to a Unicode string in unmanaged memory.

Each of these methods creates a clear-text string in unmanaged memory. It is the responsibility of the developer to zero out and free that memory as soon as it is no longer needed. Each of the string conversion and memory allocation methods has a corresponding method to zero out and free the allocated memory:

Expand table

Allocation and conversion method	Zero and free method
Marshal.SecureStringToBSTR	Marshal.ZeroFreeBSTR
Marshal.SecureStringToCoTaskMemAnsi	Marshal.ZeroFreeCoTaskMemAnsi
Marshal.SecureStringToCoTaskMemUnicode	Marshal.ZeroFreeCoTaskMemUnicode
Marshal.SecureStringToGlobalAllocAnsi	Marshal.ZeroFreeGlobalAllocAnsi
Marshal.SecureStringToGlobalAllocUnicode	Marshal.ZeroFreeGlobalAllocUnicode

How secure is SecureString?

When created properly, a SecureString instance provides more data protection than a String. When creating a string from a character-at-a-time source, String creates multiple intermediate in memory, whereas SecureString creates just a single instance. Garbage collection of String objects is non-deterministic. In addition, because its memory is not pinned, the garbage collector will make additional copies of String values when moving and compacting memory. In contrast, the memory allocated to a SecureString object is pinned, and that memory can be freed by calling the Dispose method.

Although data stored in a SecureString instance is more secure than data stored in a String instance, there are significant limitations on how secure a SecureString instance is. These include:

Platform

On the Windows operating system, the contents of a SecureString instance's internal character array are encrypted. However, whether because of missing APIs or key management issues, encryption is not available on all platforms. Because of this platform dependency, SecureString does not encrypt the internal storage on non-

Windows platform. Other techniques are used on those platforms to provide additional protection.

Duration

Even if the SecureString implementation is able to take advantage of encryption, the plain text assigned to the SecureString instance may be exposed at various times:

- Because Windows doesn't offer a secure string implementation at the operating system level, .NET still has to convert the secure string value to its plain text representation in order to use it.
- Whenever the value of the secure string is modified by methods such as AppendChar or RemoveAt, it must be decrypted (that is, converted back to plain text), modified, and then encrypted again.
- If the secure string is used in an interop call, it must be converted to an ANSI string, a Unicode string, or a binary string (BSTR). For more information, see the SecureString and interop section.

The time interval for which the SecureString instance's value is exposed is merely shortened in comparison to the String class.

Storage versus usage More generally, the SecureString class defines a storage mechanism for string values that should be protected or kept confidential. However, outside of .NET itself, no usage mechanism supports SecureString. This means that the secure string must be converted to a usable form (typically a clear text form) that can be recognized by its target, and that decryption and conversion must occur in user space.

Overall, SecureString is more secure than String because it limits the exposure of sensitive string data. However, those strings may still be exposed to any process or operation that has access to raw memory, such as a malicious process running on the host computer, a process dump, or a user-viewable swap file. Instead of using SecureString to protect passwords, the recommended alternative is to use an opaque handle to credentials that are stored outside of the process.

System.Security.Cryptography.Xml.SignedXml class

Article • 01/09/2024

This article provides supplementary remarks to the reference documentation for this API.

The SignedXml class is the .NET implementation of the World Wide Web Consortium (W3C) XML Signature Syntax and Processing Specification, also known as XMLDSIG (XML Digital Signature). XMLDSIG is a standards-based, interoperable way to sign and verify all or part of an XML document or other data that is addressable from a Uniform Resource Identifier (URI).

Use the SignedXml class whenever you need to share signed XML data between applications or organizations in a standard way. Any data signed using this class can be verified by any conforming implementation of the W3C specification for XMLDSIG.

The SignedXml class allows you to create the following three kinds of XML digital signatures:

Expand table

Signature Type	Description
Enveloped signature	The signature is contained within the XML element being signed.
Enveloping signature	The signed XML is contained within the `<Signature>` element.
Internal detached signature	The signature and signed XML are in the same document, but neither element contains the other.

There is also a fourth kind of signature called an external detached signature which is when the data and signature are in separate XML documents. External detached signatures are not supported by the SignedXml class.

Structure of an XML signature

XMLDSIG creates a `<Signature>` element, which contains a digital signature of an XML document or other data that is addressable from a URI. The `<Signature>` element can optionally contain information about where to find a key that will verify the signature and which cryptographic algorithm was used for signing. The basic structure is as follows:

XML

```
<Signature xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
    <SignedInfo>
      <CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-
c14n-20010315"/>
      <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-
sha1"/>
      <Reference URI="">
        <Transforms>
          <Transform Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-
signature"/>
        </Transforms>
        <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
        <DigestValue>Base64EncodedValue==</DigestValue>
      </Reference>
    </SignedInfo>
    <SignatureValue>AnotherBase64EncodedValue===</SignatureValue>
</Signature>
```

The main parts of this structure are:

- The `<CanonicalizationMethod>` element

  Specifies the rules for rewriting the `Signature` element from XML/text into bytes for signature validation. The default value in .NET is http://www.w3.org/TR/2001/REC-xml-c14n-20010315, which identifies a trustworthy algorithm. This element is represented by the SignedInfo.CanonicalizationMethod property.

- The `<SignatureMethod>` element

  Specifies the algorithm used for signature generation and validation, which was applied to the `<Signature>` element to produce the value in `<SignatureValue>`. In the previous example, the value http://www.w3.org/2000/09/xmldsig#rsa-sha1 identifies an RSA PKCS1 SHA-1 signature. Due to collision problems with SHA-1, Microsoft recommends a security model based on SHA-256 or better. This element is represented by the SignatureMethod property.

- The `<SignatureValue>` element

  Specifies the cryptographic signature for the `<Signature>` element. If this signature does not verify, then some portion of the `<Signature>` block was tampered with, and the document is considered invalid. As long as the `<CanonicalizationMethod>` value is trustworthy, this value is highly resistant to tampering. This element is represented by the SignatureValue property.

- The `URI` attribute of the `<Reference>` element

  Specifies a data object using a URI reference. This attribute is represented by the Reference.Uri property.

  - Not specifying the `URI` attribute, that is, setting the Reference.Uri property to `null`, means that the receiving application is expected to know the identity of the object. In most cases, a `null` URI will result in an exception being thrown. Do not use a `null` URI, unless your application is interoperating with a protocol which requires it.

  - Setting the `URI` attribute to an empty string indicates that the root element of the document is being signed, a form of enveloped signature.

  - If the value of `URI` attribute starts with #, then the value must resolve to an element in the current document. This form can be used with any of the supported signature types (enveloped signature, enveloping signature or internal detached signature).

  - Anything else is considered an external resource detached signature and is not supported by the SignedXml class.

- The `<Transforms>` element

  Contains an ordered list of `<Transform>` elements that describe how the signer obtained the data object that was digested. A transform algorithm is similar to the canonicalization method, but instead of rewriting the `<Signature>` element, it rewrites the content identified by the `URI` attribute of the `<Reference>` element. The `<Transforms>` element is represented by the TransformChain class.

  - Each transform algorithm is defined as taking either XML (an XPath node-set) or bytes as input. If the format of the current data differs from the transform input requirements, conversion rules are applied.

  - Each transform algorithm is defined as producing either XML or bytes as the output.

  - If the output of the last transform algorithm is not defined in bytes (or no transforms were specified), then the canonicalization method is used as an implicit transform (even if a different algorithm was specified in the `<CanonicalizationMethod>` element).

  - A value of http://www.w3.org/2000/09/xmldsig#enveloped-signature for the transform algorithm encodes a rule which is interpreted as remove the

`<Signature>` element from the document. Otherwise, a verifier of an enveloped signature will digest the document, including the signature, but the signer would have digested the document before the signature was applied, leading to different answers.

- The `<DigestMethod>` element

  Identifies the digest (cryptographic hash) method to apply on the transformed content identified by the `URI` attribute of the `<Reference>` element. This is represented by the Reference.DigestMethod property.

Choosing a canonicalization method

Unless interoperating with a specification which requires the use of a different value, we recommend that you use the default .NET canonicalization method, which is the XML-C14N 1.0 algorithm, whose value is http://www.w3.org/TR/2001/REC-xml-c14n-20010315. The XML-C14N 1.0 algorithm is required to be supported by all implementations of XMLDSIG, particularly as it is an implicit final transform to apply.

There are versions of canonicalization algorithms which support preserving comments. Comment-preserving canonicalization methods are not recommended because they violate the "sign what is seen" principle. That is, the comments in a `<Signature>` element will not alter the processing logic for how the signature is performed, merely what the signature value is. When combined with a weak signature algorithm, allowing the comments to be included gives an attacker unnecessary freedom to force a hash collision, making a tampered document appear legitimate. In .NET Framework, only built-in canonicalizers are supported by default. To support additional or custom canonicalizers, see the SafeCanonicalizationMethods property. If the document uses a canonicalization method that is not in the collection represented by the SafeCanonicalizationMethods property, then the CheckSignature method will return `false`.

> **Note**
>
> An extremely defensive application can remove any values it does not expect signers to use from the SafeCanonicalizationMethods collection.

Are the reference values safe from tampering?

Yes, the `<Reference>` values are safe from tampering. .NET verifies the `<SignatureValue>` computation before processing any of the `<Reference>` values and their associated transforms, and will abort early to avoid potentially malicious processing instructions.

Choose the elements to sign

We recommend that you use the value of "" for the `URI` attribute (or set the Uri property to an empty string), if possible. This means the whole document is considered for the digest computation, which means the whole document is protected from tampering.

It is very common to see `URI` values in the form of anchors such as #foo, referring to an element whose ID attribute is "foo". Unfortunately, it is easy for this to be tampered with because this includes only the content of the target element, not the context. Abusing this distinction is known as XML Signature Wrapping (XSW).

If your application considers comments to be semantic (which is not common when dealing with XML), then you should use "#xpointer(/)" instead of "", and "#xpointer(id('foo'))" instead of "#foo". The #xpointer versions are interpreted as including comments, while the shortname forms are excluding comments.

If you need to accept documents which are only partially protected and you want to ensure that you are reading the same content that the signature protected, use the GetIdElement method.

Security considerations about the KeyInfo element

The data in the optional `<KeyInfo>` element (that is, the KeyInfo property), which contains a key to validate the signature, should not be trusted.

In particular, when the KeyInfo value represents a bare RSA, DSA or ECDSA public key, the document could have been tampered with, despite the CheckSignature method reporting that the signature is valid. This can happen because the entity doing the tampering just has to generate a new key and re-sign the tampered document with that new key. So, unless your application verifies that the public key is an expected value, the document should be treated as if it were tampered with. This requires that your application examine the public key embedded within the document and verify it against a list of known values for the document context. For example, if the document could be understood to be issued by a known user, you'd check the key against a list of known keys used by that user.

You can also verify the key after processing the document by using the CheckSignatureReturningKey method, instead of using the CheckSignature method. But, for the optimal security, you should verify the key beforehand.

Alternately, consider trying the user's registered public keys, rather than reading what's in the `<KeyInfo>` element.

Security considerations about the X509Data element

The optional `<X509Data>` element is a child of the `<KeyInfo>` element and contains one or more X509 certificates or identifiers for X509 certificates. The data in the `<X509Data>` element should also not be inherently trusted.

When verifying a document with the embedded `<X509Data>` element, .NET verifies only that the data resolves to an X509 certificate whose public key can be successfully used to validate the document signature. Unlike calling the CheckSignature method with the `verifySignatureOnly` parameter set to `false`, no revocation check is performed, no chain trust is checked, and no expiration is verified. Even if your application extracts the certificate itself and passes it to the CheckSignature method with the `verifySignatureOnly` parameter set to `false`, that is still not sufficient validation to prevent document tampering. The certificate still needs to be verified as being appropriate for the document being signed.

Using an embedded signing certificate can provide useful key rotation strategies, whether in the `<X509Data>` section or in the document content. When using this approach an application should extract the certificate manually and perform validation similar to:

- The certificate was issued directly or via a chain by a Certificate Authority (CA) whose public certificate is embedded in the application.

  Using the OS-provided trust list without additional checks, such as a known subject name, is not sufficient to prevent tampering in SignedXml.

- The certificate is verified to have not been expired at the time of document signing (or "now" for near real-time document processing).

- For long-lived certificates issued by a CA which supports revocation, verify the certificate was not revoked.

- The certificate subject is verified as being appropriate to the current document.

Choosing the transform algorithm

If you are interoperating with a specification which has dictated specific values (such as XrML), then you need to follow the specification. If you have an enveloped signature (such as when signing the whole document), then you need to use http://www.w3.org/2000/09/xmldsig#enveloped-signature (represented by the XmlDsigEnvelopedSignatureTransform class). You can specify the implicit XML-C14N transform as well, but it's not necessary. For an enveloping or detached signature, no transforms are required. The implicit XML-C14N transform takes care of everything.

With the security updated introduced by the Microsoft Security Bulletin MS16-035, .NET has restricted what transforms can be used in document verification by default, with untrusted transforms causing CheckSignature to always return `false`. In particular, transforms which require additional input (specified as child elements in the XML) are no longer allowed due to their susceptibility of abuse by malicious users. The W3C advises avoiding the XPath and XSLT transforms, which are the two main transforms affected by these restrictions.

The problem with external references

If an application does not verify that external references seem appropriate for the current context, they can be abused in ways that provide for many security vulnerabilities (including Denial of Service, Distributed Reflection Denial of Service, Information Disclosure, Signature Bypass, and Remote Code Execution). Even if an application were to validate the external reference URI, there would remain a problem of the resource being loaded twice: once when your application reads it, and once when SignedXml reads it. Since there's no guarantee that the application read and document verify steps have the same content, the signature does not provide trustworthiness.

Given the risks of external references, SignedXml will throw an exception when an external reference is encountered. For more information about this issue, see KB article 3148821.